

Site Automation

Operations runbooks: health checks, troubleshooting,
ServiceNow, POA&M and patching - with AAP and, later, ACT

Setup guide for new AAP admins | site-automation 0.6.0 | September 2026

What this is. A step-by-step guide to setting up the site's operations runbooks in Ansible Automation Platform: health checks (system, compliance, MariaDB / MySQL, certificates, patching), troubleshooting, ServiceNow tickets, POA&M status and patching, and later ACT (GenAI). It starts with what each AAP object *is* and walks through every click, saying what each step does, why, and what you should see.

How to read it

New to Ansible or AAP: read chapter 1 first, then do chapter 2 in order. Inventory already in AAP? Chapter 3 replaces the inventory parts of chapter 2. The later chapters are reference: open them when you build a workflow, read a finding, add ACT, change a setting, approve commands or set up the MariaDB / MySQL check.

Rendered from [docs/](#) in the site-automation repository (version 0.6.0). The repository is the source of truth.

Contents

1. Start here: the pieces and how they fit	7
What the runbooks do	7
The ideas behind Ansible, in plain words	8
The AAP objects, and what each one is for	8
How a health check works, start to finish	9
The findings: the one idea that makes everything modular	10
Where things are	10
The order to do things in	11
2. Setting up AAP, step by step	12
Before you start	12
Step 1. Put the repository in your work Git, with your inventory	12
Step 2. Credential types	13
Step 3. Credentials	13
Step 4. Project	14
Step 5. Inventory (filled from the repository)	14
Step 6. The first job template: Health check	15
Step 7. Run it on one host and read the output	16
Step 8. Tune the thresholds	16
Step 9. The other job templates	17
Step 10. ServiceNow	21
Step 11. Workflows and schedules	21
Step 12. Who can do what	22
If something goes wrong	22
3. Using the inventory you already have in AAP	23
The easy way: your settings files, already named after your groups	23
How variables work in AAP (two minutes)	25
Step 1. Look at what you have	25
Step 2. Create the groups the runbooks use	26
Step 3. Settings for every host (inventory variables)	27
Step 4. Settings for particular groups	27
Step 5. Settings for one host (only when one host is different)	28
Step 6. Use your inventory in the job templates	28

Step 7. Check that it worked	29
Which YAML files in the repository do I change?	29
4. ServiceNow: set up, test and verify	30
What you need	30
Step 1. Ask the ServiceNow admins for an API account	30
Step 2. The addresses	31
Step 3. The network path from AAP	32
Step 4. Check from a shell first (optional, fastest)	32
Step 5. The credential in AAP	33
Step 6. Your settings	33
Step 7. The ServiceNow - test ticket template	34
Step 8. Run it: first a dry run, then for real	34
Step 9. Check the ticket in ServiceNow	35
When a step fails	36
Next	36
5. Workflows and schedules	38
How to build any workflow, click by click	38
Adding ACT to a workflow: what "on fail" means, and where site_act goes	39
Linux workflows	40
Windows workflows	46
Linux and Windows together	47
Schedules	48
Things that trip people up	48
6. Dry runs: try anything, change nothing	50
Where to set it	50
What each template does in a dry run	51
Examples	52
What a dry run cannot tell you	52
7. The runbooks: every check and what to do	53
Health check (playbooks/health_check.yml)	53
Troubleshoot (playbooks/troubleshoot.yml)	63
Certificate report (playbooks/cert_report.yml)	64
POA&M status (playbooks/poam_status.yml)	64

ServiceNow tickets (playbooks/servicenow_tickets.yml)	64
ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml)	64
ServiceNow health (playbooks/servicenow_health.yml)	65
Patch hosts (playbooks/patch_hosts.yml)	65
Apply approved ACT fix (playbooks/act_fix_approved.yml)	66
Windows hosts	66
Not built yet (and why)	66
8. Adding ACT (GenAI) later	67
How they fit: checks find WHAT, ACT finds WHY	67
The three levels	67
Turn it on	68
Both endpoint formats and :probe (ACT 0.6.19 and newer)	70
Write your own check (it works with the report, tickets and ACT automatically)	72
Use ACT in a playbook of your own	73
What ACT is told, and what it cannot do	74
9. Windows servers	75
Step 1. WinRM over HTTPS on the servers	76
Step 2. The execution environment	76
Step 3. The credential	77
Step 4. The Windows inventory	77
Step 5. Test the connection	77
Step 6. The job templates	78
Step 7. Workflows and schedules	79
Start small: the pilot order	79
Patching through Software Center	79
ACT on the servers (C:\ProgramData\act)	81
Settings: what each check looks at, and how to change it	81
If something goes wrong	83
Not in this release (next)	83
10. Secrets: where they live	84
Recommendation	84
Where each secret lives	85
Rules for every playbook in this repository	86

The real trust boundary	87
Keeping secrets out of Git	87
Rotation checklist	88
11. Settings (variables): what, where, and who wins	89
The two-minute version	89
1. What you can set	89
2. Where do I put a setting?	91
3. Who wins when the same setting is in two places?	92
Examples	94
How to verify a setting is in effect	94
If a setting does nothing	95
12. Approved commands: what ACT may run by itself	96
Read this first: two safety rules that you cannot switch off	96
How a command becomes "approved": the three ways	96
Add an approved command, step by step	97
How to verify it worked	99
Reference: what the guard rails are	100
13. The MariaDB and MySQL check, step by step	102
The short version	102
1. Which hosts the check runs on	102
2. Tell it where MariaDB is	103
3. Where to put the settings	103
4. The login: the "MariaDB monitor" credential	104
5. What it checks, and the limits you can change	105
How to verify	106
What was tested, and what was not	107
If it does not work	107
Appendix: credential types to paste into AAP	108
ServiceNow API (servicenow_api.yml)	108
MariaDB monitor (mariadb_monitor.yml)	108
Keystore password (keystore_password.yml)	109
STIG Manager API (stigman_api.yml)	109
ACT model key (act_model_key.yml)	109

ACT GenAI beta key (act_genai_beta_key.yml)	110
STIG Manager database (stigman_database.yml)	110
TLS certificate (tls_certificate.yml)	111
Container registry login (hosts) (registry_login.yml)	111
Teams webhook (teams_webhook.yml)	111

1. Start here: the pieces and how they fit

Confused by all the moving pieces?

Read `HOW_IT_FITS_TOGETHER.md` (or its PDF in [docs/pdf/](#)). It shows what the code, the inventory, the credentials and the survey each do, how a setting travels from a YAML file to a host, and how to update the repository safely.

This repository holds **runbooks**: jobs that check Linux servers and help troubleshoot them. You run them from **Ansible Automation Platform (AAP)**, by hand or on a schedule. This page explains the few ideas you need. `SETUP_AAP.md` then walks you through the setup, one click at a time.

What the runbooks do

Runbook	What it answers	When it runs
Health check	Is anything wrong on these servers right now? Disk, mounts, services, load/memory, time, network, logging, SELinux, fapolicyd, auditd, accounts, certificates, patching, MariaDB / MySQL	Daily (the "daily" set) and weekly (the "weekly" set)
Database health	Are the MariaDB / MySQL databases (on the host or in podman containers) running, with connection room, no stuck replication, and disk space?	Hourly or daily, on its own
Troubleshoot	Someone reports a problem on a host. What does it look like?	On demand
Certificate report	Which certificates expire soon, on every server, in one list?	Weekly
POA&M status	Which POA&M items are overdue or due soon? Which open CAT I/II findings have no POA&M item?	Weekly
ServiceNow tickets	Opens or updates a ticket for each problem a check found. Notes the ones that cleared	After every check (as a workflow step)
ServiceNow health	Is ServiceNow answering? Are the MID Servers up?	Hourly
Patch hosts	Installs updates, and proves every service came back. Reboots only with <code>automatic_restarts: yes</code>	Monthly, after an approval
Apply approved ACT fix	Applies exactly the fix a person approved	Only inside a workflow, after an approval

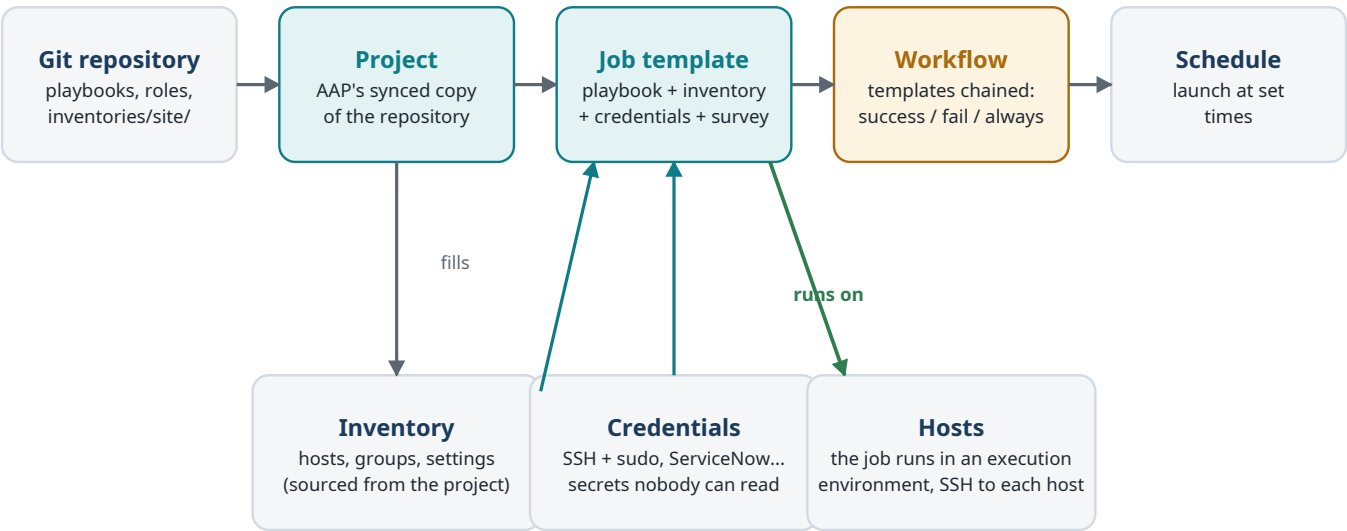
Every check is **read-only**. It looks and reports, and changes nothing. Patching is the only runbook that changes servers, and it waits for an approval first.

The ideas behind Ansible, in plain words

- **Host:** a server Ansible connects to, over SSH, the way you would.
- **Inventory:** the list of hosts, sorted into **groups** such as `stigman` (the STIG Manager servers) or `mariadb`. The built-in group `all` holds every host. A group can also carry **settings (variables)**, for example "on the database hosts, the MariaDB container is called `servicenow-mariadb`".
- **Task:** one step, such as "run `df` and read the result". A **module** is the tool a task uses (`command`, `shell`, `uri`, `set_fact`...).
- **Role:** a folder of tasks that does one job, plus its settings. `roles/check_disk/` is "check the disks". Its settings, with their defaults, are in `roles/check_disk/defaults/main.yml`.
- **Playbook:** the file you run. It says which hosts, and which roles or tasks to run on them. `playbooks/health_check.yml` says "on every host, run the checks I was given".
- **become:** run as root through sudo. The checks need it to read root-only information (the audit log, `/etc/shadow`).
- **Check mode:** a dry run. Ansible shows what it would change and changes nothing. Every runbook here works in check mode, and the checks give the same answers in both modes.
- **Idempotent:** running it twice does no harm. The second run changes nothing.

The AAP objects, and what each one is for

AAP is a web front end that runs playbooks for you. It keeps the passwords, records every run, and runs things on a schedule. You set up a handful of objects once. Each one points at the next:



The AAP objects you create once. The job template is what you launch; a workflow chains templates; a schedule launches either.

AAP object	What it is	Here
Project	A copy of a Git repository that AAP keeps in sync	site-automation (this repository)

AAP object	What it is	Here
Inventory	The hosts and groups, and their settings	<code>Linux servers</code> , filled from <code>inventories/site/</code> in this repository, or the inventory you already have in AAP (USING_YOUR_AAP_INVENTORY.md)
Credential type	A form for a kind of secret, and how to hand it to the job	<code>ServiceNow API</code> , <code>MariaDB monitor</code> , <code>ACT model key...</code> (<code>aap/credential_types/</code>)
Credential	One filled-in form: the actual secret. Nobody can read it back	<code>Linux ssh (sudo)</code> , <code>ServiceNow API...</code>
Execution environment	The container a job runs in (Ansible, Python, tools)	The Minimal execution environment is enough. This repository needs no extra collections
Job template	"Run this playbook, on this inventory, with these credentials" plus the questions to ask (survey). This is what you launch	<code>Health check</code> , <code>Troubleshoot</code> , <code>Patch hosts...</code>
Workflow	Job templates chained together. The next step can depend on the last one's result (success, failure, always). It can stop for an approval	<code>Daily health</code> = Health check, then ServiceNow tickets
Schedule	"Launch this template or workflow at these times"	Daily health every day at 06:00

Three more words you will see:

- **Survey:** the questions a template asks when you launch it, such as "which checks?". Each answer becomes a variable for the playbook.
- **Limit:** run on only some hosts or groups, for example `snowdb01.example.mil` or `mariadb_hosts`.
- **Artifacts:** what a job hands to the next step of a workflow. The health check hands its findings to the ServiceNow step this way (`set_stats` in the playbook).

How a health check works, start to finish

1. You click **Launch** on the *Health check* template (or its schedule fires).
2. AAP syncs the project, starts the execution environment, and connects to each host over SSH with the Machine credential.
3. On each host, each chosen check runs read-only commands and turns what it sees into **findings**. One finding is one thing that is wrong, in plain words. It includes a command you can paste to look further:

```
[CRITICAL] disk: /var is 96% full (critical at 95%), 1.2 GiB free of 40.0 GiB
look: timeout 60 du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -15
```

4. The job prints a summary per host and for the whole fleet. It then **fails the hosts that have findings**. The red status is on purpose: it is how a workflow knows it should open tickets (or ask ACT, or ask for approval).
5. In the *Daily health* workflow, the *ServiceNow tickets* step runs next. It opens a ticket for each finding, or updates the ticket that is already open.

The findings: the one idea that makes everything modular

Every check produces the same kind of finding:

Field	Example	Used by
check	disk	reports, tickets
id	disk:/var	ServiceNow updates the same ticket every run, instead of opening a new one
severity	critical or warning	pass/fail, ticket urgency
summary	/var is 96% full (critical at 95%)	people
hint	du -xh --max-depth=2 /var ...	people, and ACT, which runs it to collect evidence

The report, the ServiceNow tickets and ACT only ever read findings. That is why:

- a new check you write shows up in reports and tickets with no other change;
- ACT works on top of **any** check. Add `use_act=true` and ACT reads the findings and the hint commands' output, then explains the root cause (ADDING_ACT.md).

Where things are

Path	What
playbooks/	the files AAP runs (one per job template)
playbooks/group_vars/	your settings , one file per AAP group: edit these (VARIABLES.md, USING_YOUR_AAP_INVENTORY.md)
roles/check_*/	one folder per check. Its settings are in <code>defaults/main.yml</code>
roles/site_findings/	starts, reports and publishes findings, and decides pass/fail
roles/servicenow/, roles/troubleshoot/, roles/patch/, roles/poam/	the other runbooks
roles/site_act/	the bridge to ACT (GenAI)
inventories/example/	a sample inventory and settings. Copy it to <code>inventories/site/</code>
aap/credential_types/	the credential types to create in AAP
poam/poam.csv	your POA&M list (for the POA&M runbook)
docs/	this guide, SETUP_AAP.md, USING_YOUR_AAP_INVENTORY.md, VARIABLES.md, RUNBOOKS.md, MARIADB.md, WORKFLOWS_AND_SCHEDULES.md, ADDING_ACT.md, APPROVED_COMMANDS.md; PDFs in <code>docs/pdf/</code>

The order to do things in

1. **Set up and run one health check by hand** (SETUP_AAP.md, steps 1 to 8). Your inventory is already in AAP? Use USING_YOUR_AAP_INVENTORY.md for the inventory part. Read its output. Adjust thresholds in your settings files (VARIABLES.md) until the findings are the ones you care about.
2. **Add the other job templates** (step 9) and try each once.
3. **Add ServiceNow** (step 10), and build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md).
4. **Schedule** the workflows.
5. **Later: add ACT** to the runbooks you trust (ADDING_ACT.md).

2. Setting up AAP, step by step

Every step says **what** you are doing, **why**, the **clicks**, and what you should **see** when it worked. Menu names are from AAP 2.7. Older versions (2.5, 2.6) may word a label or a menu differently: follow the meaning, not the exact word. If you have not yet, read `START_HERE.md` first. It explains the words used here.

Before you start

You need:

- An AAP account that can create projects, inventories, credentials and templates in your organization (an **Organization admin** does it all).
- A **Linux account on the servers** that AAP logs in as, for example `svc_aap`. It needs an SSH key and sudo. With `NOPASSWD` sudo, add it to `check_accounts_nopasswd_allowed`, or the accounts check will report it.
- A **Git repository at work** that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
- For ServiceNow: an **API account** (step 10).

Step 1. Put the repository in your work Git, with your inventory

Already have your inventory in AAP?

Then you do not need `inventories/site/`. Push the repository without it (skip items 1-3 below), skip step 5, and follow `USING_YOUR_AAP_INVENTORY.md` for the groups and settings to add.

What. Copy this repository into your work Git, and create your real inventory in it. (Already pushed an earlier release at work? Update it with `scripts/update-from-release.ps1` in VS Code, as in `HOW_IT_FITS_TOGETHER.md`, "Updating the repository at work". Then skip to step 2.)

Why. AAP reads playbooks from Git, never from your laptop. The inventory lives in the same repository, so host lists and thresholds are versioned and reviewed like code. Every change has a who, a when and a why.

1. Extract the release zip (right-click > **Extract All**). In the extracted folder, copy the folder `inventories\example` and name the copy `inventories\site`.
2. Edit `inventories/site/hosts.yml`. Replace the example host names with yours and keep the group names: the playbooks use them. A host can be in several groups.
3. Edit `inventories/site/group_vars/all.yml`. These settings apply to every host: the LDAP and SIEM servers each host must reach, the account AAP logs in as (`svc_aap`), and the thresholds. Every setting and its default is listed in `VARIABLES_REFERENCE.md`. **Two places can hold a setting:** this inventory file (it becomes AAP's *inventory* Variables box) and the project's own settings files `playbooks/group_vars/*.yml` (`all.yml` is only switched-off placeholders; a few group files, such as `stigman.yml` and `aap.yml`, ship real values). If a setting is in both, the project file wins. Use one place per setting: `VARIABLES.md` explains the order.
4. Put it in your work Git, with VS Code:
 - Create an empty repository on your Git server (for example `site-automation`) and copy its clone URL.
 - In VS Code: **Source Control > Clone Repository** > paste the URL > pick a folder, such as `C:\git\site-automation`.
 - Copy everything from the extracted release folder into that folder.

- In **Source Control**: type a message, click **Commit** (answer Yes to "stage all changes"), then **Sync Changes / Publish Branch**.

The same from a command line: `git add -A, git commit -m "site-automation <version>",` then `git push`.

5. Note the two values AAP needs in step 4: the **clone URL** (the one you pasted above) and the **branch** (shown at the bottom left of VS Code, usually `main`). AAP does not read them from your copy; you type them in once.

You should see the files in your Git server's web page, including `inventories/site/`.

Step 2. Credential types

What. Teach AAP the kinds of secrets these runbooks use, one form per kind.

Why. AAP ships with types for SSH (Machine) and Git (Source Control). The ServiceNow login and the others are custom. A credential type says which fields the form has and how the secret reaches the job. Every type here passes it as an **environment variable** in the execution environment. The playbook must ask for it on purpose, and it never shows in job output or Git.

For each file in `aap/credential_types/` that you need now:

1. **Automation Execution > Infrastructure > Credential Types > Create credential type.**
2. **Name:** the name at the top of the file (for example `ServiceNow API`).
3. **Input configuration:** paste the `inputs:` part of the file, *without* the `inputs:` line.
4. **Injector configuration:** paste the `injectors:` part, *without* the `injectors:` line.
5. Click **Create credential type**.

File	Name	Needed for
<code>servicenow_api.yml</code>	ServiceNow API	tickets, ServiceNow health (step 10)
<code>mariadb_monitor.yml</code>	MariaDB monitor	the MariaDB check on a containerized database (root login refused there); optional on host installs
<code>keystore_password.yml</code>	Keystore password	optional: certs check on a PKCS12 keystore
<code>stigman_api.yml</code>	STIG Manager API	optional: POA&M cross-check with STIG Manager
<code>act_model_key.yml</code>	ACT model key	later, for ACT (ADDING_ACT.md): GenAI.mil and Ask Sage keys
<code>act_genai_beta_key.yml</code>	ACT GenAI beta key	only for ACT with the GenAI.mil beta provider

You should see each new type in the Credential Types list.

Step 3. Credentials

What. Store the actual secrets, once.

Why. This is the only place a password or key is kept. Templates point at the credential by name. Nobody, including you, can read the secret back, and it is never in Git or job output.

Automation Execution > Infrastructure > Credentials > Create credential:

Name	Credential type	Fill in
Linux ssh (sudo)	Machine	Username <code>svc_aap</code> , SSH Private Key , Privilege Escalation Method <code>sudo</code> (plus the sudo password if sudo asks for one)
site-automation git	Source Control	only if the repository is private: user name and token, or an SSH key
ServiceNow API	ServiceNow API	instance URL, API user, password (step 10)
MariaDB monitor	MariaDB monitor	the read-only database account, for MariaDB and MySQL ; needed when the database runs in a container (the ServiceNow database), usually needed for MySQL on a host, optional for MariaDB on a host. See MARIADB.md for the setup and the account to ask the DBA for

You should see the credentials listed, with secret fields shown as **ENCRYPTED**.

Step 4. Project

What. Tell AAP where the repository is.

Why. A project is AAP's copy of the repository. Templates pick their playbook from it.

1. **Automation Execution > Projects > Create project.**
2. **Name** `site-automation`. **Organization:** yours.
3. **Execution environment:** **Minimal execution environment** (`ee-minimal`). These runbooks use only modules that come with Ansible itself, so no extra collections or custom execution environment are needed (useful on a disconnected network).
4. **Source control type** `Git`. **Source control URL:** from step 1. **Source control branch/tag/commit:** `main`. Later, pin a release tag such as `v<version>` (the tags are listed on the repository's Releases page).
Source control credential: `site-automation git` if the repository is private.
5. Tick **Update revision on launch**. Every job then syncs first, so a change you push is used right away.
6. Click **Create project**.

You should see the project's **Last job status** turn **Successful** after a few seconds. If it fails, open the sync job: it is almost always the URL, the branch name, or the credential.

Step 5. Inventory (filled from the repository)

(Skip this step if your inventory is already in AAP: see `USING_YOUR_AAP_INVENTORY.md`.)

What. Create the `Linux servers` inventory and fill it from `inventories/site/hosts.yml`.

Why. Sourcing the inventory from the project brings in hosts, groups and the settings in `group_vars/`, all at once, and keeps them in step with Git. You do not type hosts into AAP by hand.

1. **Automation Execution > Infrastructure > Inventories > Create inventory > Create inventory.** Name `Linux servers`. Click **Create inventory**.
2. Open it > **Sources** tab > **Create source:**
 - **Name** `site inventory from Git`
 - **Source** `Sourced from a Project`

- **Project** `site-automation`
- **Inventory file** `inventories/site/hosts.yml`
- Tick **Overwrite**, **Overwrite variables** and **Update on launch**.

3. Click **Create source**, then **Launch inventory update**. The rocket icon also starts it.

You should see, under **Groups**, `stigman`, `mariadb_hosts`, `patch_hosts`... Under **Hosts** you should see your servers. Open a group: its **Variables** are the ones from `group_vars/<group>.yml`.

Step 6. The first job template: Health check

What. A template that runs `playbooks/health_check.yml`.

Why. A job template is the thing you launch. It fixes which playbook, which inventory and which credentials. Its survey asks which checks to run.

1. **Automation Execution > Templates > Create template > Create job template.**
2. Fill in:

Field	Value	Why
Name	<code>Health check</code>	
Job type	Run	(Check also works: the checks read the same way)
Inventory	<code>Linux servers</code>	
Project	<code>site-automation</code>	
Playbook	<code>playbooks/health_check.yml</code>	
Execution environment	Minimal execution environment	
Credentials	<code>Linux ssh (sudo)</code> (+ <code>MariaDB monitor</code> if you made it)	how AAP logs in
Limit	leave empty, tick Prompt on launch	so you can run it on one host
Privilege escalation	ticked	the checks read root-only files

3. Click **Create job template**.
4. **Survey tab > Create survey question:**
 - **Question** `Which checks?`
 - **Answer variable name** `health_checks`
 - **Answer type** *Multiple Choice (multiple select)*
 - **Choices**, one per line: `daily`, `weekly`, `all`, `disk`, `mounts`, `services`, `performance`, `time`, `network`, `logging`, `selinux`, `fapolicyd`, `auditd`, `accounts`, `certs`, `patching`, `mariadb`, `mysql`, `database`, `containers`
 - **Default answer** `daily`. Tick **Required**. Click **Create question**.

Made this survey with an earlier release? AAP never updates a survey for you: open the **Health check** template > **Survey tab** > click the `health_checks` question > **Edit** > add `containers` on a new line under **Choices** > **Save**. (`daily` and `all` already include the containers check without that; the choice is for running

it on its own.)

5. Turn the survey **on** (the switch at the top of the Survey tab).

You should see the template in the Templates list with a rocket icon.

Step 7. Run it on one host and read the output

What. Launch the health check on a single server.

Why. Start small. You will see what the checks find on a real host, and whether the thresholds suit you.

1. Click the rocket on **Health check**. **Limit:** one host name. Survey: **daily**. Click **Next**, then **Launch**.
2. The job output scrolls by. The parts to read:
 - **Check disk | run, Check services | run...**: each check running (green **ok** lines are normal).
 - **Report | result for this host:** the verdict in plain words. Either **HEALTHY. Checks run: ...** or a list of **[CRITICAL] / [WARNING]** findings. Each finding has a **look:** line: a command you can paste on the host to see more.
 - **Report | fleet summary:** one line for all hosts, for example **3 healthy, 1 with findings**.
 - A red **FAIL this host because it has findings** at the end. This is **on purpose**: a job with findings fails, so a workflow can react (open tickets). It does not mean the playbook broke.
3. The **Details** tab shows who launched it, when, and the exact revision of the repository.

You should see a finished job, green if the host is healthy, red with findings otherwise.

Step 8. Tune the thresholds

What. Make the findings match what you care about.

Why. Defaults are a starting point. If a warning is expected on some host, change the setting for that host or group, not the code.

1. Change the setting in the project's settings file `playbooks/group_vars/<group>.yaml` (every host: `all.yaml`; one host: `playbooks/host_vars/<host>.yaml`, create it). This is the place `VARIABLES.md` recommends, and it wins over the same setting in `inventories/site/group_vars/` or in an AAP group or inventory Variables box. The file name must be the exact group or host name. Examples:

YAML

```
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}      # this filesystem: warn earlier
check_services_ignore_failed: ['^dnf-makecache']
site_fail_on: [critical]                   # warnings are reported but do not fail the job
```

(With `[critical]`, a check that could not run, `<check>:check-error`, still fails the host: a check that did not run is not a healthy result.)

2. Commit and push. `playbooks/group_vars/` is read straight from the project, which syncs on launch: nothing else to do. Only if you changed `inventories/site/group_vars/` instead, run the inventory source again in AAP (**Inventories > Linux servers > Sources > Launch inventory update**): those files reach AAP only through the inventory source.
3. Launch the health check again.

You should see the finding gone, or at its new threshold.

Step 9. The other job templates

Create each the same way as step 6: **Inventory** `Linux servers`, **Project** `site-automation`, **Execution environment** Minimal. For the ones that log in to hosts, tick **Privilege escalation** and leave **Limit** empty with **Prompt on launch** ticked. Only the differences are listed. A survey question is *Multiple Choice (single select)* unless it says otherwise.

Three templates run on the controller, not on hosts:

POA&M status, ServiceNow tickets and ServiceNow health. For these, do **not** tick Prompt on launch for Limit. A workflow's Limit (for example `patch_hosts`) is passed to every step that prompts for one. These playbooks run on `localhost`, so the Limit would match nothing and the step would silently do nothing, and still show green.

Troubleshoot

On demand, when someone reports a problem. Always launch it with a **Limit**: the host with the problem. Findings do not fail this job.

Field	Value
Playbook	<code>playbooks/troubleshoot.yml</code>
Credentials	<code>Linux ssh (sudo)</code>
Survey: <code>ts_area</code>	"What kind of problem?" Choices: <code>overview</code> , <code>disk</code> , <code>performance</code> , <code>service</code> , <code>network</code> , <code>selinux</code> , <code>fail2ban</code> , <code>login</code> , <code>time</code> , <code>logs</code> , <code>mariadb</code> , <code>mysql</code> . Default <code>overview</code>
Survey: <code>ts_service</code>	Text, not required. "Service name (for area = service), e.g. nginx"
Survey: <code>ts_target</code>	Text, not required. "host:port it cannot reach (for area = network)"
Survey: <code>ts_since</code>	Text. "How far back to read logs". Default <code>-2h</code>

Database health - MariaDB / MySQL

The database check on its own, for **MariaDB and MySQL**, installed on the host or in **podman containers** (one, several, found automatically, rootless). Give it to the database team. It only reads: no data is read, nothing is changed. Set the databases up first (MARIADB.md): the container name in a settings file, and the account for the credential.

Field	Value
Name	<code>Database health - MariaDB / MySQL</code>
Playbook	<code>playbooks/database_health.yml</code>
Credentials	<code>Linux ssh (sudo)</code> and <code>MariaDB monitor</code> (the read-only database account)
Privilege escalation	ticked
Job type	Run, with Prompt on launch ticked, so you can pick <i>Check</i> first

Field	Value
Limit	empty, Prompt on launch ticked. The job runs on the groups <code>mariadb_hosts</code> , <code>mysql_hosts</code> , <code>database_hosts</code> , <code>mariadb</code> and <code>mysql</code> ; a Limit narrows that to one host or group. A Limit cannot add a server outside those groups: for that, put <code>target: <group or host></code> in the template's Variables
Survey	none. (Optional: a <i>Multiple Choice</i> <code>use_act</code> , choices <code>no</code> and <code>yes</code> , to add ACT's explanation; then also attach the <i>ACT model key</i> credential, <code>ADDING_ACT.md</code>)

Steps: create the template as in step 6, then launch it twice.

1. Launch with **Job type** `Check`. You should see, for every database host, a `MariaDB | status` line such as `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections ...`, then `HEALTHY` (or a list of findings, each with a command to look further), and at the end `N healthy, M with findings`. *Check* is safe here: the job only reads either way.
2. Launch with **Job type** `Run`. The result is the same. A host with findings shows as failed, so a workflow can open a ticket (workflow 8).

If a host prints `mariadb:connect` with `Access denied`, the credential is missing or the account lacks a privilege: `MARIADB.md`.

Service watch

Two templates and a workflow: the steps are in `SERVICE_WATCH_DEMO.md` (Parts 5 and 6). The one field to decide here is which hosts it watches.

Field	Value
Playbooks	<code>playbooks/service_watch.yml</code> (<i>Service watch - check</i>) and <code>playbooks/service_fix_approved.yml</code> (<i>Service watch - apply approved fix</i>)
Credentials	<code>Linux ssh (sudo)</code> , plus <code>ACT model key</code> on the check template
Variables	optional: <code>target: <group or host></code> . Without it the jobs run on the group <code>stigman</code> - a group inside the template's inventory , not an inventory called <code>stigman</code> (if there is no such group, the job stops and lists the groups the inventory has). To watch the containers of other servers, put that group (or <code>all</code>) here, on both templates. Which containers are watched on each host is found by itself: every container that should be running, root's and each user's (<code>SERVICE_WATCH_DEMO.md</code> , "Which containers are watched")
Survey	on the workflow: <code>watch_mode</code> (<code>approval</code> / <code>self-heal</code>)

Certificate report

Weekly: one table of every certificate on every host, soonest expiry first.

Field	Value
Playbook	<code>playbooks/cert_report.yml</code>
Credentials	<code>Linux ssh (sudo)</code> (+ <code>Keystore password</code> if you use it)
Survey: <code>check_certs_warn_days</code>	<i>Integer</i> . "Warn this many days before expiry". Default <code>30</code>

POA&M status

Weekly. Runs on the controller and reads `poam/poam.csv` from the project.

Field	Value
Playbook	<code>playbooks/poam_status.yml</code>
Credentials	none (+ STIG Manager API for the cross-check)
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey: <code>poam_stigman_api</code>	<i>Text</i> , not required. "STIG Manager API URL (blank = skip the cross-check)"

ServiceNow - test ticket

Run once before the tickets step, and again after a password or certificate change. It opens a test incident, reads it back, adds a work note and resolves it, with PASS / WARN / FAIL for each step (SERVICENOW_SETUP.md). Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_test_ticket.yml</code>
Credentials	ServiceNow API
Job type	Run, with Prompt on launch ticked, so you can pick <i>Check</i> (a dry run: it only logs in and reads)
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

ServiceNow tickets

Only useful as a workflow step after a check. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_tickets.yml</code>
Credentials	ServiceNow API
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

ServiceNow health

Hourly. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_health.yml</code>
Credentials	<code>ServiceNow API</code>
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

Patch hosts

Monthly, inside the *Patch with checks* workflow. Run it as *Check* first: it lists what would be updated and changes nothing.

Field	Value
Playbook	<code>playbooks/patch_hosts.yml</code>
Credentials	<code>Linux ssh (sudo)</code>
Job type	Run, with Prompt on launch ticked, so you can pick <i>Check</i> (a dry run)
Variables	required: <code>target: <group></code> , the name of a group (or host) inside the inventory - not the inventory's own name. For example <code>target: stigman</code> , or <code>target: all</code> when the template's inventory holds only the servers to patch. Without it the job looks for a group called <code>patch_hosts</code> , and stops with a message that lists the groups your inventory does have
Survey: <code>patch_security_only</code>	"Security updates only?" Choices <code>false</code> , <code>true</code> . Default <code>false</code>
Survey: <code>automatic_restarts</code>	"Reboot hosts automatically when the updates need it?" Choices <code>no</code> , <code>yes</code> . Default <code>no</code> (no automatic restarts: the job lists the hosts that need a reboot)

Apply approved ACT fix

Only inside the approval workflow (ADDING_ACT.md). Create it when you add ACT.

Field	Value
Playbook	<code>playbooks/act_fix_approved.yml</code>
Credentials	<code>Linux ssh (sudo)</code> (no ACT key: it runs the approved commands itself)
Job type	Run, with Prompt on launch ticked, so a workflow can run it as <i>Check</i> (a dry run that only shows what it would run)
Variables	leave Prompt on launch unticked
Survey	none

Try each one

With a Limit of one host:

- **Troubleshoot:** `ts_area = disk`. Each command's output appears under a `### what it shows` line.
- **Patch hosts:** Job type **Check**. It lists what would be updated and changes nothing.
- **Certificate report:** read the table under `All certificates on all hosts, soonest expiry first`.

Step 10. ServiceNow

What. An API account, its credential, and a test ticket that proves it works.

Why. The tickets step and the ServiceNow health check use ServiceNow's REST Table API with a user name and password. No plugin or collection is installed.

The whole setup, step by step, is in `SERVICENOW_SETUP.md`: the account to ask for (with a request you can copy), the instance URL and every API call, the network path (firewall, proxy, the DoD CA), the credential, the settings, the test ticket, and how to check the ticket in ServiceNow. In short:

1. Ask the ServiceNow admins for an **integration account** (web service access only) with:
 - `itil` (or a role that can create and update incidents);
 - `mid_server` (read the MID Server list), if you want the MID Server check.
2. Create the **ServiceNow API** credential (step 3) with the instance URL, for example `https://servicenow.example.mil`, and the account.
3. In your settings file (`playbooks/group_vars/all.yml`; the example inventory's `inventories/site/group_vars/all.yml` already has these two lines, so change the value in one place only), set who gets the tickets:

YAML

```
servicenow_assignment_group: Linux Operations # a group that exists in ServiceNow
servicenow_min_severity: warning # or critical: tickets for critical findings only
```

4. Create **ServiceNow - test ticket** (step 9) and launch it, first as *Check*, then as *Run*. **You should see** six steps that say PASS, and a link to the test incident. It is resolved by the same job.
5. Launch **ServiceNow health** once: `API answers in N s`, then the MID Servers and their status.
6. Build the *Daily health* workflow (`WORKFLOWS_AND_SCHEDULES.md`). Its tickets step then runs after every check.

Try it safely first: launch the workflow **without** the ServiceNow API credential on the tickets template. The step then only prints `NOT CONNECTED - what would be opened` and sends nothing.

Step 11. Workflows and schedules

See `WORKFLOWS_AND_SCHEDULES.md`: *Daily health*, *Weekly compliance*, *Patch with checks*, *Fix with approval (ACT)*, and when to schedule each.

Going further, once the basics work:

- ACT (the model): `ADDING_ACT.md`. If a model answers `HTTP 400`, see its section Both endpoint formats and `:probe`.
- Letting ACT run fixes you trust without asking: `APPROVED_COMMANDS.md`.
- Every setting you can change, and where to put it: `VARIABLES.md`.

- MariaDB and MySQL checks (also in podman containers): MARIADB.md.

Step 12. Who can do what

Open a template or workflow › **User Access** (or **Team Access**) › **Add roles**:

Role	Can
Execute	launch it (and answer its survey)
Approve	approve or deny its approval steps (workflows)
Read	see it and its job output
Admin	change it

For example, give the help desk **Execute** on *Troubleshoot*, and only the server team **Execute** on *Patch hosts* and **Approve** on the patch workflow.

If something goes wrong

You see	It means	Do
Project sync fails	wrong URL, branch or Git credential	open the sync job and read the last lines
UNREACHABLE! ... Permission denied (publickey)	SSH key or user wrong for that host	check the Machine credential; test <code>ssh svc_aap@host</code> from a workstation
Missing sudo password / a password is required	sudo wants a password	put it in the Machine credential (Privilege Escalation Password)
Unknown check(s): ...	a survey choice that is not a check name	fix the survey choices (step 6)
the <name> check could not run: ...	that check hit an error. The others still ran	the message says why, often a missing command such as <code>semanage</code>
skipped: the MariaDB/MySQL check is off for <host>	the database check only runs on database hosts	add the host to <code>mariadb_hosts</code> , <code>mysql_hosts</code> or <code>database_hosts</code> in the inventory, or name its container (MARIADB.md)
Findings you do not care about	defaults do not fit that host	step 8: change the setting for its group or host
NOT CONNECTED - what would be opened	the tickets template has no ServiceNow API credential	attach it (step 10)
Tickets step fails: No check results reached this job	the check step before it failed before checking any host, or it was launched on its own	read the first red job in the workflow; run the tickets step only inside a workflow
A controller step (tickets, POA&M, ServiceNow health) shows skipping: no hosts matched	its template prompts for a Limit and the workflow passed one	untick Prompt on launch for Limit on that template (step 9)

3. Using the inventory you already have in AAP

You imported your inventory into AAP. You do not need `inventories/site/` in Git, and you can skip the Git inventory parts of `SETUP_AAP.md` (the `cp inventories/example ...` in step 1, and all of step 5). Your hosts and groups stay in AAP. Your **settings** can go in either of two places, and you can use both as long as **one setting lives in one place**:

- **this project's settings files** in `playbooks/group_vars/` (edited in VS Code, reviewed in Git, never overwritten by an update); the next section; or
- **AAP's Variables boxes** (inventory, group, host): the steps after it. What you paste there is YAML, and the blocks to paste are below.

Which place wins when a setting is in both, and the full list of places, is in `VARIABLES.md`. Every setting and its default: `VARIABLES_REFERENCE.md`.

The runbooks need two things from your inventory:

1. **Groups**: which hosts are which (steps 1-2). The safety lists are built in and already cover the usual group names (`aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`...).
2. **Settings (variables)**: your LDAP server, your SIEM, the account AAP logs in as, the ServiceNow assignment group... (steps 3-5).

The easy way: your settings files, already named after your groups

The repository has one settings file per AAP group, in `playbooks/group_vars/`. **The file name is the group name**: `mariadb.yml` applies to the hosts in AAP group `mariadb`, and `all.yml` to every host. Ansible reads these files by itself on every run, and the hosts and groups stay in AAP. **If your group has another name, copy the file to your group's name** (for example `mariadb.yml` to `mariadb_hosts.yml`, `aap.yml` to `aap_hosts.yml`, `netapp.yml` to `netapp_console_hosts.yml`, which are the names the example inventory uses). A file for a group your inventory does not have is never used, without any error. The files are:

File	For AAP group	What is in it
<code>all.yml</code>	every host (and the ServiceNow / POA&M jobs)	placeholders, all switched off: the safety lists (already built in), ACT provider and models, LDAP/log server, service account, ServiceNow assignment group
<code>stigman.yml</code>	<code>stigman</code>	the containers service watch watches (<code>stigman</code> , <code>nginx</code>); the web certificate
<code>mariadb.yml</code> , <code>mysql.yml</code>	<code>mariadb</code> , <code>mysql</code>	placeholders: turn the database check on, and the container's name (MariaDB or MySQL; <code>MARIADB.md</code>)
<code>aap.yml</code> , <code>sat.yml</code> , <code>idm.yml</code> , <code>logstash.yml</code>	the same names	disk limits, certificates, services that must run
<code>netapp.yml</code>	<code>netapp</code>	nothing yet (the group is protected by the built-in lists)
<code>rhel8_all.yml</code>	<code>rhel8_all</code>	a placeholder for Python 3.9 (only with a newer AAP)

To change a setting:

1. Open the file in VS Code (for example `playbooks\group_vars\mariadb.yml`).
2. A line that starts with `#` is **off**. To turn a placeholder on, delete the `#` in front of it (and in front of the lines that belong to it), and replace `CHANGE-ME` with your value. Keep the indentation exactly as shown: spaces, never tabs.
3. **Commit** and **Sync Changes**. AAP uses it on the next run: the project syncs itself when **Update revision on launch** is ticked.

Example (the full guide, with several containers and rootless ones, is MARIADB.md): MariaDB runs in a container called `snow-mariadb`. In `mariadb.yml`, change

```
check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME
```

into

```
YAML
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
```

A group that has no file yet (`sn_prod` here is only an example name; use one of yours): create `playbooks/group_vars/sn_prod.yml` with the settings for it. The same name as the AAP group, `.yml` at the end. For one host only: `playbooks/host_vars/<host name>.yml` (create the folder).

One thing the files cannot do: choose which hosts a template runs on. That is decided before any setting is read. Each playbook therefore has a built-in choice, and the job template's own **Variables** box (the *Extra variables* on its Details page, not the inventory's) can change it with one `target:` line:

Job templates	Variables box empty: runs on	To run on something else
Health check, Troubleshoot, Certificate report, Apply approved ACT fix	every host in the template's inventory (the built-in group <code>all</code>)	one run: the Limit at launch (tick Prompt on launch next to Limit). Always: <code>target: "rhel8_all:rhel9_all"</code>
Database health - MariaDB / MySQL	the groups <code>mariadb_hosts</code> , <code>mysql_hosts</code> , <code>database_hosts</code> , <code>mariadb</code> , <code>mysql</code>	<code>target: <group or host></code> (a Limit only narrows those groups; it cannot add a server outside them)
Service watch, Service watch - apply approved fix, STIG Manager - deploy	the group <code>stigman</code>	<code>target: <group></code>
Patch hosts	the group <code>patch_hosts</code> only (no such group: the job stops and lists your groups)	always set it , for example <code>target: "rhel8_all:rhel9_all"</code> , and pick the hosts with the Limit at launch. <code>target</code> is a group or host inside the inventory, never the inventory's own name

`target` and the Limit take group names, host names, or several joined with `:`. `stigman:mariadb` means either group, `rhel9_all:&sn_prod` only hosts in both, and `rhel9_all:!sn_dev` all of `rhel9_all` except `sn_dev`. Put a space after `target:`, quote a value that has a `:` in it, and use the group name exactly as the inventory's **Groups** tab shows it (`rhel9_all`, not `rhel9`).

Is every host in this inventory a Linux server? If some are not (firewalls, switches, appliances), an empty Variables box tries them too: they show as failed, and in a workflow that opens ServiceNow tickets each of them gets a ticket (a host that never reports is ticketed), on every run. Then put `target: "rhel8_all:rhel9_all"` on those templates **before** you schedule anything.

These files are yours. The update script adds a settings file you do not have yet, and never changes one you already have. A new release can therefore never overwrite your settings.

Or put the same lines in AAP instead. Everything below explains the AAP Variables boxes, which take exactly the same lines. **Keep each setting in one place.** If the same setting is in both a file here and an AAP *group* or *inventory* box, **the file here wins** (every job prints a note when that happens). The exception is one host: a *host's* Variables box in AAP beats a group file, and only a `playbooks/host_vars/` file beats the host box. Full order: VARIABLES.md.

How variables work in AAP (two minutes)

A variable is a setting with a name, such as `check_disk_warn_pct: 85`. Where you type it decides which hosts get it:

Where in AAP	Applies to	The Git-file equivalent
Inventories › your inventory › Edit inventory › Variables	every host in the inventory	<code>group_vars/all.yml</code>
... › Groups › a group › Edit group › Variables	the hosts in that group (and in its child groups)	<code>group_vars/<group>.yml</code>
... › Hosts › a host › Edit host › Variables	that one host	<code>host_vars/<host>.yml</code>

- **Inside AAP the more specific one wins:** host beats group, a child group beats its parent group, and a group beats the inventory. A survey answer, or the template's own **Variables**, beats all of them. **Files in this project's** `playbooks/group_vars/` **beat the AAP group and inventory boxes** (see the previous section).
- **Anything you do not set uses the default** in `roles/<role>/defaults/main.yml`. Every setting is explained in VARIABLES_REFERENCE.md. Do not edit the role files: set the value in a settings file or in AAP instead. A new release would overwrite edits in the roles, but never your settings files or AAP variables.
- The boxes take YAML. Keep the indentation exactly as shown (spaces, not tabs). AAP refuses to save YAML it cannot read.

Step 1. Look at what you have

Inventories › **your inventory**: open the **Groups** tab, then the **Hosts** tab. Write down which of your groups (or hosts) are:

- all your Linux servers;
- the AAP server(s);
- the NetApp Console host;
- the STIG Manager host(s);
- the ServiceNow database host(s);

- the ServiceNow MID Server host(s);
- the servers monthly patching may touch.

On the old ansible-core machine, `ansible-inventory -i <your inventory> --graph` prints the same tree.

Step 2. Create the groups the runbooks use

A host can be in many groups. Creating these groups leaves your existing groups as they are.

Create group	Put in it	Why	Needed?
<code>aap_hosts</code>	the AAP server(s)	safety: ACT never fixes anything here by itself, and patching never reboots it	yes
<code>netapp_console_hosts</code>	the NetApp Console host	safety: ACT only diagnoses here (vendor-managed)	yes , if you have one
<code>no_patch</code>	the groups <code>aap_hosts</code> and <code>netapp_console_hosts</code> (as children)	safety: <i>Patch hosts</i> skips these even when a job targets them	yes
<code>patch_hosts</code>	the servers monthly patching may touch	<i>Patch hosts</i> only patches these	only for patching
<code>mariadb_hosts</code> , <code>mysql_hosts</code> , <code>database_hosts</code>	the database host(s), MariaDB or MySQL (any of the three, or the groups <code>mariadb</code> / <code>mysql</code> ; the check treats them alike)	the database check runs here, and the Database health template targets these by default	only for the database check
<code>stigman</code>	the STIG Manager host(s)	service watch and STIG Manager deploy run here	only for those

The clicks:

1. **Inventories > your inventory > Groups > Create group.** Type the **Name** (exactly as in the table), then **Create group**.
2. Add hosts: open the new group > **Hosts** tab > **Add existing host** > tick the hosts > **Add**.
3. Or add one of your groups inside it: open the new group > **Related Groups** tab > **Add existing group** > tick your group > **Add**. Its hosts are now in the new group too.

Keep your own names instead? If you already have, say, an `aap` group and do not want a second group, tell the runbooks your names. Put these in the inventory **Variables** (step 3). A list you write **replaces** the shipped one, so repeat every name you want kept; `aap` and `aap_hosts` are always protected, whatever you write:

YAML

```
site_act_diagnose_only_groups: [aap, netapp]    # ACT never fixes these
patch_never_reboot_groups: [aap]              # patching never reboots these
patch_never_patch_groups: [aap, netapp]       # patching skips these
```

Updated from 0.4.1 or older? Your `playbooks/group_vars/all.yml` then still sets these three lists (and `site_act_provider`, `site_act_models`) as real lines, and that file silently wins over the inventory Variables box. Open it and put `#` in front of those lines (or delete them), or make your change there instead. The job's **NOTE - settings in this project's files win ...** line lists what the file still sets.

The checks run on every host of the inventory. To keep them to your Linux servers, put **target: linux_servers** (your all-Linux group) in the **Variables** of the Health check, Troubleshoot, Certificate report and Apply approved ACT fix templates.

Step 3. Settings for every host (inventory variables)

Inventories › your inventory › Edit inventory › Variables. If something is already there, add below it; do not delete what is there. Paste, then change every line marked **CHANGE**:

```

--- what every host must be able to reach, from itself ---

check_network_dns_names: [ldap.yoursite.mil]           # CHANGE: names that must resolve
check_network_tcp:                                     # CHANGE: services every host needs
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_logging_remote:                                  # CHANGE: your SIEM / log collector
  - {name: SIEM, host: siem.yoursite.mil, port: 6514}

# ---- the account AAP logs in as ----
check_accounts_nopasswd_allowed: [svc_aap]             # CHANGE: your AAP service account
check_accounts_expiry_watch: [svc_aap]                # CHANGE: the same account

# ---- STIG ----
check_fapolicyd_required: true                         # false if some hosts do not run
  fapolicyd

# ---- where tickets go ----
servicenow_assignment_group: Linux Operations         # CHANGE: a group that exists in
  ServiceNow                                           # or critical: tickets for critical
servicenow_min_severity: warning                      only

```

The ACT provider and models are **not** set here: they are chosen in `playbooks/group_vars/all.yml` (lines ready as comments) or per template (ADDING_ACT.md, step 2). Do not also set them here: if the file ever sets them, the file wins and the value here is ignored.

Not sure about a value yet? Leave the line out. The check then uses its default, or checks nothing for it (no `check_network_tcp` means no TCP checks).

Step 4. Settings for particular groups

Open the group › **Edit group › Variables**, paste, change the **CHANGE** lines, **Save**. Put these on your own groups or on the groups from step 2. Both work.

The AAP server(s), for example on `aap_hosts`:

```

YAML

check_disk_overrides:
  /var: {warn: 80, crit: 90}      # AAP keeps job output and container images under /var

```

The STIG Manager host(s), on `stigman`:

YAML

```

check_certs_endpoints:          # the certificate browsers actually see
- {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.yoursite.mil} # CHANGE
check_certs_files: [/etc/stigman/*.crt, /etc/stigman/*.pem] # CHANGE:
  where your cert files are
watch_containers:              # optional since 0.6.0: service watch finds the containers
- name: stigman                 # that should run by itself; this list is watched first,
- name: nginx                   # in this order (dependencies first), and a missing one is
  reported

```

If STIG Manager runs as systemd units (`systemctl list-units | grep -i stig` shows them), also add them, so the services check watches them: `check_services_required_extra: [<unit>, <unit>]`.

The ServiceNow database host(s) (MariaDB or MySQL): also add this group to `mariadb_hosts` or `mysql_hosts` (step 2).

YAML

```

check_mariadb_container: servicenow-mariadb # CHANGE: the container name from `sudo podman ps
-a`

```

Then attach the **MariaDB monitor** credential to the **Database health**, Health check and Troubleshoot templates (it serves MySQL too). Container images give root a password, so the check logs in with a monitoring account instead. The whole setup (several containers, discovery, rootless containers, the account to ask the DBA for, and how to verify) is in MARIADB.md.

The ServiceNow MID Server host(s):

YAML

```

check_services_required_extra: [mid] # CHANGE: its unit name (systemctl list-units |
  grep -i mid)
check_certs_keystores:
- path: /opt/servicenow/mid/agent/security/agent_keystore # CHANGE: your MID Server's path

```

The NetApp Console host: nothing is needed. It is checked like every host; being in `netapp_console_hosts` and `no_patch` keeps automation from changing it.

Step 5. Settings for one host (only when one host is different)

Hosts › the host › Edit host › Variables. For example, a host whose `/data` is always nearly full on purpose:

YAML

```

check_disk_overrides:
  /data: {warn: 95, crit: 98}

```

Or a host with a known-broken vendor unit you want ignored:

YAML

```

check_services_ignore_failed: ['^vendor-agent']

```

Step 6. Use your inventory in the job templates

On every job template (and workflow), set **Inventory** to your inventory. That is the only link between them. Everything else (project, playbook, credentials, survey) is as in SETUP_AAP.md, steps 6 and 9.

Step 7. Check that it worked

1. Launch **Health check** with **Limit** = one server and the survey set to **daily**.
2. In **Report | result for this host**, look for your own values. A network finding names *your* LDAP server; an accounts finding names *your* service account. If a value is not there, the variable did not reach the host: check its spelling, and which group or host it is on.
3. **skipping: no hosts matched** (a green job that did nothing) means the Limit or **target** names a group or host this inventory does not have. Check the spelling against the inventory's **Groups** tab, and that the template uses the right **Inventory**.
4. **skipped: the MariaDB/MySQL check is off for <host>** means the host is in none of **mariadb_hosts**, **mysql_hosts**, **database_hosts**, **mariadb** or **mysql** and has no container name set (MARIADB.md).
5. Findings you do not care about: change the setting for that group or host (steps 4-5), not the code.

Which YAML files in the repository do I change?

File	Change it?
playbooks/group_vars/*.yaml, playbooks/host_vars/*.yaml	Yes, these are your settings (the update script never overwrites them)
inventories/example/...	No. Only a reference to copy blocks from; AAP does not read it
roles/*/defaults/main.yaml	No. Read VARIABLES_REFERENCE.md to see every setting; set values in a settings file or in AAP instead
playbooks/*.yaml	No
aap/credential_types/*.yaml	No. Paste their contents into AAP (credential types)
ansible.cfg	No. AAP ignores its inventory = line and uses the template's inventory
poam/poam.csv	Yes. Your POA&M export (see poam/README.md)

4. ServiceNow: set up, test and verify

The runbooks open ServiceNow incidents for the problems the checks find (the **ServiceNow tickets** step), and check that ServiceNow itself is healthy (**ServiceNow health**). This page takes you from nothing to a **verified test ticket**, one step at a time:

1. an API account from the ServiceNow admins;
2. the addresses: the instance URL and the API calls;
3. the network path from AAP to ServiceNow;
4. a quick check from a shell (optional);
5. the credential in AAP;
6. your settings: which assignment group gets the tickets;
7. the **ServiceNow - test ticket** template;
8. a dry run, then a real run;
9. checking the ticket in ServiceNow.

Sign-in method. This integration uses **basic authentication only** (an account name and password over HTTPS). OAuth, CAC/PKI login and SSO are **not supported**. If your instance requires one of them for the API, ask the ServiceNow admins for an integration account that is allowed to use basic authentication for the Table API, and exempt from SSO.

Plan about 30 minutes, plus the time the ServiceNow admins need for step 1.

What you need

Item	Example	Who gives it to you
The instance URL	<code>https://yourinstance.servicenowservices.com</code> or <code>https://servicenow.example.mil</code>	the ServiceNow admins (or your browser's address bar)
An API account and its password	<code>svc_aap_sn</code>	the ServiceNow admins
Roles on that account	<code>itil</code> (and <code>mid_server</code> for the MID Server check)	the ServiceNow admins
The assignment group for the tickets	<code>Linux Operations</code>	the ServiceNow admins or your team lead
A network path from AAP to the instance	TCP 443 from the AAP execution nodes	the network or firewall team
Trust for the instance's certificate	the DoD root CA, if the instance uses a DoD certificate	you (step 3)

Step 1. Ask the ServiceNow admins for an API account

What. An account that only the automation uses, allowed to read, open and update incidents.

Why. The playbooks log in to ServiceNow's REST API with a user name and password (basic authentication). A dedicated account keeps the tickets traceable ("opened by `svc_aap_sn`"), and its access can be limited to what the automation needs.

Send them this (change the names in brackets):

```
Request: a ServiceNow integration account for Ansible Automation Platform (AAP)

- User ID: svc_aap_sn (or your naming standard)
- "Web service access only": ticked (API use only, no interactive login)
- "Internal Integration User": ticked, if our release has it
- Roles: itil (read, create and update incidents; read groups and users)
        mid_server (read the MID Server list; only if we want the MID Server check)
  No admin role.
- Authentication: basic authentication over HTTPS for the Table API. If MFA or single
  sign-on is enforced, this account must be exempt, as web-service accounts usually are.
- If IP address access control is on: allow the AAP execution nodes' addresses: [addresses]
- Password: set, with "Password needs reset" unticked. Tell me the expiry or rotation
  rule, so the AAP credential can be updated in time.
- Tickets go to the assignment group: [Linux Operations]. Please confirm its exact name.
- If our instance has custom incident states or close codes: the value for "Resolved"
  and a valid close code.
```

You should get back the user ID, the password (through your normal secure channel), the exact assignment group name, and, if they are custom, the resolved state and close code.

Step 2. The addresses

The instance URL is the start of the address you see in your browser on ServiceNow: the `https://` and the host name, **nothing after it**.

In the browser	Instance URL for AAP
<code>https://yourinstance.servicenowservices.com/now/nav/ui/home</code>	<code>https://yourinstance.servicenowservices.com</code>
<code>https://servicenow.example.mil/nav_to.do?uri=incident_list.do</code>	<code>https://servicenow.example.mil</code>

The API calls. Everything goes to that host over HTTPS (TCP 443), with basic authentication and JSON. These are the only calls the playbooks make:

Method and path	What for	Used by
<code>GET /api/now/table/incident?sysparm_limit=1</code>	log in and read (the connection test)	test ticket, health
<code>GET /api/now/table/sys_user_group?sysparm_query=name=<group></code>	does the assignment group exist?	test ticket
<code>GET /api/now/table/sys_user?sysparm_query=user_name=<caller></code>	does the caller exist?	test ticket
<code>GET /api/now/table/incident?sysparm_query=correlation_id=...</code>	is a ticket for this problem already open?	tickets
<code>POST /api/now/table/incident</code>	open an incident	test ticket, tickets

Method and path	What for	Used by
GET /api/now/table/incident/<sys_id>	read an incident back	test ticket
PATCH /api/now/table/incident/<sys_id>	add a work note; resolve	test ticket, tickets
GET /api/now/table/ecc_agent	MID Servers: up and validated?	health

Step 3. The network path from AAP

Which machine talks to ServiceNow. The ServiceNow jobs run on the **AAP execution node**, inside the execution environment container, not on your managed hosts. To see the nodes, open **Automation Execution > Infrastructure > Instances** (or **Instance Groups**). Those are the addresses the firewall (and ServiceNow's IP access control, if any) must allow.

Firewall. Allow the execution nodes > the instance, TCP 443.

A proxy. If the execution nodes reach the outside only through a proxy, give the jobs the proxy settings. Go to **Settings > Automation Execution > Job > Edit > Extra Environment Variables**, and set (add to what is already there):

```
{"https_proxy": "http://proxy.example.mil:8080", "no_proxy": "localhost,127.0.0.1,.example.mil"}
```

This applies to every job. Leave hosts that must not go through the proxy in **no_proxy**.

The certificate. A ServiceNow instance with a DoD certificate is only trusted if the execution environment trusts the DoD root CA. Choose one:

Option	How	When
A. The CA in the execution environment	have the AAP admins build the execution environment with the DoD root CAs in its trust store	best: every job trusts it
B. A CA file in the repository	put the CA certificate (PEM) in the project, for example (an example path: the folder does not exist until you create it) <code>playbooks/files/ca/servicenow-ca.pem</code> , and set <code>servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"</code> (step 6)	when you cannot change the execution environment. CA certificates are public, so this is fine
C. Switch the check off	<code>servicenow_validate_certs: false</code>	only for one test, never for real use

Step 4. Check from a shell first (optional, fastest)

On the AAP execution node, or any Linux machine on the same network:

```
shell
curl -sS -u svc_aap_sn -H 'Accept: application/json' \
  'https://yourinstance.servicenowservices.com/api/now/table/incident?sysparm_limit=1&sysparm_fields=number'
```

`curl` asks for the password, so it is not left in your shell history.

You should see `{"result":[{"number":"INC0012345"}]}` (any number). Anything else:

What you get	What it means
SSL certificate problem: unable to get local issuer certificate	the CA is not trusted (step 3, the certificate)
Could not resolve host	DNS: check the host name
nothing, then Connection timed out, or Connection refused	firewall or proxy (step 3)
<code>{"error":{"message":"User Not Authenticated"...}}</code> (HTTP 401)	wrong user or password, the account is locked, or it is not allowed to use basic authentication
<code>{"error":{"message":"Operation Failed","detail":"ACL Exception..."}}</code> (HTTP 403)	the account lacks the <code>itil</code> role
an HTML page	the URL has something after the host name, or a login page (SSO) answered: the account must be exempt from SSO

Step 5. The credential in AAP

- If you have not yet, create the **ServiceNow API** credential type (SETUP_AAP.md, step 2). It uses the file `aap/credential_types/servicenow_api.yml`.
- Automation Execution > Infrastructure > Credentials > Create credential:**
 - Name** `ServiceNow API`;
 - Credential type** `ServiceNow API`;
 - Instance URL**: from step 2 (only `https://` and the host name);
 - API user name** and **API password**: from step 1.
- Create credential.**

You should see the credential listed, with the password shown as `ENCRYPTED`. The job gets the three values as `SN_HOST`, `SN_USERNAME` and `SN_PASSWORD`. They never appear in job output, and never reach your managed hosts.

Step 6. Your settings

In VS Code, open `playbooks/group_vars/all.yml`. (If your inventory comes from Git, the same lines in `inventories/site/group_vars/all.yml` end up in AAP's inventory Variables box, and the project's file wins over them: keep each setting in **one** place. See VARIABLES.md.) In the ServiceNow block, delete the `#` in front of the lines and fill in your values:

```

---- ServiceNow tickets -----
servicenow_assignment_group: Linux Operations    # the exact name from step 1
servicenow_min_severity: warning                # or critical: tickets for critical findings only
# servicenow_caller: svc_aap_sn                  # optional; empty = the API account is the caller
# servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem" # step 3, option B

```

Commit and Sync Changes. Every other ServiceNow setting (urgency, impact, resolve, close code) is described in `roles/servicenow/defaults/main.yml`.

Step 7. The ServiceNow - test ticket template

Automation Execution › Templates › Create template › Create job template:

Field	Value
Name	ServiceNow - test ticket
Inventory	any (for example <code>rhel-inventory</code>); it runs on AAP itself, not on these hosts
Project	site-automation
Playbook	playbooks/servicenow_test_ticket.yml
Credentials	ServiceNow API
Job type	Run, with Prompt on launch ticked (so you can pick <i>Check</i> , a dry run)
Limit	empty, and no Prompt on launch
Privilege escalation	not needed

Create job template.

Step 8. Run it: first a dry run, then for real

The dry run. Launch › Job type `Check` › Launch. Steps 1 and 2 run for real, because they only read. Steps 3 to 6 are only printed:

```

PASS 1 connection: the instance answers and svc_aap_sn logged in and read the incident table (0 s)
PASS 2 settings: assignment group "Linux Operations" exists; caller not set: the tickets show the
  API account as the caller
DRY RUN (Check mode): steps 1-2 ran (they only read). Steps 3-6 would:
3 open: POST https://yourinstance.servicenowservices.com/api/now/table/incident {...}

```

Nothing was created in ServiceNow.

The real run. Low priority does not mean silent: ServiceNow may still send its usual emails, to the assignment group for the new incident and to the caller when it is resolved. Warn the group first, or point the test at a test group (`servicenow_assignment_group: <test group>` in the template's **Variables**), or at a test instance if you have one.

Launch › Job type `Run`. At the end it prints a summary like this:

```

ServiceNow test at https://yourinstance.servicenowservices.com as svc_aap_sn:
PASS | 1 connection | the instance answers and svc_aap_sn logged in and read the incident table (0
s)
PASS | 2 settings | assignment group "Linux Operations" exists; caller not set: ...
PASS | 3 open | opened INC0012346 (sys_id 5f1c...)
PASS | 4 read back | INC0012346: state New, priority 5 - Planning, assignment group Linux
Operations, ...
PASS | 5 work note | added a work note to INC0012346 (see its Activity stream)
PASS | 6 resolve | resolved INC0012346 (state 6)

```

Step 4 also prints a link: **Open it in ServiceNow:** https://.../nav_to.do?uri=incident.do?sys_id=...

Result	Meaning
PASS	that step works
WARN	it works, but look at it (for example, no assignment group is set, or the instance would not resolve the ticket). The job stays green
FAIL	that step does not work. The job is red, and the line says why and what usually causes it

To leave the test ticket open and look at it first, add `servicenow_test_resolve: false` to the template's **Variables**.

Step 9. Check the ticket in ServiceNow

- Click the link from step 4 of the job output, or go to **Incident > All** and search for the number (INC0012346).
- Check:
 - Short description:** `[site-automation test] test ticket from AAP job <number> - safe to close;`
 - Assignment group:** yours (step 6);
 - Caller:** your `servicenow_caller`, or the API account;
 - Urgency** 3 and **Impact** 3, so normally **Priority** 5 - Planning, the lowest (your instance's priority table decides). The real tickets use `servicenow_urgency` and `servicenow_impact`;
 - Activity:** the work note `Test work note from AAP job ...;`
 - State:** Resolved, with the close notes `Test ticket from AAP job ...: resolved automatically by the same job` (unless you set `servicenow_test_resolve: false`).
- To find every test ticket: filter **Incident > All** on **Correlation display** = `site-health-test`.

The test passes when all six steps say PASS (or WARN for step 6, if you do not resolve tickets automatically), and the ticket in ServiceNow looks like the list above.

When a step fails

Step and message	Cause	Fix
1: Attach the "ServiceNow API" credential	the template has no ServiceNow credential	step 7: add it under Credentials
1: HTTP -1 ... certificate verify failed	the execution environment does not trust the instance's CA	step 3, the certificate
1: HTTP -1 ... Name or service not known	DNS	the Instance URL's host name; DNS from the execution node
1: HTTP -1 ... timed out or Connection refused	firewall or proxy	step 3
1: HTTP 401	wrong user or password; account locked or inactive; "Password needs reset"; basic authentication not allowed	step 1 with the admins, then update the credential
1: HTTP 403	the account may not read incidents	the <code>itil</code> role
1: HTTP 404	the Instance URL has a path after the host name	only <code>https://</code> and the host name
2: assignment group "... does not exist	the name differs from ServiceNow's, even in capitals or spaces	copy the exact name from ServiceNow into <code>servicenow_assignment_group</code>
2: could not look up ... (WARN)	the account may not read groups or users	fine if step 4 shows the group; otherwise ask for read access to <code>sys_user_group</code>
3: HTTP 403 ... ACL Exception Insert Failed	the account may read but not create incidents	the <code>itil</code> role
3: HTTP 400	ServiceNow refused a value, and says which	usually <code>servicenow_category</code> or a custom field
4: did not stick (WARN)	the ticket was opened without the group	the group name (step 6)
5: HTTP 403	the account may create but not update	the <code>itil</code> role, or an ACL on work notes
6: could not resolve ... Data Policy Exception (WARN)	the instance needs other values to resolve	ask the admins for the resolved state and close code; set <code>servicenow_resolved_state</code> and <code>servicenow_close_code</code> . Only needed if you use <code>servicenow_resolve_cleared: true</code>

Next

- **ServiceNow health** (hourly): the instance's API and the MID Servers (RUNBOOKS.md).
- **The tickets step** in the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md): one incident per problem, updated on later runs, noted or resolved when it clears.

- Keep **ServiceNow - test ticket**. Run it again after a password change, a new certificate, or a ServiceNow upgrade.

5. Workflows and schedules

A **workflow** chains job templates. Each arrow says when the next step runs:

- **Run on success:** only if the step before it succeeded.
- **Run on fail:** only if it failed.
- **Run always:** either way.

The checks *fail on purpose* when they find something. So "Run on fail" means "a problem was found", and "Run always" means "whatever happened". A step also receives what the steps before it published (**artifacts**). That is how the tickets step learns what the health check found.

This chapter first shows how to build any workflow, then a catalog of ready-made workflows for Linux, for Windows, and for both, and how to schedule them.

How to build any workflow, click by click

1. **Automation Execution › Templates › Create template › Create workflow job template.**
2. **Name** (e.g. **Daily health**), **Organization**.
 - **Inventory: leave it empty.** Each step then uses its own template's inventory, so one workflow can hold Linux and Windows steps. A workflow inventory only reaches steps whose template prompts for the inventory.
 - Tick **Prompt on launch** next to **Limit** if you want to pick hosts at launch.
 - Leave **Variables** empty: workflow variables override every step's own answers.
3. **Create workflow job template.** The **workflow visualizer** opens.
4. **The first step: Add step › Node type:**
 - *Job Template:* one of the templates (the usual case);
 - *Approval:* a person must click Approve (give it a **Timeout**);
 - *Workflow Job Template:* another workflow, as one step;
 - *Project Sync / Inventory Source Sync:* refresh the code or the inventory first.

Pick the template, then fill in **this step's** answers:

- **Survey answers**, e.g. `health_checks = daily`;
- **Job type** **Check** for a dry run (DRY_RUNS.md);
- a **Limit**.

The wizard only offers what the template asks in its survey or has **Prompt on launch** ticked for. › **Finish**.

5. **Next steps:** hover over a box › **Add step and link** (the plus sign) › the node, and the **link type** (success / fail / always).
 - **Two things at once:** add a second first step with **Add step**. Both start together.
 - **Two arrows into one step:** open the step › **Convergence:** *Any* runs it when one parent finishes as linked, *All* waits for all of them.
6. **Save** (top right). Closing without saving throws the graph away.
7. **Try it:** launch it once with the risky steps set to **Check** (step 4), then set them back.

Who may approve: open the workflow › **User Access** (or **Team Access**) › **Add roles** › **Approve**. Pending approvals are in **Automation Execution › Administration › Workflow Approvals**, and in the workflow

job's view.

Which templates fit where:

Template	Fails when	Publishes (for the next step)	Typical next step
Health check / Windows health check	a finding (<code>site_fail_on</code>), or ACT proposes a fix	the findings (<code>site_health</code>)	ServiceNow tickets (always); an approval (on fail)
Certificate report / Windows certificate report	a certificate expires soon	the findings	ServiceNow tickets (always)
POA&M status	an item is overdue / due soon	the findings	ServiceNow tickets (always)
Troubleshoot / Windows troubleshoot	never for findings (a look, not a verdict)	the findings	a health check, tickets if you want them
Patch hosts / Windows patch	a host fails, or its services do not come back	-	a health check (always)
Apply approved ACT fix	the problem is still there	the findings of the re-check	ServiceNow tickets (always)
Service watch - check	a container is down (NEEDS APPROVAL)	the fix to approve	an approval (on fail)
ServiceNow tickets	problems exist (it opened or updated tickets)	-	- (the end)

Adding ACT to a workflow: what "on fail" means, and where `site_act` goes

You do not add `site_act` to the workflow. There is no ACT template and no ACT box. ACT runs **inside the check job**. The check playbooks (Health check, Troubleshoot, Service watch, the Windows ones) already include `roles/site_act`, and it runs when that job's answers say `use_act = yes`, at the level in `site_act_level`. In the workflow you only give the check box those two answers.

"On fail" is the check job's own result. Nothing extra is checked. The check playbook decides by itself, against its thresholds (the role defaults and your `group_vars`: disk full %, certificate days, ...). The check box ends **red** when:

- it found a problem at or above `site_fail_on` (default: `critical` and `warning`), or
- ACT (`diagnose`) proposed a fix that waits for a person: `NEEDS APPROVAL ... ACT proposes: <commands>`, or
- a host could not be checked (unreachable, login or sudo failed).

A **Run on fail** link from the check box therefore leads to the approval. **Run on success** means the check box ended green; **Run always**, either way.

Box	Link into it	Runs when
1. Health check (<code>use_act = yes</code> , <code>site_act_level = diagnose</code>)	(start)	always
2. Approval	Run on fail , from 1	the check job ended red

Box	Link into it	Runs when
3. Apply approved ACT fix	Run on success , from 2	a person clicked Approve
4. ServiceNow tickets	Run always , from 1 and from 3	after the check, and after the fix

The steps in words:

1. **Health check** box: answers `use_act = yes`, `site_act_level = diagnose`. This needs those two survey questions on the template (ADDING_ACT.md, step 3).
2. **Approval** box, **Run on fail** from 1.
3. **Apply approved ACT fix** box (`playbooks/act_fix_approved.yml`), **Run on success** from 2. It needs no answers: AAP hands it ACT's proposed commands from the health check job, and it runs exactly those.
4. **ServiceNow tickets** boxes, **Run always**, from 1 and from 3.

The approver opens the health check job and reads `ACT | what ACT says` and the red `NEEDS APPROVAL` line. If there is no such line (the job is red because of findings ACT had no fix for, or an unreachable host), there is nothing to approve: **Deny**. Approving does no harm either: the apply step skips every host with nothing approved.

Without approvals: `site_act_level = self-heal` with the fixes you allow in `site_act_allow`. ACT applies those itself, inside the check job, and the checks run again. Then the workflow is simply `[Health check] --Run always--> [ServiceNow tickets]`.

A playbook of your own gets ACT in the **playbook**, not in the workflow: it includes `site_findings` (start, your check), then `site_act`, then `site_findings` report. See ADDING_ACT.md, "Use ACT in a playbook of your own". Its job template then gets the same two survey questions, and it fits the same workflow.

Linux workflows

1. Daily health



Daily health: every run hands its findings to the tickets step, healthy or not.

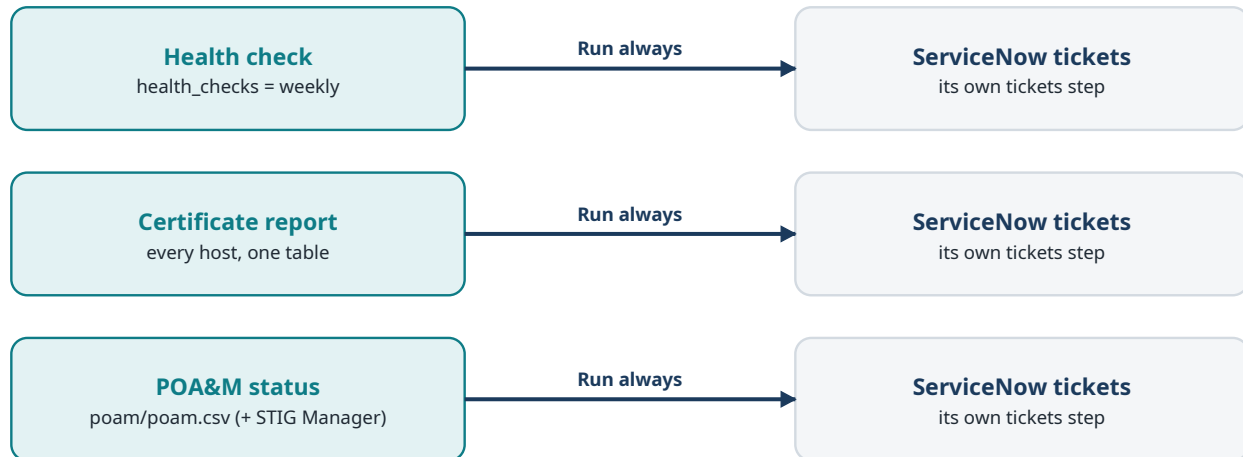
Step	Template	Answers	Link
1	Health check	<code>health_checks = daily</code>	(start)
2	ServiceNow tickets	none	Run always , from step 1

Why "always". The tickets step opens tickets for what failed, and also notes the tickets of problems that cleared. That needs the healthy runs too. It fails when problems exist, so the workflow shows red until they are fixed.

Schedule: every day at 06:00.

2. Weekly compliance

Three independent chains in one workflow. Add each first box with **Add step** (not "Add step and link"), so they all start together:

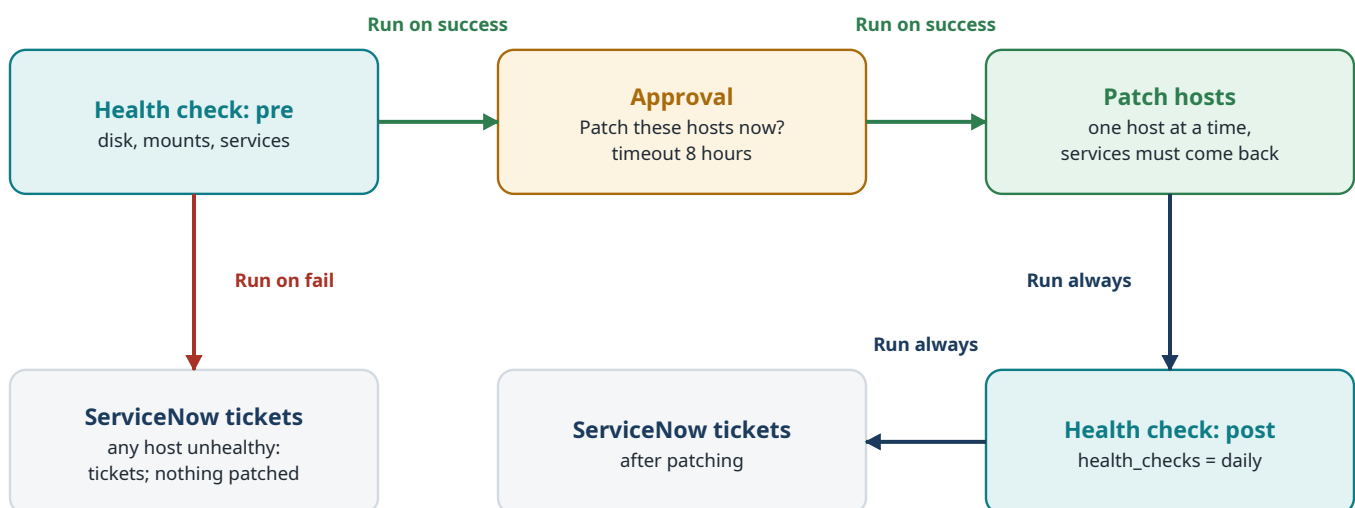


Weekly compliance: three independent chains, each with its own tickets step (a step with two parents would get only one parent's findings).

Give each chain **its own** tickets box. A box with two parents gets only one parent's findings: the artifacts overwrite each other.

Schedule: Mondays at 07:00.

3. Patch with checks



Patch with checks: every host healthy first (one failing host stops the whole run), a person approves, patch one host at a time, then check again.

Step	Template	Answers	Link
1	Health check	<code>health_checks = disk, mounts, services</code>	(start)

Step	Template	Answers	Link
2	<i>Approval</i>	Name Patch these hosts now? , Timeout 8 hours	Run on success , from 1
3	Patch hosts	patch_security_only as you like; automatic_restarts (see below)	Run on success , from 2
4	Health check	health_checks = daily	Run always , from 3
5	ServiceNow tickets		Run always , from 4
6	ServiceNow tickets		Run on fail , from 1 (tickets for what the pre-check found)

Why these links:

- **Healthy first.** Patching a host that is already failing makes the next problem impossible to diagnose. The pre-check must pass before anyone is asked. **If any one host fails it, the whole run stops before the approval and no host is patched.** Fix that host (the ticket says what is wrong), or launch again with a Limit that leaves it out. Warnings count too while **site_fail_on** includes **warning**. A disk at 86% blocks patching, so clear it before the maintenance window.
- **A person approves.** The approval step waits (up to its **Timeout**). Deny, or no answer, stops the workflow and nothing is changed.
- **Always check after.** The post-check runs whatever happened in patching, so a half-patched host is still reported.

No automatic restarts unless you ask for them. Patching installs the updates; a host that then needs a reboot is **not** rebooted, and the job says so. The post-check's **patching** finding ("reboot needed") keeps it visible until someone reboots it. To let the job reboot hosts that need it, give step 3 **automatic_restarts** = **yes**. Add a survey question for it on the Patch hosts template (*Multiple Choice, no / yes*, default **no**), or put **automatic_restarts: true** in the step's variables. Hosts in **patch_never_reboot_groups** are never rebooted either way.

Launch with **Limit** **patch_hosts**, or one group at a time (**stigman** this week, **servicenow_mid_hosts** next week). **Schedule:** monthly, in your maintenance window, for example the second Wednesday at 20:00. The approval then waits for the on-call person.

4. Patch with a dry run first

The same as 3, but the approver sees exactly what will be installed before approving:

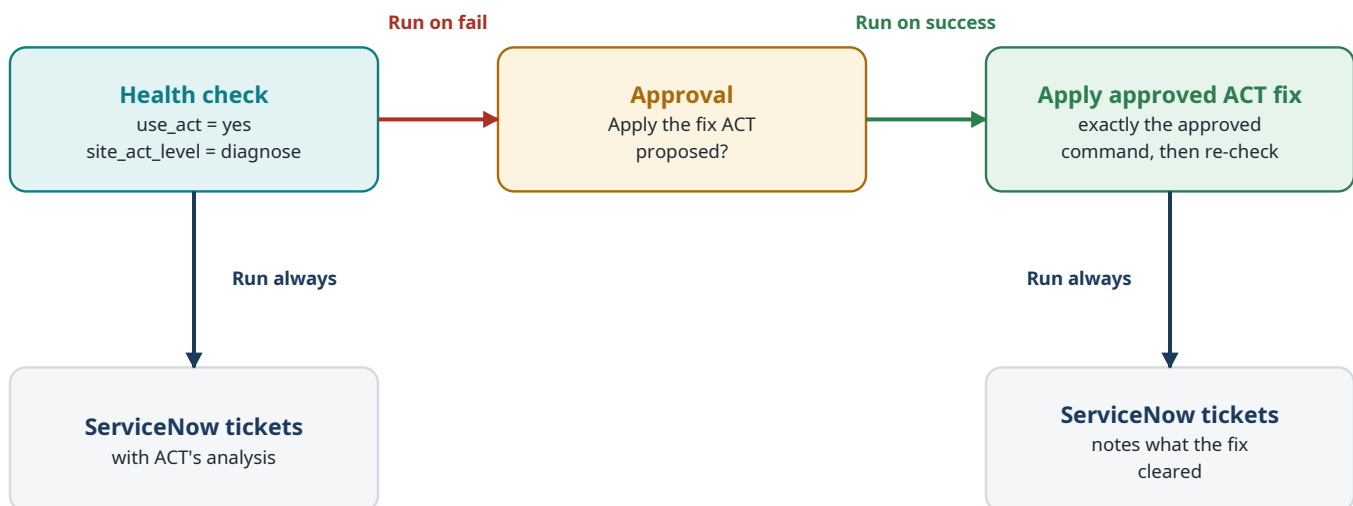
```
[Health check: pre]      health_checks = disk, mounts, services
| Run on success
[Patch hosts]           Job type = Check: lists what it would install
| Run on success
[Approval]              "Install what the dry run listed?"
| Run on success
[Patch hosts]           Job type = Run
| Run always
[Health check: post]    health_checks = daily
| Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	<code>health_checks = disk, mounts, services</code>	(start)
2	Patch hosts	Job type <code>Check</code>	Run on success , from 1
3	<i>Approval</i>	Name <code>Install</code> what the dry run listed?. Description: Open the "Patch hosts" dry-run job: it lists every package per host.	Run on success , from 2
4	Patch hosts	Job type <code>Run</code> ; <code>automatic_restarts</code> as you decide	Run on success , from 3
5	Health check	<code>health_checks = daily</code>	Run always , from 4
6	ServiceNow tickets		Run always , from 5

Needs **Prompt on launch** for **Job type** on the Patch hosts template (steps 2 and 4 set it).

5. Fix with approval (ACT)

For when you use ACT. See `ADDING_ACT.md` for the three levels.



Fix with approval (ACT): the check job fails with `NEEDS APPROVAL` only when ACT proposed a fix; deny or timeout changes nothing.

Step	Template	Answers	Link
1	Health check (with the ACT survey questions)	<code>use_act = yes, site_act_level = diagnose</code>	(start)
2	ServiceNow tickets		Run always , from 1
3	<i>Approval</i>	Name <code>Apply the fix ACT proposed?</code> , Timeout 30 minutes. Description: Read ACT's report and the <code>NEEDS APPROVAL</code> line in the health check job first.	Run on fail , from 1

Step	Template	Answers	Link
4	Apply approved ACT fix		Run on success , from 3
5	ServiceNow tickets		Run always , from 4 (notes the tickets whose problem the fix cleared)

Caveat: not yet proven in a live AAP. The whole approve-and-fix path depends on the check job's result (the proposed commands) reaching the apply job through the Approval node. The playbooks were tested on their own, but that hand-over inside AAP itself has not been run end to end. Rehearse it first with the dry run below, and read SERVICE_WATCH_DEMO.md (Part 9) if the apply job says **Nothing to apply**.

The approver reads, in the health check job, **ACT | what ACT says** (root cause, evidence, the fix) and the red **NEEDS APPROVAL ... ACT proposes: <command>** line. **That exact command** is all the apply step can run: it runs it itself (no model, no key), stops at the first command that fails, and runs the same checks again. The tickets carry ACT's analysis too.

Rehearse it: see ACT's fixes, change nothing.

- **One job, no workflow.** Launch **Health check** (or **Troubleshoot**) with `use_act = yes` and `site_act_level = diagnose`. ACT investigates with read-only commands and proposes fixes; every change is refused. The job ends red with **NEEDS APPROVAL ... ACT proposes: <commands>**: that list is what would run. Nothing on the host changed.
- **The whole workflow, as a dry run.** On the **Apply approved ACT fix** template (and *Service watch - apply approved fix*) tick **Prompt on launch** next to **Job type**. In the workflow visualizer click the apply box › **Edit** (pencil) › **Job type Check** › **Save**. Now run the workflow and approve: the apply job prints **DRY RUN on <host>: approved, and would run: <commands>. Nothing was changed (Check mode).** and ends green. When you trust it, set that node's **Job type** back to **Run**.

6. Security posture (weekly)

The STIG-related checks, and the POA&M list, with tickets:

```
[Health check] --Run always--> [ServiceNow tickets]
    selinux, fapolicyd, auditd, accounts
[POA&M status] --Run always--> [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	health_checks = selinux, fapolicyd, auditd, accounts	(start)
2	ServiceNow tickets		Run always , from 1
3	POA&M status		(start, with Add step)
4	ServiceNow tickets		Run always , from 3

Schedule: Wednesdays 07:00. Use it instead of the **weekly** set in *Weekly compliance* if your security team wants these on their own day.

7. Certificate watch (daily)

Certificates expire on a day, not in a week:

```
[Certificate report] --Run always--> [ServiceNow tickets]
```

Schedule: every day 06:30. Set `check_certs_warn_days` (survey) to 30 so tickets start a month ahead. The de-duplication keeps it to one ticket per certificate.

8. Database watch: MariaDB and MySQL (hourly)

```
[Database health - MariaDB / MySQL] --Run always--> [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Database health - MariaDB / MySQL	none; it targets <code>mariadb_hosts</code> , <code>mysql_hosts</code> and <code>database_hosts</code> by default (a Limit narrows it)	(start)
2	ServiceNow tickets		Run always, from 1

The database template runs only the MariaDB / MySQL check (host installs and podman containers), so it is quick and the database team can own it and its tickets (MARIADB.md). Prefer to keep it in the general Health check? Use the *Health check* as step 1 with `health_checks = disk, services, mysql` (`mariadb` and `database` are the same check) and a **Limit** of your database group. The template *Database health* also supports `use_act=true`.

Schedule: hourly. It is quick, and a ticket comes within the hour of a problem.

9. Investigate a host (on demand)

One launch gives the troubleshooting commands, ACT's explanation and every check:

```
[Troubleshoot] --Run always--> [Health check]
ts_area, use_act = yes,          health_checks = all
site_act_level = explain
```

Step	Template	Answers	Link
1	Troubleshoot	<code>ts_area = overview</code> (or asked at launch), <code>use_act = yes</code> , <code>site_act_level = explain</code>	(start)
2	Health check	<code>health_checks = all</code>	Run always, from 1

Give the workflow **Prompt on launch** for **Limit** and launch it with one host. No tickets: you are already looking. Add `[ServiceNow tickets]` after step 2 (**Run always**) if you want them.

10. STIG Manager deploy with checks

```
[STIG Manager - deploy]      Job type = Check (+ Show changes): what would change
| Run on success
[Approval]                  "Apply the changes the dry run listed?"
| Run on success
[STIG Manager - deploy]      Job type = Run
| Run always
[Service watch - check]
| Run always
[Health check]              health_checks = certs, services
```

Step	Template	Answers	Link
1	STIG Manager - deploy	Job type Check (and Show changes)	(start)
2	<i>Approval</i>	Apply the changes the dry run listed?	Run on success , from 1
3	STIG Manager - deploy	Job type Run	Run on success , from 2
4	Service watch - check		Run always , from 3
5	Health check	health_checks = certs, services	Run always , from 4

The dry run shows which files and units would change, and how. The approver approves only changes they expected. (This dry run has not been tried on a real server yet: run step 1 on its own once, on the test server, before you build the workflow.)

11. Service watch

The approval or self-heal workflow for containers: SERVICE_WATCH_DEMO.md, Part 6.

If you schedule it every 15 minutes, turn **off** "Enable concurrent jobs" on the workflow (edit the workflow), because its approval waits up to 30 minutes: without that, a new run starts every 15 minutes while the last one still waits for a person.

Windows workflows

Build them the same way, with the Windows templates (WINDOWS.md). The ServiceNow tickets step is the same template as for Linux: it runs on AAP itself.

12. Windows daily health

```
[Windows health check] --Run always--> [ServiceNow tickets]
health_checks = daily
```

Schedule: every day 06:15.

13. Windows weekly compliance

```
[Windows health check] --Run always--> [ServiceNow tickets]
health_checks = weekly
[Windows certificate report] --Run always--> [ServiceNow tickets]
```

Schedule: Mondays 07:15.

14. Windows patch with a dry run first

```
[Windows health check: pre] health_checks = disk, services
| Run on success
[Windows patch] Job type = Check: lists the updates Software Center has
| Run on success
[Approval] "Install what the dry run listed?"
| Run on success
[Windows patch] Job type = Run (automatic_restarts?)
| Run always
[Windows health check: post] health_checks = daily
| Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Windows health check	<code>health_checks = disk, services</code>	(start)
2	Windows patch	Job type <code>Check</code>	Run on success , from 1
3	<i>Approval</i>	<code>Install what Software Center offers (listed in the dry run)?, Timeout 8 hours</code>	Run on success , from 2
4	Windows patch	Job type <code>Run; automatic_restarts</code> as you decide	Run on success , from 3
5	Windows health check	<code>health_checks = daily</code>	Run always , from 4
6	ServiceNow tickets		Run always , from 5

No automatic restarts unless step 4 gets `automatic_restarts = yes`: the servers that need a restart are listed, and the post-check's `patching` finding ("a restart is pending") keeps them visible. **Schedule:** inside the servers' ConfigMgr maintenance window. Outside it, ConfigMgr holds the updates and the patch step stops and says so.

15. ACT for Windows rollout

```
[ACT for Windows - install]    Job type = Check: would install / update / already current
| Run on success
[Approval]
| Run on success
[ACT for Windows - install]    Job type = Run
```

The dry run lists, per server, whether ACT would be installed, updated, or is already current. Launch it with a Limit (a group) to roll out group by group. Run it again after every update of this repository.

16. Windows: investigate a server (on demand)

```
[Windows troubleshoot] --Run always--> [Windows health check]
ts_area                health_checks = all
```

Launch with one server in the Limit.

Linux and Windows together

17. Everything, every morning

One workflow, two independent chains (each first box added with **Add step**). Each step uses its own template's inventory, so leave the workflow's Inventory empty:

```
[Health check]                --Run always--> [ServiceNow tickets]
health_checks = daily
[Windows health check]        --Run always--> [ServiceNow tickets]
health_checks = daily
[Certificate report]          --Run always--> [ServiceNow tickets]
```

Schedule: every day 06:00. It replaces workflows 1, 7 and 12.

Schedules

What. A schedule launches a template or workflow at set times.

How. Open the workflow (or template) › **Schedules** tab › **Create schedule**:

1. **Name, Start date/time, Time zone** (use your site's, not UTC, so "06:00" means 06:00).
2. **Repeat frequency** and its details: *Day* (every 1 day), *Week* (on Monday), *Month* (on the second Wednesday)...
3. **Prompts**: the survey answers, the Limit and the Job type **for this schedule**. Without them, the schedule uses the template's **defaults**. A schedule can only change what the template **prompts on launch** or asks in its survey.
4. **Finish**. The schedule's page shows the next run times. Check them once.

What	When	Launches
Daily health (or <i>Everything, every morning</i>)	every day 06:00	workflow 1 (or 17)
Certificate watch	every day 06:30	workflow 7
Database watch	hourly	workflow 8
ServiceNow health	hourly	template <i>ServiceNow health</i> , with an email notification on failure
Weekly compliance	Monday 07:00	workflow 2
Security posture	Wednesday 07:00	workflow 6
Patch preview	the Monday before patch night	template <i>Patch hosts</i> , Prompts › Job type Check
Patch with checks	monthly, in your window	workflow 3 or 4
Service watch	every 15 minutes (with Enable concurrent jobs switched off on the workflow)	workflow <i>Service watch - approve or self-heal</i>
Windows daily health / weekly compliance	06:15 daily / Monday 07:15	workflows 12 and 13
Windows patch	monthly, in the ConfigMgr maintenance window	workflow 14

Notifications (optional). **Automation Execution › Administration › Notifiers › Create notifier** (type *Email*, your mail relay). Then on a template or workflow › **Notifications** tab › switch on **Failure**. Use it where a ticket cannot help. ServiceNow health is the main case: if ServiceNow is down, it cannot take a ticket about itself.

Things that trip people up

- **Red is not broken.** A check job with findings is red by design. Read **Report | result for this host**: findings mean the playbook worked.
- **Save the visualizer.** Closing without **Save** loses the graph.
- **One tickets box per check box.** Artifacts from two parents overwrite each other.

- **A step can only be set to Check** (or given a Limit, or survey answers) if its template has **Prompt on launch** for it, or a survey.
- **Schedules use survey defaults** unless the schedule sets its own answers (Prompts step).
- **Approvals need a timeout.** Otherwise a forgotten approval keeps the workflow open forever. After the timeout it counts as denied.
- **Workflow variables override node answers.** Leave the workflow's **Variables** empty, and set answers on each step.
- **Leave the workflow's Inventory empty** unless every step should use it: it only reaches steps whose template prompts for the inventory, and a Linux inventory on a Windows step matches no host.
- **Limit on the workflow** applies to every step whose template prompts for a Limit. The controller-side templates (ServiceNow tickets, ServiceNow health, POA&M status) must **not** prompt for one, or a workflow Limit makes them match no host and silently do nothing.
- **A tickets step fails when nothing reached it.** If the check step died before checking any host (credential, project sync, execution environment), the tickets step fails too, so a broken schedule shows red instead of green. Read the first red job.
- **No automatic restarts** unless a patch step gets `automatic_restarts = yes`.

6. Dry runs: try anything, change nothing

A **dry run** runs a job without changing anything. Ansible calls it *check mode*; AAP calls it **Job type: Check**. Every step that would change something only says what it *would* change. The steps that only read (the health checks, looking at logs, asking ServiceNow what is open) still run, so the answers are real.

Use a dry run the first time you use any template, on one host, and before every change you are not sure about: patching, applying an approved fix, installing ACT.

Where to set it

There are five places. Pick the one that fits:

Where	How	Good for
1. The template	Template › Edit › Job type Check	a template that is <i>always</i> a dry run, e.g. a copy named Patch hosts (dry run) that more people may launch
2. At launch	Template › Edit › tick Prompt on launch next to Job type . Then at launch, pick Check or Run	the usual way: the same template, you choose each time
3. A workflow step	(needs 2 on the template) Workflow › Visualizer › click the step › Edit (pencil) › Job type Check › Save	rehearsing a whole workflow, or a "dry run first, then real" workflow
4. A schedule	(needs 2) Template or workflow › Schedules › the schedule › Prompts › Job type Check	a regular preview, e.g. "what would patch night install" a week before
5. Command line	ansible-playbook ... --check (add --diff to see file changes)	testing from a shell

Show changes. Tick **Show changes** on the template (or **--diff**) and a dry run also shows the line-by-line changes it would make to files, for example in STIG Manager's configuration.

How to tell. The job's **Details** tab says **Job type: Check**. Our playbooks also print **DRY RUN (Check mode): would ...** where it matters.

What each template does in a dry run

Template	In a dry run it...
Health check, Database health, Troubleshoot, Certificate report	runs exactly as usual : they only read, so you get the same findings. Nothing is written: no syslog lines, no disk-forecast sample, and on Windows no event log entries
POA&M status	as usual: reads the CSV and asks STIG Manager
ServiceNow tickets	asks ServiceNow what is already open, then prints would open: ..., would add a note to: ..., would resolve: Nothing is sent
ServiceNow health	as usual: only reads
ServiceNow - test ticket	logs in and checks the assignment group and caller (it only reads), then prints the incident it would open, note and resolve. Nothing is created
Patch hosts (Linux)	lists the packages dnf would update. Installs nothing, restarts nothing
Apply approved ACT fix	DRY RUN on <host>: approved, and would run: <commands> . Runs nothing. A command the apply-time guard would refuse (reboot, mkfs, ...) shows Refused on <host>: ... and that host goes red, as in the real run
Service watch - check	checks the containers and shows what it would propose. Writes no incident, does not call ACT
Service watch - apply approved fix	DRY RUN on <host>: approved, and would run: <commands> (or Refused on <host>: ... , as above)
STIG Manager - deploy	says which files and units would change (with Show changes: how), and verifies nothing (nothing was started). Changes nothing. <i>Not yet tried on a real server: try it once on the test server</i>
Windows health check, troubleshoot, certificate report, connection test	as usual: they only read
Windows patch	asks Software Center to rescan (without starting anything) and prints DRY RUN (Check mode): would install <updates>
ACT for Windows - install	DRY RUN: would install ACT in C:\ProgramData\act because ... , or that it is already current

ACT does not run in a dry run. It would call the model, and ACT could change things. A dry run with `use_act = yes` reports the checks and says ACT gave no result. To see ACT's ideas without changes, do a *real* run with `site_act_level = diagnose`: ACT then only reads and proposes (ADDING_ACT.md).

Examples

Before patching one server. Launch **Patch hosts** › **Job type** **Check**, **Limit** **web01**. The output lists every package that would be updated. Nothing is installed.

```
TASK [patch : Patch | what was updated (dry run - what WOULD be updated)]
ok: [web01] => {"msg": "DRY RUN (Check mode), nothing was installed - would make 4 change(s):
  Installed: expat-2.5.0-6.el9_8.5.x86_64; Installed: libxml2-2.9.13-14.el9_8.5.x86_64;
  Removed: libxml2-2.9.13-14.el9_8.4.x86_64; Removed: expat-2.5.0-6.el9_8.3.x86_64"}
```

(**Removed** is the old version that the new one replaces.)

Before patch night, every month. On **Patch hosts** add a schedule for the Monday before patch night, with **Prompts** › **Job type** **Check**. The job output shows what patch night will install on every host, in time to exclude something.

A dry-run copy for the team. Template **Patch hosts** › ... › **Copy**. Rename the copy **Patch hosts (dry run)**, set **Job type** **Check**, untick **Prompt on launch** for it, and give your team **Execute** on the copy only. They can preview; only you can patch.

Which tickets would a run open? In the *Daily health* workflow click the **ServiceNow tickets** step › **Edit** › **Job type** **Check**. Run the workflow: the tickets job prints what it would open, note and resolve. Set it back to **Run** when the tickets look right.

A Windows server, before its first real patch. Launch **Windows patch** › **Job type** **Check**, **Limit** one server:

```
"DRY RUN (Check mode): would install 2026-09 Cumulative Update (KB5040001); Malicious Software
Removal Tool (KB890830)"
```

The approve-and-fix path, end to end. See WORKFLOWS_AND_SCHEDULES.md, "Fix with approval (ACT)": set the apply step to **Check**, run the workflow, approve. The apply job shows what it would run.

From a shell (on a machine with Ansible and this repository):

```
shell
ansible-playbook -i <inventory> playbooks/patch_hosts.yml --check --diff -l web01
ansible-playbook -i <windows inventory> playbooks/win_patch.yml --check -l web01
```

What a dry run cannot tell you

- Whether the updates will **install cleanly**, or how long they take: it only lists them.
- Whether a **restart** would succeed: nothing restarts in a dry run.
- Anything that **depends on an earlier change**. In a dry-run workflow the post-check after "patching" still sees the unpatched host, because nothing was patched.
- What **ACT** would say (see above).

7. The runbooks: every check and what to do

For each runbook: what it answers, its settings (defaults in `roles/<role>/defaults/main.yml`; change them in your settings files or AAP's Variables boxes, as `VARIABLES.md` shows), and each finding with what usually fixes it. The `look:` command printed with a finding is always the first thing to run.

Health check (playbooks/health_check.yml)

Runs the checks you pick on each host: a list of names, or the shortcuts:

- `daily` = disk, mounts, services, performance, time, network, logging, mariadb (the database check: MariaDB or MySQL; `mysql` and `database` also pick it), containers (podman containers, rootful and rootless)
- `weekly` = selinux, fapolicyd, auditd, accounts, certs, patching
- `all` = both

A host with a finding at a severity in `site_fail_on` (default: critical and warning) fails. That is what a workflow reacts to. A check that itself breaks becomes a warning finding (the X check could not run: `..., id <check>:check-error`) and the other checks still run. That finding fails the host even when `site_fail_on` is `[critical]` (a check that did not run is not a healthy result); only `site_fail_on: []` never fails.

disk

Finding	Means	Usually
<code>/var is 96% full</code>	local filesystem space (<code>check_disk_warn_pct 85</code> , <code>check_disk_crit_pct 95</code>)	find what grew with the <code>look:</code> command: old logs, core dumps, a runaway log file. <code>journalctl --vacuum-size=500M</code> for the journal
<code>... has used 97% of its inodes</code>	too many small files (space may still be free)	usually a directory with millions of small files (session/cache/spool); <code>look:</code> finds it
<code>/var will be full in about 3.2 days</code>	growth-rate forecast from earlier runs (needs a day of history)	same as above, before it becomes an outage
<code>could not read filesystem usage</code>	<code>df</code> failed or timed out	a hung mount; see the mounts check

Per-mount limits: `check_disk_overrides: {/var/log/audit: {warn: 70, crit: 85}}`. The forecast keeps a small history in `/var/lib/site-health/disk-history.json` (the only file the checks write; a Check-mode run does not write it).

mounts

Finding	Means	Usually
<code>/data (xfs) is in /etc/fstab but is not mounted</code>	it did not mount at boot, or was unmounted	<code>journalctl -b</code> for the mount error; a missing disk or LUN; a typo in fstab
<code>/var is mounted READ-ONLY</code>	the kernel remounted it read-only after disk/filesystem errors	urgent: <code>journalctl -k</code> for I/O errors; storage team; plan an fsck
<code>network mount /mnt/share does not answer within 5s</code>	NFS/CIFS server down, network, or a stale handle	check the server and the network; diagnose only (NetApp is vendor-managed)
<code>network mount ... is 96% full</code>	the share is full	the storage owner

services

Finding	Means	Usually
<code>systemd unit X has FAILED</code>	a unit crashed or exited with an error	<code>systemctl status X</code> , <code>journalctl -u X</code> ; fix the cause, then <code>systemctl restart X</code>
<code>required service X is inactive</code>	a service that must run (<code>check_services_required + _extra</code>) is stopped	same; find out why it stopped before starting it
<code>required service X is not installed</code>	the unit does not exist here	remove it from the list for this group, or install it
<code>required service X is disabled</code>	it will not start after a reboot	<code>systemctl enable X</code>
<code>X has restarted 5 times</code>	a restart loop: it keeps crashing	its log shows the crash

Per group: `check_services_required_extra: [mariadb]`. To ignore a failed unit: `check_services_ignore_failed: ['^dnf-makecache']`.

performance

Finding	Means	Usually
15-minute load is 14.2 on 4 CPUs (3.55 per CPU)	more work than CPUs, for 15 minutes	the top CPU processes are in the finding; a runaway job, or undersized
only 4% of memory is available	memory is nearly exhausted	the top memory processes are in the finding; a leak, or undersized
63% of swap is in use	the host is short of memory	as above
CPUs spend 35% of their time waiting for disk	slow or overloaded storage	<code>iostat -x</code> shows which device; storage latency
the hypervisor takes 15% of CPU time (steal)	the VM host is overcommitted	the virtualization team
the kernel killed processes for lack of memory	OOM kills in the last 24 h (with the victims)	the victim's memory limit, or the host's memory
27 zombie processes	a parent process is not reaping its children	restart the parent (<code>look</code> : shows it)

time

Finding	Means	Usually
the clock is NOT synchronized	chrony uses no time source	<code>chronyc sources -v</code> : unreachable servers, firewall (UDP 123)
chrony is installed but not answering	chronyd is not running	<code>systemctl start chronyd</code>
the clock is 2.4s ahead of NTP time	offset above <code>check_time_offset_warn</code>	chrony corrects it slowly; very large offsets break Kerberos/AD logins at 5 minutes

network

Finding	Means	Usually
there is no default route	the host cannot reach other networks	NetworkManager connection, gateway
this host's own name ... does not resolve	its FQDN is not in DNS or <code>/etc/hosts</code>	DNS record
ldap.example.mil does not resolve (DNS)	a name in <code>check_network_dns_names</code> fails	<code>/etc/resolv.conf</code> , DNS server
LDAP (ldap.example.mil:636) does not answer within 5s	a TCP port in <code>check_network_tcp</code> is unreachable from this host	firewall (local or network), the service down, routing

logging

Finding	Means	Usually
the journal is kept in memory only	<code>/var/log/journal</code> missing: logs vanish at reboot	<code>mkdir /var/log/journal && systemctl restart systemd-journald (or Storage=persistent)</code>
log server SIEM (...) does not answer	forwarding target unreachable	network/firewall; the SIEM collector
rsyslog has 850 MB of undelivered messages queued	the log server has been unreachable for a while	as above; the queue drains once it answers
1203 error messages in the journal	something complains a lot (top senders listed)	look at the top sender first

selinux

Finding	Means	Usually
SELinux is Permissive, it must be enforcing	not enforcing (STIG)	<code>setenforce 1</code> after checking denials; <code>SELINUX=enforcing</code> in <code>/etc/selinux/config</code>
<code>/etc/selinux/config</code> says <code>SELINUX=permissive</code>	a reboot would change the mode	fix the config file
locally added permissive domain(s)	SELinux only logs for those domains	<code>semanage permissive -d <domain></code> once the policy is right
12 SELinux denial(s) since yesterday - top: nginx (httpd_t) denied read on a file labeled user_home_t	SELinux blocked something	a wrong label (fix: <code>restorecon -Rv <path></code>), a boolean (<code>setsebool -P ...</code>), or a real policy need. <code>audit2why</code> says which
SELinux boolean X is off, expected on	a boolean in <code>check_selinux_booleans</code> differs	<code>setsebool -P X on</code>
N file(s) under <code>/etc/nginx</code> have the wrong SELinux label	a dry-run <code>restorecon</code> would relabel them (<code>check_selinux_relabel_paths</code>)	<code>restorecon -Rv <path></code>

fapolicyd

Finding	Means	Usually
fapolicyd is inactive	application allow-listing is off	<code>systemctl start fapolicyd</code> ; its log says why it stopped
fapolicyd is in permissive mode	it logs but blocks nothing	<code>permissive = 0</code> in <code>/etc/fapolicyd/fapolicyd.conf</code>
N file(s) no longer match the fapolicyd trust database	files changed outside RPM	<code>fapolicyd-cli --update</code> , or re-add the file's trust entry
N fapolicyd denial(s) since yesterday - top: /usr/bin/bash -> /opt/app/tool	it blocked programs	a legitimate program needs a trust entry (<code>fapolicyd-cli --file add <path></code>), after review

`check_fapolicyd_required: true` (the example inventory sets it) makes a missing fapolicyd a finding.

auditd

Finding	Means	Usually
auditd is failed	nothing is being audited	<code>systemctl status auditd</code> ; often a full audit log partition
auditing is DISABLED in the kernel	<code>auditctl -s</code> says enabled 0	<code>audit=1</code> on the kernel command line
only 1 audit rule(s) are loaded	the STIG rules are missing or failed to load	<code>augenrules --check</code> , <code>augenrules --load</code> ; a syntax error in <code>/etc/audit/rules.d/</code>
N audit event(s) were LOST	the kernel backlog overflowed	raise <code>-b</code> (backlog) in the rules
the audit log filesystem is 92% full	auditd may stop, or halt the host, when it fills	rotate/offload; check <code>space_left_action</code>
audit records are not forwarded	au-remote is not active (<code>check_auditd_remote_required</code>)	configure audisp-remote

accounts

Finding	Means	Usually
account(s) other than root have UID 0	full root power under another name	investigate first ; remove or renumber the account
account(s) with an EMPTY password	log in without a password	<code>passwd -l <user></code>
3 account(s) not used for 35+ days	inactive and not locked (STIG)	lock them (<code>usermod -L</code>), or list break-glass accounts in <code>check_accounts_ignore_inactive</code>
sudo without a password (NOPASSWD) for: alice	a sudoers entry skips the password	remove it, or allow it on purpose in <code>check_accounts_nopasswd_allowed</code> (e.g. <code>svc_aap</code>)
the password of svc_aap expires in 6 days	an account in <code>check_accounts_expiry_watch</code> expires soon	change it before it breaks automation (and update the Machine credential)

certs

Finding	Means	Usually
certificate CN=... in /etc/stigman/stigman.crt expires in 9 days	a certificate file expires within <code>check_certs_crit_days</code> (critical) / <code>warn_days</code> (warning)	renew it. For STIG Manager: update the <i>TLS certificate</i> credential and run <i>STIG Manager - deploy</i> ; it restarts only nginx
... in STIG Manager (https://127.0.0.1:443) expires in ...	the certificate a TLS endpoint serves	the service's certificate, even if the file on disk was already replaced (restart the service)
... in /opt/.../agent_keystore (alias mid) expires in ...	a Java keystore entry	renew in the keystore (MID Server: ServiceNow's procedure)
could not check the certificate in X	unreadable file, endpoint not answering, or a PKCS12 keystore without its password	fix the path, or add the Keystore password credential

What it looks at: `check_certs_files` (globs), the usual directories (`check_certs_discover_dirs`, skipping CA bundles), `check_certs_endpoints`, `check_certs_keystores`. For a PKCS12 keystore, name the variable that holds the password (`password_env: KEYSTORE_PASSWORD`), never the password.

patching

Finding	Means	Usually
12 security advisories pending, 2 of them rated Critical	updates that fix security issues are waiting	patch (the <i>Patch with checks</i> workflow)
a reboot is needed to finish earlier updates	new kernel or core libraries are not in use yet	reboot in a window
no package has been updated for 60 days	older than <code>check_patching_max_age_days</code>	patch
cannot check for updates	dnf cannot reach its repositories (Satellite, mirror)	repository configuration, network, subscription

mariadb and mysql (health only): the database check

Checks **MariaDB and MySQL** (5.7, 8.0, 8.4; MariaDB 10.x and 11.x; Percona). Pick it in the Health check as `mariadb`, `mysql` or `database` (the same check), or run it alone with the **Database health - MariaDB / MySQL** template (`playbooks/database_health.yml`: only this check, on every database server it is pointed at; it fails a host that has findings, so a workflow can open a ticket). In the Health check it runs on hosts in `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql`, and on any host that has a container name set. It runs status queries only: no data is read and nothing is changed. It handles a database installed on the host and one in **podman containers** (one, several, found automatically, or rootless): the client inside each container is used. **How to set it up, step by step, and how to verify it: MARIADB.md.**

Finding	Means	Usually
MariaDB is not running	the service or container is down	its log; disk full; a crash
MariaDB is running but the status query failed	cannot log in, or it hangs	the login (see below); <code>max_connections</code> reached
MariaDB restarted 12 minutes ago	a restart nobody may have planned	the error log: crash, OOM kill
143 of 151 connections in use (95%)	clients are about to be refused	the <code>look</code> : query shows which user and client hold them; a connection leak in an app
N client connection(s) were REFUSED	<code>max_connections</code> was reached since the last start	as above; raise <code>max_connections</code> only if the load is real
40 queries are executing at this moment	a pile-up: locking or one slow query	<code>look</code> : lists the oldest
2 query/queries running longer than 300s	long-running queries (user / db / seconds / state; never the query text)	the owner decides; <code>KILL <id></code> only after that

Finding	Means	Usually
replication (default) is broken: IO thread No	a replica stopped copying	<code>Last_IO_Error</code> / <code>Last_SQL_Error</code> in the finding
replica ... is 1900 seconds behind	replication lag	load on the replica; long transactions on the primary
Galera node is not healthy / cluster has 2 node(s), expected 3	Galera cluster trouble	the node's error log; network between nodes
MySQL Group Replication is not healthy / has 2 ONLINE member(s), expected 3	a Group Replication member is not ONLINE, or the group is short (<code>check_mariadb_group_size</code>)	<code>SELECT ... FROM performance_schema.replication_group_members</code> (in the finding); the member's error log
could not read the replication status / ... Group Replication members	the monitoring account lacks a privilege	the grant named in the finding (MARIADB.md)
only 91% of reads come from memory	<code>innodb_buffer_pool_size</code> may be too small	the DBA
3 [ERROR] line(s) in the MariaDB log	errors in its log in the last day (latest shown)	read them
the MariaDB data directory is on a filesystem 93% full	MariaDB stops writing when it fills	disk space

Logging in (the full steps are in MARIADB.md). Without a credential, the check logs in as root through the local socket. That is MariaDB's default for a host install on RHEL (no password needed with sudo); MySQL's root usually has a password. **A database in a container usually gives root a password** (the official images do), and the check then reports `Access denied ... attach the "MariaDB monitor" credential`. For containers, or whenever you prefer not to use root, ask the DBA to create a monitoring account with **no data access**, inside that database with host `localhost`, and attach the *MariaDB monitor* credential to the Health check template:

```
SQL
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';    -- MySQL; MariaDB before 10.5.9
GRANT PROCESS, REPLICATION MONITOR ON *.* TO 'aap_monitor'@'localhost';  -- MariaDB 10.5.9 and newer instead
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';      -- MySQL Group Replication only
```

`PROCESS` shows other sessions (for long queries). Replication status needs `REPLICATION CLIENT` (MySQL) or, on MariaDB 10.5.9 and newer, `REPLICATION MONITOR: REPLICATION CLIENT` **alone is not enough there**. The password reaches the client only through its environment, never its command line.

containers: podman containers, root's and every user's

Finds the podman containers on the host by itself - you list nothing - and checks each one. Hosts without podman are skipped (`skipped: podman is not installed`, no finding). It is read-only: it runs `podman ps`, one `podman container inspect` and `systemctl show`, and nothing that starts, stops or creates anything.

Which containers. Root's (rootful podman) and each user's (rootless podman: only that user can see them). A user is looked at when a podman process runs as them, when they have `linger` on (a file in `/var/lib/systemd/linger/`), or when they are in `/etc/subuid` and have container storage in their home (`~/.local/share/containers/storage`). A user's containers are read **as that user** (`runuser -u USER -- env XDG_RUNTIME_DIR=/run/user/UID podman ...`), and only when `/run/user/UID` exists: otherwise podman would create it, and a check must not change anything. Such a user is listed as `not checked` instead (and reported, below).

Which should be running. A container should be running when a systemd unit runs it and starts it at boot (a Quadlet `.container` file with an `[Install]` section, or an enabled unit from `podman generate systemd` or written by hand), or when its restart policy is `always` or `unless-stopped`. A container that has neither and stopped with exit code 0 was stopped on purpose: no finding. A Quadlet or `--new` unit removes its container while the unit is stopped, so the check also reads the Quadlet files and unit files: a container that should run but does not exist at all is found too (state `missing`).

The output has a table of what was found:

OWNER RUN	NAME	STATE	UNIT	SHOULD
root	systemd-db	running	db.service (active)	yes
root	web	running	-	yes
alice	web	missing	web.service (failed)	yes
bob	web	exited (3)	-	no
not checked: carol - no /run/user/1003 (the user is not logged in and linger is off): ...				

Every finding names the owner, and its `look:` command runs as that owner (paste it as root):

Finding	Means	Usually
container X (owner 0) should be running (...) but it does not exist / it exited with code N (critical)	a container that should run is down, or its systemd unit failed	read the <code>look:</code> output (the unit's status, the container's log, the journal). Start it the right way: <code>systemctl start UNIT</code> for a unit (as a user: <code>runuser -u 0 -- env XDG_RUNTIME_DIR=/run/user/UID systemctl --user start UNIT</code>), <code>podman start X</code> for a plain container. Service watch does this for you, with approval
required container X ... does not exist/is not running (critical)	a container in <code>check_containers_required</code> is missing or stopped	create or start it; fix its name in the setting
container X (owner 0) is running but its healthcheck reports unhealthy (critical)	its own healthcheck fails	<code>podman inspect X</code> shows the last healthcheck output (<code>State.Health.Log</code>); the container's log

Finding	Means	Usually
container X (owner O) stopped with an error: it exited with code N, 2.5 hours ago (warning)	a container that is not meant to run all the time stopped with an error in the last <code>check_containers_recent_hours</code> (24) hours	its log. Older crashes are not reported
container X (owner O) has restarted N times - a restart loop (warning)	more restarts than <code>check_containers_restart_warn</code> (3), counted by podman or by its systemd unit	its log shows why it keeps stopping
container X (owner O) was killed by the out-of-memory killer (warning)	the kernel killed it for memory	more memory on the host, or a higher memory limit on the container
user U (uid N) has containers that should run (...) but linger is off (warning)	the user's containers stop when they log out and do not start at boot	<code>loginctl enable-linger U</code> (as root)
user U (uid N) has podman containers ... and N unit(s) that should start at boot, but they could not be checked: no /run/user/N (warning)	the user has units that should start at boot (a Quadlet with <code>[Install]</code> , an enabled unit) but is not logged in and has no linger: none of their containers run now. A user with only container storage (an admin who once ran podman) is listed under <code>not checked</code> : and gets no finding	if they should run: <code>loginctl enable-linger U</code> . If not, leave them out: <code>podman_discover_ignore: ['U/.*']</code>
the containers of O could not be read: podman ps failed ... (warning)	podman (or systemctl) failed for that owner: a blind spot	run the <code>look</code> : command to see the error (storage problems, <code>podman system migrate</code> needed after an upgrade ...)

Settings (roles `podman_discover` and `check_containers`; `VARIABLES_REFERENCE.md`):

Setting	Default	What it does
<code>podman_discover_enabled</code>	<code>true</code>	<code>false</code> = do not look for containers (the check says <code>skipped</code>)
<code>podman_discover_rootless</code>	<code>true</code>	also look at users' containers
<code>podman_discover_users</code>	<code>[]</code>	more users to look at, e.g. <code>[appsvc]</code> when the rules above miss one
<code>podman_discover_ignore</code>	<code>[]</code>	containers to leave out: regular expressions that must match the whole name or OWNER/NAME , e.g. <code>['test-.*', 'alice/.*', 'root/scratch']</code>
<code>check_containers_required</code>	<code>[]</code>	containers that must exist and run: a name (any owner) or OWNER/NAME , e.g. <code>[stigman-api, alice/web]</code>

Setting	Default	What it does
<code>check_containers_recent_hours</code>	24	how recent a crash must be to be reported
<code>check_containers_restart_warn</code>	3	restarts above this = a restart loop

To verify it on one host: run the Health check with `health_checks: containers` and a Limit of that host; compare the table with `sudo podman ps -a` and, for a user, `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/$(id -u alice) podman ps -a`. Service watch uses the same way of finding containers (SERVICE_WATCH_DEMO.md).

Troubleshoot (playbooks/troubleshoot.yml)

For "something is wrong on this host": pick the area and the job runs the commands an experienced admin runs first. Each is printed under a line saying what it shows, and the matching health checks run too. Read-only. Findings do not fail this job: you asked for a look, not a verdict.

Area	Commands (what they show)	Checks
overview	uptime, failed units, disk, memory, recent errors, recent reboots	disk, mounts, services, performance, time, logging
disk	space and inodes, biggest directories on the fullest filesystem, deleted-but-open files, files over 1 GiB, journal size	disk, mounts
performance	load, top, vmstat, memory users, disk latency (iostat), load history (sar), OOM kills	performance
service (<code>ts_service</code>)	its status, why it stopped (result, exit code, restarts), its log, dependencies, unit file, listening ports	services (plus that unit)
network (<code>ts_target</code> host:port)	addresses, routes, DNS servers, name lookup, TCP connect, route taken, firewall, connection counts	network (plus that target)
selinux	mode, today's denials, audit2why, changed booleans, local file labels	selinux
fapolicyd	status, today's denials, rules, enforcing?	fapolicyd
login	sshd log, faillock lockouts, failed logins, SSSD domain status, authselect, clock offset	accounts
time	chrony tracking and sources, timedatectl, configured servers	time
logs	errors, top error senders, rsyslog and its queue, journal size	logging
mariadb (or mysql)	(the MariaDB / MySQL health check)	mariadb

Add `use_act=true` and ACT reads everything collected and explains the root cause (ADDING_ACT.md).

Certificate report (playbooks/cert_report.yml)

The certs check on every host, and **one table**, soonest expiry first: days left, date, host, subject, and where the certificate is. Hand it to whoever renews certificates. It fails the hosts with a certificate inside `check_certs_warn_days`, so a workflow can open tickets.

POA&M status (playbooks/poam_status.yml)

Reads `poam/poam.csv` (see `poam/README.md` for the columns and how to export from eMASS):

Finding	Means
POA&M 1001 [CAT II]: ... - OVERDUE by 12 days	open item past its scheduled completion date (critical)
POA&M 1002 ...: due in 20 days	due within <code>poam_due_soon_days</code> (30)
... scheduled completion date "TBD" is missing or not a date	fix the date in the list
open CAT I finding V-123456 (...) on 2 asset(s) in collection Production has no POA&M item	STIG Manager cross-check: an open CAT I/II finding that no POA&M item covers

It also lists "maybe ready to close": items whose STIG findings are no longer open in STIG Manager.

The STIG Manager cross-check is optional. It needs a Keycloak client that may use the client-credentials grant, with the scope `stig-manager:collection:read`, and a grant in the collections it reads. **Your realm owner creates that client**; nothing here changes the realm. Then create the *STIG Manager API* credential (token URL `https://<keycloak>/realms/<realm>/protocol/openid-connect/token`, client ID, secret), and set `poam_stigman_api: https://<stigman>/api`.

ServiceNow tickets (playbooks/servicenow_tickets.yml)

A workflow step after a check. For each finding at or above `servicenow_min_severity`:

- **no open ticket** for that problem on that host › it opens an incident: short description `[site-health] host: summary`; description with the host, check, finding, the `look: command`, and ACT's analysis if ACT ran; urgency/impact from the severity; assignment group, category and configuration item from the settings.
- **an open ticket exists** › it adds a work note `Still failing at ... (servicenow_note_repeats)`.
- **a host that was targeted but never reported** › a ticket too (unreachable, login failed).
- **a problem that cleared** (its check ran again and passed) › a work note on its ticket, or it resolves it with `servicenow_resolve_cleared: true`.

Tickets are matched by a correlation ID (`site-health:<host>:<check>:<finding>`, hashed), never by text, so the same problem always updates the same ticket. Without the ServiceNow API credential it only prints what it would open. If no check results reach it at all (the check step died before checking any host), it fails, so a broken schedule shows red instead of green.

ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml)

Proves the ServiceNow setup end to end, with the same calls and settings as the tickets step. Six steps, each PASS, WARN or FAIL with what usually causes a failure:

Step	What it does	FAIL usually means
1 connection	logs in and reads one incident	certificate (CA), DNS, firewall or proxy; HTTP 401 = user or password; 403 = no <code>itil</code> role; 404 = a path after the host name in the Instance URL
2 settings	<code>servicenow_assignment_group</code> and <code>servicenow_caller</code> exist	the name differs from ServiceNow's
3 open	opens a low-priority incident: <code>[site-automation test] ... safe to close</code>	the account may not create incidents (<code>itil</code>), or ServiceNow refused a value (it says which)
4 read back	reads it back: number, state, group, caller; prints a link	the account may not read it; WARN if the group did not stick
5 work note	adds a work note	the account may not update incidents
6 resolve	resolves it (<code>servicenow_test_resolve: false</code> leaves it open)	WARN only: your instance needs another resolved state or close code (<code>servicenow_resolved_state</code> , <code>servicenow_close_code</code>)

As *Check* (a dry run) steps 1 and 2 run, and 3 to 6 are only printed. Set up, run and verify it: `SERVICENOW_SETUP.md`.

ServiceNow health (playbooks/servicenow_health.yml)

From the controller: does the instance answer its REST API, and how fast (`servicenow_slow_seconds`)? Is every MID Server **Up** and **validated**? Hourly is a good schedule. Alert with an AAP **notification** (email) on failure, not with a ticket: a broken ServiceNow cannot take tickets. The MID Server **hosts** (the Linux service, its keystore) are covered by the health check: see

`inventories/example/group_vars/servicenow_mid_hosts.yml`.

Patch hosts (playbooks/patch_hosts.yml)

Installs updates with dnf (`patch_security_only` for security fixes only; `patch_exclude` for packages never updated here). Then:

1. **One host at a time** (`patch_serial`). The rollout **stops at the first host that fails**.
2. Refuses with less than `patch_min_free_mb` free in `/var`.
3. **No automatic restarts** unless `automatic_restarts: true` (the survey or the template's Variables). Without it, a host that needs a reboot is listed ("reboot it by hand, or run again with `automatic_restarts: true`"). With it, the job reboots only when `needs-restarting -r` says so (`patch_reboot: when_needed`), and **never** hosts in `patch_never_reboot_groups` (`aap_hosts`).
4. After the update (and reboot), **every service that was running before must be running again**. If one is not, the host fails and the rollout stops.
5. Never patches hosts in `no_patch` (vendor appliances), `aap`, `aap_hosts`, `sat`, `netapp` or `netapp_console_hosts`. The AAP host is **always** protected: even a list you write yourself (`patch_never_patch_groups`) cannot remove `aap` or `aap_hosts`. It also refuses a host that is the machine running the job, but only when the connection is `local` or the name is `localhost`; a normal SSH host that happens to be the controller is stopped by the group rule above, not by this one.

Run it as **Job type: Check** first: it lists what would be updated and changes nothing (DRY_RUNS.md).

Apply approved ACT fix (playbooks/act_fix_approved.yml)

The step after an approval in the *Fix with approval (ACT)* workflow. It runs **exactly** the commands ACT proposed and a person approved, then re-runs the checks that found the problem. It refuses to run on its own, and skips diagnose-only hosts (`aap_hosts`, `netapp_console_hosts`): there a person applies the fix by hand. See ADDING_ACT.md.

Windows hosts

Windows has its own playbooks, checks and troubleshooting areas (for example `updates` and `security` in *Windows troubleshoot*). They are described, with their settings, in WINDOWS.md. Settings for them are read the same way as any other: VARIABLES.md.

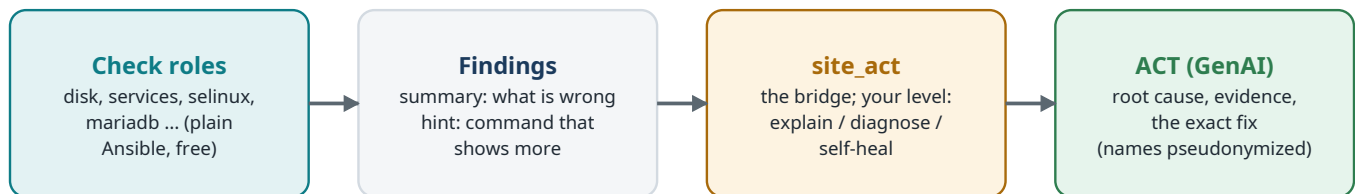
Not built yet (and why)

Idea	Why not yet
SCAP scan › import into STIG Manager	Needs the same Keycloak client as the POA&M cross-check, plus a decision on SCC vs OpenSCAP content. STIG Manager's own importer (stigman-watcher) is the supported path
Certificate renewal workflow	Depends on how your PKI issues certificates. For STIG Manager, renewal already works: update the TLS credential, run <i>STIG Manager - deploy</i>
AAP self-health (job failure rate, execution environment and token age)	Needs an AAP API credential; AAP's own dashboards cover most of it today
Container runbooks (image updates with rollback, drift)	Deferred on purpose; <i>Service watch</i> covers container outages
MariaDB backups, security baseline, password rotation	Out of scope for now: health only, as asked

8. Adding ACT (GenAI) later

The runbooks work without ACT. Add it when the checks are running well and you want the next step done for you: **why** it broke and **what** fixes it. This page shows how the runbooks and ACT fit together, how to turn it on per template, and how to write your own check so it works with ACT automatically.

How they fit: checks find WHAT, ACT finds WHY



Report, ServiceNow tickets and the approval workflow read the same findings - with or without ACT.

Checks find WHAT is wrong; ACT finds WHY. A new check works with ACT automatically through its findings' hints.

- The **checks** decide *whether* something is wrong. They are plain Ansible: the same answer every time, and they are free. The model is only called for hosts with findings.
- Every finding carries a **hint**: the read-only command a person would run next. ACT gets those commands' output as evidence. A new check you write is therefore usable by ACT with no extra work.
- `roles/site_act` is the only place that talks to ACT. Every runbook uses it the same way. Turning ACT on is a variable, not a code change.
- Host names, IP addresses and user names are **pseudonymized** before anything reaches the model (ACT 0.6.18 and newer). Its replies are translated back on the host.

The three levels

Level	ACT does	Changes to hosts	Approval step?
explain	reads the evidence (the hint commands' output, collected by Ansible) and writes the root cause and a suggested fix. It runs no commands itself	none	no
diagnose	investigates by running read-only commands itself, then proposes exact fix commands. Every change is refused and waits for a person	only after approval	yes: the job fails with NEEDS APPROVAL › approval step › <i>Apply approved ACT fix</i>
self-heal	like diagnose , but may run fixes that match <code>site_act_allow</code> by itself (for example <code>systemctl restart chronyd</code>). Anything else still waits for approval. The checks then run again to prove the fix worked	only fixes you listed	only for the other fixes

Start with **explain**. Move a runbook to **diagnose** when ACT's explanations have been right for a while. Use **self-heal** only for fixes you would let a junior admin do unasked: start a stopped service, restart a hung

one.

Never fixed automatically, at any level: hosts in `site_act_diagnose_only_groups` (default: `aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`; `aap` and `aap_hosts` are always kept, whatever list you write). There `self-heal` becomes `diagnose`, and the apply step skips them: it prints the approved command for a person to run by hand.

Turn it on

1. The ACT model key credential

Create the credential type `ACT model key` (`aap/credential_types/act_model_key.yml`) and one credential with your key: the GenAI key for GenAI.mil, or the AskSage key for Ask Sage (SETUP_AAP.md, steps 2-3). For the GenAI beta proxy, also create `ACT GenAI beta key` (`act_genai_beta_key.yml`). Attach the credential(s) to the **Health check** and **Troubleshoot** templates (and *Service watch - check*), next to `Linux ssh (sudo)`. This is the only place a key lives. The apply steps do not call the model: they run the approved commands themselves, so they need no key.

2. Pick the model provider

ACT can talk to three providers. Pick one with `site_act_provider`:

site_act_provider	Provider	Key (credential)	Also set
<code>genai</code> (default)	GenAI.mil	<i>ACT model key</i> , GenAI API key field	nothing (optional: <code>site_act_model</code>)
<code>asksage</code>	Ask Sage	<i>ACT model key</i> , AskSage API key field	<code>site_act_url</code> : your organization's Ask Sage API URL. The default is <code>https://api.genai.army.mil/server/openai/v1/chat/completions</code>
<code>genai-beta</code>	GenAI.mil beta proxy (preview models)	<i>ACT GenAI beta key</i> (<code>aap/credential_types/act_genai_beta_key.yml</code>)	nothing

The settings:

Setting	What	Default
<code>site_act_provider</code>	<code>genai</code> , <code>asksage</code> or <code>genai-beta</code>	<code>genai</code>
<code>site_act_models</code>	the model for each provider , so the model follows the provider you pick, e.g. <code>{genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov}</code>	ACT's default for the provider
<code>site_act_urls</code>	the API URL for each provider, e.g. <code>{asksage: https://<host>/server/openai/v1/chat/completions}</code>	ACT's default for the provider
<code>site_act_model</code> , <code>site_act_url</code>	one model / URL for this run, whatever the provider (a survey answer)	empty: use the two above
<code>site_act_ca</code>	a CA bundle file on the hosts , if the provider's certificate comes from a CA the hosts do not trust yet (for example a DoD CA not in <code>/etc/pki/ca-trust</code>)	the host's normal trust store

Where to set them:

- **For everyone:** `playbooks/group_vars/all.yml` in this project (the file has these lines ready as comments). A setting there beats the *inventory's* Variables box in AAP, so put it in one of the two places, not both. Which place wins in every case: `VARIABLES.md`.

YAML

```
site_act_provider: genai          # the default provider
site_act_models:                 # the model follows the provider you pick
  genai: gemini-3.8-flash
  asksage: gpt-5.6-sol-gov
site_act_urls:                   # only if your Ask Sage is not ACT's default URL
  asksage: https://<your ask sage host>/server/openai/v1/chat/completions
```

To make Ask Sage the default, change the first line to `site_act_provider: asksage`.

- **For one group:** the group's settings file (`playbooks/group_vars/<group>.yml`), or the group's **Variables** box in AAP. For example `site_act_provider: asksage` there, and only that group's hosts use Ask Sage (with its model from `site_act_models`).
- **For one template:** its **Variables** box (*Extra variables*), for example `site_act_provider: asksage`. It beats everything above, for every run of that template.
- **Per launch:** the survey questions in step 3 (a survey answer beats everything too).

The API **key** is never a variable: it comes from the *ACT model* key credential on the template.

These settings apply to every runbook that uses ACT, including *Service watch* and *Apply approved ACT fix*. If the key for the chosen provider is missing, ACT does not run: the job says `ACT did not run: this job has no API key for provider asksage. Attach ...`, and the findings are still reported.

Network: ACT runs **on each host**, not in AAP. So each host must be able to reach the provider's URL over HTTPS. If they go through a proxy, add it: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}`.

Models served only on `/v1/messages` (ACT 0.6.19 and newer). Some gateways offer a model only through the Anthropic endpoint and answer HTTP 400 for it on `.../v1/chat/completions`. ACT tries the other endpoint by itself, with nothing to set. To test your models and set the switches on purpose, follow Both endpoint formats and `:probe` below.

Anything else ACT should know about your site goes in the same places (for example `playbooks/group_vars/all.yml`):

YAML

```
site_act_extra_instructions: >-
  RHEL 9 servers under DISA STIG. fapolicyd and SELinux are enforcing. Never suggest disabling
  them.
```

3. Survey questions

On **Health check** and **Troubleshoot** › **Survey** › **Create survey question**:

Question	Variable	Type	Choices	Default
Ask ACT (GenAI) about the findings?	<code>use_act</code>	Multiple Choice (single select)	<code>no, yes</code>	<code>no</code>

Question	Variable	Type	Choices	Default
What may ACT do?	<code>site_act_level</code>	Multiple Choice (single select)	<code>explain</code> , <code>diagnose</code> , <code>self-heal</code>	<code>explain</code>
Which model provider? (optional)	<code>site_act_provider</code>	Multiple Choice (single select)	<code>genai</code> , <code>asksage</code> , <code>genai-beta</code>	the one in your settings
Which model? (optional, blank = the provider's model from <code>site_act_models</code>)	<code>site_act_model</code>	Text, not required		blank

Leave out the last two if everyone uses the same provider. Your setting (section 2) then applies.

4. Try explain by hand

Launch **Troubleshoot** on a host with a known problem: `ts_area = service`, `ts_service = the unit`, `use_act = yes`, `site_act_level = explain`. In the output, read `ACT | what ACT says`. With ACT on, the host's ServiceNow ticket includes this analysis too.

5. diagnose with the approval workflow

Build *Fix with approval (ACT)* (WORKFLOWS_AND_SCHEDULES.md). What happens:

1. The health check finds, for example, `required service chronyd.service is inactive`.
2. ACT (`diagnose`) looks at the unit, its log and the config, and proposes `systemctl start chronyd`. The command is refused, so the job fails: `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at the approval step. The approver reads ACT's report and approves.
4. **Apply approved ACT fix** runs **exactly** `systemctl start chronyd` itself: no model and no key, so nothing can be reworded between the approval and the fix. A last check refuses a few commands even after approval (reboot, `mkfs`, `usermod` ...; see APPROVED_COMMANDS.md).
5. The same checks run again, without ACT, and the job is green only if the problem is gone. The tickets step notes the ticket as cleared.

6. self-heal for the fixes you trust

```
playbooks/group_vars/all.yml (or the settings file of one group)

site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Each entry is a regular expression that must match the **whole** command. Keep them narrow: name the services, never `.*`. Then pick `site_act_level = self-heal`. After ACT applies an allowed fix, the checks with findings run again. The report shows `cleared: ...` or `still there: ...` and reflects the state **after** the fix.

Both endpoint formats and :probe (ACT 0.6.19 and newer)

What this is for. A model gateway can offer a model on two web addresses: the *OpenAI* one (the provider URL, ending in `/v1/chat/completions`) and the *Anthropic* one (ending in `/v1/messages`). Some models answer on only one of them. If ACT prints `HTTP 400 ... bad request` for a model that the gateway lists, this is the usual cause. ACT can now speak both, and `:probe` tests which one each model needs.

Two facts that shape the steps:

- `:probe` is a command you type at the ACT prompt. **It cannot run inside an AAP job** (jobs are not interactive). So you run it **once, by hand, on one lab host**, and then put the result into settings the jobs read.
- In a job, ACT already works out the format by itself (one refused request, then it switches), so the jobs may need no change at all. The settings below only save that one wasted request per run, or point at an Anthropic address that is not the default.

Not verified against a real gateway. These steps were written from ACT's own documentation and tests. Nobody has run them against your gateway yet; treat the first run as the test.

Step 1: check which ACT the project carries

`vendor/act/VERSION` must say `0.6.19` or newer. This release carries **0.6.21**, a hardening release: it sends the key only to an `https://` provider URL (an `http://` URL is refused unless `site_act_env: {ACT_ALLOW_HTTP: "1"}`), and more dangerous commands always need a person (see `APPROVED_COMMANDS.md`). If it is older, update it first: README section "Updating ACT" (`scripts/update-act.sh`).

Step 2: put ACT on one lab Linux host

Any host that can reach the gateway will do; it does not have to be one of your managed hosts. Copy the file `vendor/act/act` there (WinSCP, or `scp` in PowerShell), then on the host:

```
shell
chmod 700 act
```

Step 3: give it your key without writing the key anywhere

Type this on the host. It asks for the key without showing it, and keeps it only in that terminal session (use `GENAI_BETA_KEY` or `ASKSAGE_KEY` for the other providers):

```
shell
read -rs GENAI_KEY; export GENAI_KEY
```

Paste the key, press Enter, and do not `echo` it. If the provider URL is not ACT's default, also `export GENAI_URL='https://<your gateway>/v1/chat/completions'`.

Step 4: run `:probe`

```
shell
./act
```

At the ACT prompt:

```
:probe all
```

`:probe all` tries every model the key lists, on both endpoints, with a small request and then ACT's full request. For each it shows what worked and, for a refusal, **the gateway's own reason**. `:probe <model name>` tests just one. Write down, for each model, which endpoint worked (`openai` or `anthropic`). Leave ACT with an empty line (or Ctrl-D).

The result is remembered for the session. If a setup file already exists for your user on that host (`~/.config/act/config.json`), ACT also saves the format there (only the format table, never the key).

Windows. ACT for Windows (`vendor/act-windows/act.ps1`) has the same `:probe` command. Run it the same way on a lab Windows host, in PowerShell, and enter the key with `:setup`. Its result goes into the Windows jobs' settings as above. (Not run by hand for this page.)

Step 5: tell the jobs

A job does not read the lab host's file (it runs as another account, on another host), so put the result in settings, in the same places as the other ACT settings (section 2):

Your probe result	Setting in playbooks/group_vars/all.yml (or a group file)
All the models you use work on the OpenAI endpoint	nothing to set
Some models work only on the Anthropic endpoint	nothing to set (ACT learns it during the run); optional, to skip the one refused request per run when every model you use is Anthropic-only: <code>site_act_env: {ACT_API_FORMAT: anthropic}</code>
The Anthropic endpoint is at a different address than the OpenAI one with <code>/messages</code> in place of <code>/chat/completions</code>	<code>site_act_env: {GENAI_ANTHROPIC_URL: "https://<host>/.../v1/messages"}</code> (<code>ASKSAGE_ANTHROPIC_URL</code> or <code>GENAI_BETA_ANTHROPIC_URL</code> for the other providers)
A model you want is listed for the key but works on neither	it is not available to your key; ask the gateway's owner. Choose another model in <code>site_act_models</code>

`ACT_API_FORMAT: anthropic` applies to **every** model ACT uses in the job. If you use both kinds, leave it out.

How to verify

1. **The probe result:** in step 4 every model you plan to use shows one endpoint that worked.
2. **A job:** launch **Troubleshoot** with `use_act = yes`, `site_act_level = explain`, and the model in `site_act_model` (or set through `site_act_models`) on a host with a known finding. In the output, task `ACT | what ACT says` holds an analysis, not an `HTTP 400` message.
3. **The old symptom is gone:** search the job output for `400`. A line saying ACT switched to the other endpoint is fine; a final `HTTP 400` for the model is not (then the probe said neither endpoint works: step 5, last row).

If a step fails, the message after `HTTP 400` is the gateway's reason: copy it exactly when you ask its owner for help, but remove the key and host names first.

Write your own check (it works with the report, tickets and ACT automatically)

A check is a role named `check_<name>` that **adds findings** to `site_findings`. That is the whole contract. As an example, here is a check that `/tmp` is not world-writable without the sticky bit.

1. Create `roles/check_tmp/defaults/main.yml`, with its settings and a comment saying what it checks:

```
YAML
---
# TMP: /tmp and /var/tmp must have the sticky bit (mode 1777).
check_tmp_paths: [/tmp, /var/tmp]
```

2. Create `roles/check_tmp/tasks/main.yml`. Collect with read-only commands, and always use `changed_when: false` and `check_mode: false`: read commands then run in a dry run too. Turn what you

saw into findings, and add them:

YAML

```
---
- name: tmp | modes
  ansible.builtin.command:
    argv: "{{ ['stat', '-c', '%a %n'] + check_tmp_paths }}"
  register: _tmp
  changed_when: false
  failed_when: false
  check_mode: false

- name: tmp | findings
  ansible.builtin.set_fact:
    _tmp_findings: >-
      {%- set f = [] -%}
      {%- for line in _tmp.stdout_lines if line.split()[0] != '1777' -%}
      {%-   set path = line.split()[1] -%}
      {%-   set _ = f.append({'check': 'tmp', 'id': 'tmp:' ~ path, 'severity': 'warning',
                            'summary': path ~ ' has mode ' ~ line.split()[0] ~ ' (expected
                            1777)',
                            'hint': 'stat ' ~ path ~ '; findmnt ' ~ path}) -%}

      {%- endfor -%}
      {{ f }}

- name: tmp | add them to this host's findings
  ansible.builtin.set_fact:
    site_findings: "{{ site_findings | default([]) + _tmp_findings }}"
```

3. Let the health check know it: in `playbooks/health_check.yml`, add `tmp` to `_health_sets` (to `weekly` and `all`), and add `tmp` to the survey choices of the Health check template.
4. Test it on one host: `ansible-playbook playbooks/health_check.yml -l <host> -e health_checks=tmp`. Then break the condition on purpose (`chmod 777 /tmp` on a test host) and check that the finding appears. A check that never fires proves nothing.

That is all. The report prints it, `set_stats` publishes it, ServiceNow tickets it (the `id` keeps it on one ticket), and with `use_act=yes` ACT receives the summary and runs the `hint` as evidence.

Rules for a good finding:

- **summary**: one line a person understands, with the number and the limit (`/var is 96% full (critical at 95%)`).
- **id**: stable across runs, with no numbers that change (`disk:/var`, not `disk:/var:96`). Otherwise every run opens a new ticket.
- **hint**: **read-only**, runs in under a minute (`timeout 60 ...`), and shows the cause, not just the symptom. It runs as root when ACT is on (Ansible runs it in `explain` mode and hands ACT the output), so never put a command that changes something there.
- **severity**: `critical` = act now (outage, security); `warning` = act soon.

Use ACT in a playbook of your own

Any playbook that fills `site_findings` can hand them to ACT the same way:

YAML

```
- name: My check
  hosts: my_hosts
  become: true
  gather_facts: false
  tasks:
    - ansible.builtin.include_role: {name: site_findings, tasks_from: start.yml}
    - ansible.builtin.include_role: {name: site_findings, tasks_from: run_check.yml}
      vars: {_site_check: tmp}                                # runs roles/check_tmp
    - ansible.builtin.include_role: {name: site_act}
      when: use_act | default(false) | bool and site_findings | length > 0
    - ansible.builtin.include_role: {name: site_findings, tasks_from: report.yml}
```

`report.yml` goes last. It prints, publishes (for tickets and the approval workflow), and fails the host when there are findings or a fix waits for approval.

What ACT is told, and what it cannot do

- Its task: the findings (one line each), "find WHY, confirm it with evidence, give the exact fix; several findings may share one cause", plus your `site_act_extra_instructions`.
- `explain`: it runs nothing. Ansible ran the hint commands.
- `diagnose`: only commands ACT can prove are read-only run on their own. Everything else is refused and becomes a proposal.
- `self-heal`: in addition, only commands matching `site_act_allow`.
- Apply step: no model at all. The playbook runs only the approved commands, character for character, in order, and stops at the first one that fails. Run as *Check*, it only shows what it would run (a dry run: see WORKFLOWS_AND_SCHEDULES.md, "Rehearse it").
- Every run is recorded in the AAP job output. The ticket carries ACT's analysis. For the service-watch style incident log, see SERVICE_WATCH_DEMO.md.

9. Windows servers

The Windows runbooks work the way the Linux ones do. Checks find what is wrong and say it in plain words, with a command to look further. The same report, the same ServiceNow tickets and POA&M, and the same workflows and schedules follow. Only the checks themselves are different: each one is a PowerShell script that runs on the server over WinRM.

Nothing extra to install on AAP or on the servers. The playbooks use only Ansible's built-in `script` module, so no Ansible collection is needed. The execution environment needs the Python library for WinRM (`pywinrm`); step 2 shows how to check. The servers need WinRM over HTTPS, which you already use. The check scripts run in Windows PowerShell 5.1 and PowerShell 7, and also in Constrained Language Mode (see step 5).

Template	Playbook	What it does
Windows connection test	<code>playbooks/win_connection_test.yml</code>	run it first: can AAP reach the servers, with the right account?
Windows health check	<code>playbooks/win_health_check.yml</code>	the checks below; daily and weekly sets
Windows troubleshoot	<code>playbooks/win_troubleshoot.yml</code>	on demand: an admin's first PowerShell commands for one kind of problem
Windows certificate report	<code>playbooks/win_cert_report.yml</code>	every certificate on every server, soonest expiry first
Windows patch	<code>playbooks/win_patch.yml</code>	install what Software Center offers (ConfigMgr); restart only with <code>automatic_restarts: yes</code> ; prove the server is back
ACT for Windows - install	<code>playbooks/win_act_install.yml</code>	put ACT on the servers in <code>C:\ProgramData\act</code> , locked down and unblocked; keep it current
ServiceNow tickets, POA&M status	the ones you have	unchanged: they take the Windows results the same way

The checks (`roles/win_check_<name>`):

Set	Check	Finds
daily	<code>disk</code>	volumes too full
daily	<code>services</code>	required services not running; automatic services that stopped with an error; services that keep crashing
daily	<code>performance</code>	CPU, memory or page file too busy; names the busiest processes
daily	<code>time</code>	Windows Time not running, not synchronized, synced too long ago, clock off
daily	<code>network</code>	no gateway or DNS; names that do not resolve; ports that do not answer; a broken domain trust

Set	Check	Finds
daily	<code>eventlog</code>	unexpected shutdowns, crashes, disk errors, a cleared Security log, many errors, full logs
weekly	<code>security</code>	Defender real-time protection off or old signatures; endpoint services down; firewall off; SMBv1; RDP without NLA; UAC off; TLS 1.0/1.1
weekly	<code>audit</code>	audit policy settings the STIG asks for; event log sizes; the log forwarding agent
weekly	<code>accounts</code>	Guest enabled; local passwords old or never expiring; inactive local accounts; unexpected local Administrators
weekly	<code>certs</code>	certificates that expire or expired, including the WinRM listener AAP connects through
weekly	<code>patching</code>	a restart pending for installed updates; days since the last update

Everything is read-only. The one write is one Application event log entry per finding (source `site-health`), so a SIEM that collects event logs sees the findings too.

Step 1. WinRM over HTTPS on the servers

You already use it. To see it on a server:

```
winrm enumerate winrm/config/listener
```

It shows a listener with `Transport = HTTPS`, `Port = 5986` and a `CertificateThumbprint`. The firewall must let port 5986 in from the AAP execution nodes. The `certs` check warns before that listener's certificate expires: when it expires, AAP can no longer connect.

Step 2. The execution environment

The job runs inside an execution environment (EE), and that EE needs these Python libraries:

- `pywinrm`, always;
- `requests-ntlm`, for NTLM (the usual choice);
- or `requests-kerberos` plus a Kerberos client (`kinit`), for Kerberos.

AAP's *Default execution environment* has them. Your own EE (for example `platforms-ee`) may not. **The Windows connection test tells you:** its first part runs inside the EE and prints `YES` or `no` for each library. If `pywinrm` is missing, you have two options:

- use AAP's Default execution environment for the Windows templates; or
- add the libraries to your EE's definition and rebuild it:

```
YAML
dependencies:
  python:
    - pywinrm
    - requests-ntlm
```

Step 3. The credential

Automation Execution > Infrastructure > Credentials > Create credential:

- **Name:** `Windows (WinRM)`; **Credential type:** `Machine`.
- **Username:** `YOURDOMAIN\svc_aap` for NTLM. For Kerberos, use `svc_aap@YOURDOMAIN.MIL`, with the domain in capitals.
- **Password:** the account's password.
- **Privilege escalation:** leave it empty. An administrator's WinRM session is already elevated.

The account must be in the **local Administrators group** on every server, usually through a domain group added by GPO. The checks need that to read the Security log, the audit policy, local accounts and certificate stores.

Step 4. The Windows inventory

Keep the Windows servers in their **own inventory**. Go to **Inventories > Create inventory**, name it `Windows servers`, and paste this in its **Variables** box:

```
YAML
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_scheme: https
ansible_winrm_transport: ntlm # or kerberos
ansible_winrm_server_cert_validation: validate # needs your CA bundle in the EE, see below
ansible_winrm_ca_trust_path: /etc/pki/ca-trust/source/anchors/your-ca.pem # path inside the EE
```

- **Certificate validation.** `validate` means the EE checks the server's WinRM certificate against the CAs it trusts. EEs usually do not include your organisation's root CAs, so `validate` fails with `CERTIFICATE_VERIFY_FAILED` until you add your CA bundle to the EE and point `ansible_winrm_ca_trust_path` at it (the path inside the EE, as shown above). **Use this setting for real servers.** `ignore` keeps the traffic encrypted but does not check who the server is, so anyone who can intercept the connection could pose as it: use it **only** for a short test on a lab network, then switch back.
- **Hosts.** Add them by full name (`web01.yoursite.mil`); Kerberos needs the full name.
- **Groups.** Use whatever helps: by site, by role (`win_dc`, `win_web...`). The settings below can be set per group. `inventories/example-windows/hosts.yml` shows an example.

Linux and Windows do not mix: the Linux templates skip Windows servers, and the Windows templates skip Linux servers. So a template pointed at the wrong inventory changes nothing and opens no tickets.

Step 5. Test the connection

Create a job template, then launch it with **Limit** set to one server:

Field	Value
Name	<code>Windows connection test</code>
Inventory	<code>Windows servers</code>
Project	<code>site-automation</code>
Playbook	<code>playbooks/win_connection_test.yml</code>

Field	Value
Execution environment	one with <code>pywinrm</code> (step 2)
Credentials	<code>Windows (WinRM)</code>
Limit	empty, Prompt on launch ticked

A good result ends with:

```
Connected: WEB01 (Microsoft Windows Server 2022 Standard, build 20348), domain yoursite.mil
PowerShell 5.1.20348.2849, FullLanguage
Logged on as yoursite\svc_aap; local administrator: yes
OK: the Windows runbooks can run here.
```

`ConstrainedLanguage` instead of `FullLanguage` means WDAC or AppLocker restricts PowerShell on that server. The check scripts are written and tested for that mode; only two optional parts are skipped in it (TLS endpoints and the search for missing updates). Whether Ansible itself can run on such a server depends on your ansible-core version and the policy: if this test got through and printed its result, it can.

Step 6. The job templates

For all of these, use: **Inventory** `Windows servers`, **Project** `site-automation`, the EE from step 2, **Credentials** `Windows (WinRM)`, **Limit** empty with **Prompt on launch** ticked. Leave **Variables** empty: with no `target`, they run on every server in the inventory.

Windows health check

Field	Value
Playbook	<code>playbooks/win_health_check.yml</code>
Survey: <code>health_checks</code>	<i>Multiple Choice (multiple select):</i> <code>daily, weekly, all, disk, services, performance, time, network, eventlog, security, audit, accounts, certs, patching</code> . Default <code>daily</code>

Windows troubleshoot

Field	Value
Playbook	<code>playbooks/win_troubleshoot.yml</code>
Survey: <code>ts_area</code>	<code>overview, disk, performance, service, network, login, time, logs, updates, security</code> . Default <code>overview</code>
Survey: <code>ts_service</code>	<i>Text</i> , not required. "Service name (for area = service), e.g. W3SVC"
Survey: <code>ts_target</code>	<i>Text</i> , not required. "host:port it cannot reach (for area = network)"
Survey: <code>ts_hours</code>	<i>Integer</i> , default <code>2</code> . "How many hours of event logs to read"

Each command's output appears under a `### what it shows` line, followed by the matching health checks. Findings do not fail this job.

Windows certificate report

Field	Value
Playbook	<code>playbooks/win_cert_report.yml</code>
Survey	none

Step 7. Workflows and schedules

These are the same as for Linux, with the Windows templates. The full list, with every step and link, is in `WORKFLOWS_AND_SCHEDULES.md` (workflows 12 to 17):

Workflow	Steps	Schedule
Windows daily health	Windows health check (daily) › ServiceNow tickets (Run always)	every day 06:15
Windows weekly compliance	Windows health check (weekly) › tickets; Windows certificate report › its own tickets box (both Run always)	Monday 07:15
Windows patch with a dry run first	Windows health check (disk, services) › Windows patch as Check › <i>Approval</i> › Windows patch › Windows health check (daily) › tickets	monthly, inside the servers' ConfigMgr maintenance window

The ServiceNow tickets template runs on AAP itself, so it stays as it is.

Start small: the pilot order

Everything Windows here is new, so try it on **one** server that does not matter much, in this order, and widen only when each step looks right:

1. **Windows connection test**: the account, WinRM and PowerShell work.
2. **Windows health check**: the findings make sense for that server.
3. **Windows patch as Check** (the dry run): it lists what Software Center would install.
4. **Windows patch for real**, inside that server's ConfigMgr maintenance window.
5. **ACT for Windows - install** on the same server (tell your security team first, see below).

Every step can be a dry run first: `DRY_RUNS.md`.

Patching through Software Center

`playbooks/win_patch.yml` does, on each server, what an admin does in Software Center: install every software update your ConfigMgr admins deployed to it ("Install all"), required and available alike. AAP decides nothing about *which* updates: ConfigMgr does, as it does today. Applications listed in Software Center are not installed; only software updates are. Then:

1. **No automatic restarts** unless you ask for them with `automatic_restarts: yes` (the survey, or the template's Variables). Without it, the job installs the updates and lists the servers that need a restart: **A RESTART IS NEEDED to finish the updates ... automatic restarts are off**. A person restarts them (the post-check's `patching` finding keeps them visible). With it, the job restarts a server only when the updates or the ConfigMgr client say so, and never one in `win_patch_never_reboot_groups`.

2. **Prove the server is back** (after a restart, and also when none was needed). A new boot time, and every automatic service that ran before runs again. Delayed services get `win_patch_services_grace_min` (5 minutes) to start.
3. **Look again.** After a restart, servicing-stack and cumulative updates often reveal more updates. There is a second round (`win_patch_max_rounds: 2`), only when it restarted.

Safety.

- One server at a time, and the rollout stops at the first server that fails.
- By default it only patches the group `win_patch_hosts`. Set `target:` in the template's Variables (for example `target: all`) and pick the servers with the Limit at launch.
- Servers in `win_patch_never_patch_groups` (default `no_patch`) are skipped.
- **Maintenance windows still apply.** Outside the server's ConfigMgr maintenance window, ConfigMgr holds the updates. The job then stops with `ConfigMgr will not install these now ... (waiting for a maintenance window)` instead of waiting forever. Run it inside the window, or ask the ConfigMgr admin for a window that covers the patch time.
- **A dry run first:** run the template as *Check*. It rescans and lists what it would install, and changes nothing (DRY_RUNS.md). The *Windows patch with a dry run first* workflow puts that list in front of the approver.

Needs: the ConfigMgr client (Software Center) on the servers, and full PowerShell (not Constrained Language Mode) for the install call. Restarts use `shutdown.exe`, and AAP watches the WinRM port go down and come back. No Ansible collection is needed.

Scheduling. With `target: all` and no Limit, a scheduled run patches every server in the Windows inventory, one after another. Choose that on purpose. Otherwise, make one schedule per group (for example the test servers the week before the others), each with its own Limit.

The job template:

Field	Value
Name	<code>Windows patch</code>
Playbook	<code>playbooks/win_patch.yml</code>
Credentials	<code>Windows (WinRM)</code>
Job type	Run, with Prompt on launch ticked (Check = the dry run)
Variables	<code>target: all</code> (or a group), and pick servers with the Limit
Survey: <code>automatic_restarts</code>	"Restart servers automatically when the updates need it?" Choices <code>no</code> , <code>yes</code> . Default <code>no</code>

If a server fails:

What you see	What to do
<code>has no working ConfigMgr client</code>	the SMS Agent Host service (CcmExec) is missing or stopped
<code>waiting for a maintenance window</code>	run inside the window, or ask the ConfigMgr admin

What you see	What to do
these updates failed: ... (error 0x...)	C:\Windows\CCM\Logs\UpdatesDeployment.log and WUAHandler.log on the server
still not done after 120 minutes	Software Center on the server; C:\Windows\CCM\Logs\UpdatesHandler.log
did not accept a logon after the restart	look at the server's console
... ran before patching and do not run now	the service did not come back: <code>Get-Service <name></code> , System log

ACT on the servers (C:\ProgramData\act)

playbooks/win_act_install.yml puts ACT for Windows (`vendor/act-windows/act.ps1`) on the servers in `C:\ProgramData\act` (setting `win_act_dir`). Run it again after you update this repository, and every server gets the new version.

- **Locked down.** Ordinary users may create folders and files in `C:\ProgramData` (and in `C:\temp`). Someone could then place, or swap, `act.ps1` before an administrator runs it, and their code would run with administrator rights. So the job:
 - takes ownership of `C:\ProgramData\act`;
 - lets only Administrators and SYSTEM in (nothing inherited from `C:\ProgramData`);
 - checks the file's SHA256 against the one in the repository, every run, and replaces it if it differs.
- **Unblocked.** The file is written on the server itself, so Windows never marks it as "downloaded from the internet", and the job runs `Unblock-File` on it anyway. Every run also checks it is not blocked, and fixes it if it is.
- **Using it on a server:** an administrator opens PowerShell and runs `powershell -ExecutionPolicy Bypass -File C:\ProgramData\act\act.ps1`. The first time, `:setup` asks for their own GenAI or Ask Sage key and keeps it in their profile. No key is put on the server for everyone.
- **Dry run:** run as `Check` and it says what it would install or fix. To remove it: `win_act_state: absent`.
- **Tell your security team before the first run.** The installer is a large PowerShell script that writes a `.ps1` file, and some endpoint-protection products (Defender for Endpoint, Trellix) flag that pattern. Let them know it is coming from AAP, and try one server first.
- **Versions and `:probe`.** ACT for Windows has the same endpoint formats and `:probe` command as the Linux one (see `ADDING_ACT.md`); the copy in `vendor/act-windows/` is refreshed when the repository's vendored ACT is. On Windows, an unattended run never approves the danger tier of commands: only a person at the prompt can. See `APPROVED_COMMANDS.md`.
- ACT itself needs full PowerShell. Where WDAC or AppLocker enforce Constrained Language Mode, ACT's own notes (sign it with your code-signing certificate) apply.

Settings: what each check looks at, and how to change it

Every setting is in `roles/win_check_<name>/defaults/main.yml`, with a comment saying what it does. To change one:

- **for all Windows servers,** use the `Windows servers` inventory's **Variables** box;

- **for a group**, use the group's **Variables** box, or a settings file named after the group (for example `playbooks/group_vars/win_web.yml` for a group called `win_web`);
- **for one server**, use the host's **Variables** box.

The settings you will most likely want:

```
network: what every Windows server must resolve and reach

win_check_network_dns_names: [dc01.yoursite.mil]
win_check_network_tcp:
  - {name: LDAPS, host: dc01.yoursite.mil, port: 636}
  - {name: SIEM, host: siem.yoursite.mil, port: 9997}

# security: your endpoint protection's services (for example Trellix / McAfee)
win_check_security_av_services: [masvc, mfemms]

# audit: your log forwarding agent
win_check_audit_forwarder_services: [SplunkForwarder]

# accounts: who may be a local administrator (wildcards allowed)
win_check_accounts_admins_allowed: ['*\Administrator', 'YOURDOMAIN\Domain Admins',
  'YOURDOMAIN\svc_aap']

# services: more services that must run, on a group (for example IIS servers)
win_check_services_required_extra: [W3SVC, WAS]
```

A few defaults worth knowing:

- **disk**: warning at 85% used, critical at 95%. Per volume: `win_check_disk_overrides: {'D:': {warn: 92, crit: 98}}`.
- **services**: `EventLog`, `RpcSs`, `Dnscache`, `W32Time`, `WinRM` and `mpssvc` (the firewall) must run. Automatic services that stop cleanly by design are not reported.
- **security**: Defender is checked only when it is the active antivirus (`win_check_security_av: auto`). With another product, list its services instead.
- **certs**: warning 30 days before expiry, critical 7 days before. Old expired certificates that nothing uses are listed in the report, not raised as findings.
- **patching**: warning when the last update is 35 days old, critical at 60 days.

If something goes wrong

What you see	What to do
<code>winrm or requests is not installed / connection test: no winrm</code>	the EE has no pywinrm: step 2
<code>CERTIFICATE_VERIFY_FAILED</code>	the CA bundle (step 4); <code>ansible_winrm_server_cert_validation: ignore</code> only as a short lab test
<code>the specified credentials were rejected by the server</code>	username format (<code>DOMAIN\user</code> for NTLM), password, or the account is locked
<code>Connection refused / timed out on 5986</code>	the firewall, or no HTTPS listener (step 1)
<code>kerberos: ... kinit or Server not found in Kerberos database</code>	no Kerberos in the EE, or a short host name: use NTLM, or full names
<code>local administrator: NO</code>	add the account to the local Administrators group
<code>the <name> check could not run: ...</code>	the text after it is PowerShell's error. The other checks still ran
<code>skipping: no hosts matched</code>	the template's inventory is not <code>Windows servers</code> , or the Limit matches no server there
<code>... is not digitally signed or running scripts is disabled on this system</code>	a GPO sets the PowerShell execution policy for the machine (for example <code>AllSigned</code>), which overrides what Ansible asks for. Set it to <code>RemoteSigned</code> by GPO for these servers, or sign the scripts with your code-signing certificate

Not in this release (next)

- ACT in the Windows jobs: `use_act` on the Windows health check and troubleshoot (explain, propose fixes for approval), like on Linux. ACT is already on the servers (above).
- Service watch for Windows services and IIS application pools.
- Checks for domain controllers, IIS and SQL Server.

10. Secrets: where they live

Recommendation

Store every secret as an AAP credential, and inject it into jobs as environment variables or files through custom credential types. Nothing secret goes into Git, inventory, job template variables, or surveys.

- AAP encrypts credential values at rest and never shows them again after you save them. People can be allowed to **use** a credential in a job without being able to **see** it.
- If your organization already runs **CyberArk, HashiCorp Vault, or Azure Key Vault**, keep the secrets there and link the AAP credential fields to it with AAP's credential lookup plugins (on the credential's form, the key icon next to a secret field). AAP fetches the value when a job starts; nothing else in this repository changes.
- Use **Ansible Vault** only for playbooks you must also run outside AAP. It is weaker: one vault password decrypts every value, and the encrypted values sit in Git forever (rotating the password does not un-publish old ciphertext).

Where each secret lives

Secret	AAP credential type	Reaches the playbook as
SSH key + sudo for the hosts	built-in Machine	the SSH connection (never visible to tasks)
WinRM login for Windows servers	built-in Machine (user name and password; Kerberos or NTLM)	the WinRM connection (never visible to tasks). Use a separate credential from the SSH one; see WINDOWS.md
Git access for the project	built-in Source Control	used only by the project sync
MySQL root / STIG Manager DB passwords	STIG Manager database (aap/credential_types/stigman_database.yml)	env STIGMAN_MYSQL_ROOT_PASSWORD , STIGMAN_DB_PASSWORD
TLS certificate, key, CA chain	TLS certificate (tls_certificate.yml)	files; env TLS_CERT_FILE , TLS_KEY_FILE , TLS_CA_FILE hold the paths
Docker Hub / quay.io login for hosts	Container registry login (hosts) (registry_login.yml)	env REGISTRY_* , used with podman login --password-stdin
ACT model key	ACT model key (act_model_key.yml)	env GENAI_KEY / ASKSAGE_API_KEY on the controller. Note: when ACT runs on a managed host, the job hands the key to that one ACT process on the task's stdin (never on a command line, so not in ps or the sudo log) and it is gone when the process ends. So the host's administrators and its process auditing can see it while ACT runs: use a key with a spending limit, and rotate it if a host is untrusted
ACT GenAI beta key	ACT GenAI beta key (act_genai_beta_key.yml)	env GENAI_BETA_KEY
Teams webhook URL	Teams webhook (teams_webhook.yml)	env TEAMS_WEBHOOK_URL
ServiceNow API account	ServiceNow API (servicenow_api.yml)	env SN_HOST , SN_USERNAME , SN_PASSWORD (basic auth on the controller; never sent to hosts)
MariaDB monitor account	MariaDB monitor (mariadb_monitor.yml)	env MARIADB_MONITOR_USER / _PASSWORD ; the password goes to the client on stdin and reaches it only as MYSQL_PWD in its own environment (never a command line, never a fact)

Secret	AAP credential type	Reaches the playbook as
STIG Manager API client	STIG Manager API (<code>stigman_api.yml</code>)	env <code>STIGMAN_TOKEN_URL</code> , <code>STIGMAN_CLIENT_ID</code> , <code>STIGMAN_CLIENT_SECRET</code> (token request is <code>no_log</code>)
Java keystore password	Keystore password (<code>keystore_password.yml</code>)	env <code>KEYSTORE_PASSWORD</code> ; the inventory names the variable (<code>password_env</code>), never the value; the check reads it on stdin and <code>keytool</code> takes it with <code>-storepass:env</code> (never a command line)

Create one credential of each type **per environment** (dev / test / prod) so a test job can never use production secrets.

Why environment variables and files, not extra variables

A credential type can inject into `extra_vars`, but then the secret is an ordinary play variable: any `debug` task, any failed task's error message, or `-vvv` output can print it. An environment variable or a file on the controller has to be read on purpose (`lookup('ansible.builtin.env', 'NAME')`), which keeps secrets out of the variable namespace unless a task asks for them. Files are the right choice for certificates and keys: AAP writes them to private temporary files that disappear when the job ends.

Rules for every playbook in this repository

- `no_log: true` on every task that receives, compares, or writes a secret. On a loop, also set `loop_control: label:` to something harmless, or the item is printed.
- `diff: false` on any task that writes secret material (copy/template of a key, a `.cnf` with a password). With AAP's *Show changes* enabled, a diff prints the file's contents.
- Never put a secret on a command line.** Command lines are visible in `ps`, in audit logs (`execve` records), and in verbose job output. Instead:
 - pass it on **stdin**: `podman secret create NAME -, podman login --password-stdin;`
 - or in a **file** the program reads (`--defaults-extra-file` for MySQL clients), created with mode `0600` and removed afterwards;
 - never `mysql -pPassword` or `curl -u user:password`.
 - never through Ansible's `environment:` keyword on a Linux host with `become`: Ansible puts those variables on the `sudo /bin/sh -c '...'` command line, which `ps` shows and `sudo` writes to the host's journal / `/var/log/secure` - `no_log` does not prevent that. Send the secret on the task's `stdin:` to a small `sh` that exports it (see `roles/check_mariadb/tasks/health.yml`).
- On the hosts, secrets are podman secrets**, not environment lines in unit files:
 - mounted as files where the image supports it (`MYSQL_ROOT_PASSWORD_FILE`, `MYSQL_PASSWORD_FILE`), with `mode=0400` and the container user's uid;
 - injected as an environment variable only when the application cannot read a file (`STIGMAN_DB_PASSWORD`): `Secret=NAME,type=env,target=VAR`.

Podman keeps the value out of the unit file and out of `podman inspect` (checked on podman 5.8; verify on your RHEL's podman). It is still readable by root on the host and inside the container, so protect root and

the container.

5. **Private keys on disk:** 0600, owned by root; certificate and CA files 0644.
6. **Don't register command output that can contain secrets** without `no_log`, and never `debug: var=` a registered result from such a task.
7. **Fail closed:** assert a secret is present (with `no_log`) instead of letting a task run with an empty password.
8. **Changing a secret in AAP is not the same as rotating it.** For a database password the value inside MySQL must change too (`ALTER USER`). The deploy role refuses to silently swap a deployed database password; rotation is a separate, deliberate run.

The real trust boundary

AAP will not show a stored secret, but **any playbook it runs can print it**. So anyone who can:

- push to the branch or tag the AAP project uses,
- edit a job template (its playbook, credentials, or verbosity), or
- run a template with *Prompt on launch* for variables that change what the playbook does,

can extract every secret that template uses. Therefore:

- **protect the branch** (required reviews) and pin the project to a **tag**;
- give most people **Execute** on templates, not **Admin**; give only automation owners **Use** on credentials;
- keep *Prompt on launch* for variables off on templates that hold production credentials;
- review the **Activity Stream** (who changed which credential or template) regularly.

Keeping secrets out of Git

- `.gitignore` in this repository blocks common key/cert/vault-password file names.
- `.pre-commit-config.yaml` runs **gitleaks** on each commit (`pre-commit install`).
- The inventory in `inventories/example/` holds placeholders only; the real inventory lives in your work Git or as an AAP inventory.
- If a secret is ever committed: rotate it first, then remove it from history. Removing it from history alone is not enough, because clones and caches keep it.

Rotation checklist

Secret	How
TLS certificate	update the TLS certificate credential, re-run <code>stigman_deploy.yml</code> (nginx restarts)
Registry token	update the credential; the next run logs in with it
STIG Manager DB password	password-rotation run: ALTER USER in MySQL, update the podman secret, restart the API; then update the AAP credential. The playbook for this comes with the day-2 set
SSH key	add the new public key to the hosts, update the Machine credential, remove the old key
ACT / Teams	update the credential; nothing else
ServiceNow API account	change the password in ServiceNow, update the ServiceNow API credential, then run <code>ServiceNow - test ticket</code> (SERVICENOW_SETUP.md) to prove it works
MariaDB monitor account	change it in the database (ALTER USER), update the MariaDB monitor credential, run a health check with <code>health_checks=mariadb</code> (MARIADB.md)
WinRM login	change it in the directory, update the Windows Machine credential, run <code>Windows connection test</code>

11. Settings (variables): what, where, and who wins

A **variable** (also called a *setting*) is a named value the runbooks read: `check_disk_warn_pct: 85` means "warn when a disk is 85 % full". You change a variable to change how a check behaves, without touching any code. This page answers three questions:

1. **What can I set?** (find the name and the default)
2. **Where do I put it?** (for every host, for one group, for one host, for one run)
3. **What happens when the same setting is in two places?** (who wins)

You do not need to read it all. Do the two-minute version first.

The two-minute version

1. **Find the setting.** Open `VARIABLES_REFERENCE.md`, press Ctrl+F and search for a word (`disk`, `certificate`, `mariadb`). Every setting is there with its default.
2. **Put the line in one file.**
 - Every host: `playbooks/group_vars/all.yml`
 - Hosts in one AAP group: `playbooks/group_vars/<group name>.yml`
 - One host: `playbooks/host_vars/<host name>.yml`
3. **Commit and Sync Changes** in VS Code. Run the check.

That is all for a normal change. The rest of this page explains the details and the exceptions.

1. What you can set

Every setting lives, with its default and an explanation, in the role that uses it:

`roles/<role>/defaults/main.yml`. Two easy ways to read them:

- **VARIABLES_REFERENCE.md**, the same information as one searchable page (generated from the role files, so it is never out of date). Start here.
- In VS Code: **Ctrl+Shift+F** (search in all files), type the setting or a word, and look at the hits under `roles\...\defaults\main.yml`.

Setting names start with the thing they belong to:

Starts with	Belongs to	Example
<code>check_disk_</code> , <code>check_services_</code> , <code>check_mariadb_ ...</code>	one health check (<code>roles/check_<name></code>)	<code>check_disk_warn_pct</code>
<code>win_check_</code>	one Windows health check	<code>win_check_disk_warn_pct</code>
<code>site_act_</code>	ACT (GenAI)	<code>site_act_level</code>
<code>site_</code> (other)	reporting and settings	<code>site_fail_on</code>
<code>patch_</code> , <code>win_patch_</code> , <code>automatic_restarts</code>	patching	<code>patch_never_patch_groups</code>
<code>servicenow_</code>	ServiceNow tickets	<code>servicenow_assignment_group</code>
<code>watch_</code>	service watch	<code>watch_containers</code> , <code>watch_discover</code>

Starts with	Belongs to	Example
podman_discover_	finding the podman containers (the containers check and service watch share it)	podman_discover_ignore
poam_	POA&M	poam_stigman_api
stigman_	STIG Manager deployment	stigman_scope

Check names and what each finding means: RUNBOOKS.md.

Three shapes of value

YAML

```
check_disk_warn_pct: 85          # a number
check_fapolicyd_required: true   # true or false (lower case)
check_services_required: [sshd, crond] # a list: [a, b] or one "- item" per line
check_disk_overrides:           # a set of names with values (a "map")
  /var/log/audit: {warn: 70, crit: 85}
```

A list replaces the default list. It does not add to it. If the default is `[sshd, crond, rsyslog]` and you write `[chronyd]`, the hosts now require **only** `chronyd`. Copy the default list from the reference and add your entries to it. (Some settings have a separate `..._extra` version that *adds*, for example `check_services_required_extra`. The reference says which.)

Job variables (per run, not per host)

These choose what a run does. Put them in a job template's **Variables** box (*Extra variables*) or in a survey question, never in a settings file:

Variable	Meaning	Where it appears
target	which hosts or group to run on (default: all)	most playbooks
health_checks	which checks: daily , weekly , all , or a list	Health check
ts_area, ts_service, ...	what to troubleshoot	Troubleshoot
use_act, site_act_level	ask ACT, and what it may do	Health check, Troubleshoot
site_fail_on	which findings fail the job: [critical] or [critical, warning]	any check job

2. Where do I put a setting?

Place	Who gets it	How you change it	Good for
<code>roles/<role>/defaults/main.yml</code>	everyone	do not edit (updates replace it)	the built-in default
<code>playbooks/group_vars/all.yml</code>	every host	VS Code, commit, sync	site-wide values: the LDAP server, the SIEM, thresholds
<code>playbooks/group_vars/<group>.yml</code>	the hosts of that AAP group	VS Code	one kind of server: STIG Manager containers, MariaDB, AAP itself
<code>playbooks/host_vars/<host>.yml</code>	that one host	VS Code (create the folder and file)	one odd server: a bigger disk limit for <code>/var/log/audit</code>
AAP: inventory > Variables	every host	AAP web UI	values you prefer to keep in AAP
AAP: group > Variables	the hosts of that group	AAP web UI	same, per group
AAP: host > Variables	that host	AAP web UI	same, per host
Job template > Variables (<i>Extra variables</i>)	every run of that template	AAP web UI	"this template always uses <code>site_act_level: diagnose</code> "
Survey question / prompt on launch	this launch only	asked at launch	a choice made each time
AAP credential	the job	AAP web UI	secrets only (SECRETS.md)

File names matter. `playbooks/group_vars/<group>.yml` applies to the group with **exactly that name** in your AAP inventory, and `playbooks/host_vars/<host>.yml` to the host with exactly that name. A file for a group your inventory does not have is silently never used.

The files shipped in `playbooks/group_vars/` and the group each expects:

File	Group name it applies to	If your group has another name
<code>all.yml</code>	every host	(always works)
<code>aap.yml</code>	<code>aap</code>	copy it to <code>aap_hosts.yml</code> if your group is <code>aap_hosts</code> (the example inventory's name)
<code>mariadb.yml</code>	<code>mariadb</code>	the MariaDB check also runs for a group named <code>mariadb_hosts</code> , but the settings come from the file named after the group: copy it to <code>mariadb_hosts.yml</code>
<code>sat.yml</code> , <code>idm.yml</code> , <code>logstash.yml</code> , <code>stigman.yml</code> , <code>rhel8_all.yml</code>	the same name	copy or rename
<code>netapp.yml</code>	<code>netapp</code>	copy to <code>netapp_console_hosts.yml</code> if that is your group

(The built-in safety lists cover both spellings of the AAP and NetApp group names, so AAP stays protected whatever you call it.)

Updating the repository never overwrites your settings files. The update script (`scripts/update-from-release.ps1 / .sh`) protects `playbooks/group_vars/`, `playbooks/host_vars/`, `inventories/site/`, `poam/poam.csv` and anything in `.site-local`. If a new release adds a settings file you do not have, the script adds it **once** (the preview lists it as **YOURS**) and never changes it again. A setting that the release's `all.yml` mentions and yours does not is listed under **NEW SETTINGS**, so you can copy it over if you want it.

Adding "extras" to a host or a group in AAP

Where the boxes are in AAP 2.7 (older versions word a label differently):

- **Inventory: Automation Execution > Infrastructure > Inventories** > your inventory > **Edit inventory > Variables**.
- **Group:** the same inventory > **Groups** tab > the group > **Edit group > Variables**.
- **Host:** the same inventory > **Hosts** tab > the host > **Edit host > Variables**.
- **Job template: Automation Execution > Templates** > the template > **Edit template > Extra variables** (the template's *Variables* box).

The **Variables** box on a host or group takes the same YAML as a settings file. Typical entries:

a group's Variables box: hosts in this group need a bigger disk margin

```
check_disk_overrides:
  /var/lib/pgsql: {warn: 70, crit: 85}
```

a host's Variables box: its MariaDB containers belong to a rootless account

```
check_mariadb_container_user: svc_podman
```

a group's Variables box: never check or watch the test containers, or anything of one user

```
podman_discover_ignore: ['test-.*', 'devuser/.*']
```

Connection settings (`ansible_host`, `ansible_user`, `ansible_python_interpreter`, `ansible_connection: winrm`) belong here too. The *login* and *password* never do: they are credentials (SECRETS.md).

Prefer the project's settings files (this repository), because they are reviewed and versioned in Git and updates never overwrite them. Use the AAP boxes when AAP is where your team already manages the inventory. Either is fine; the rule is **one setting, one place**.

3. Who wins when the same setting is in two places?

Strongest first. This order was **proven by running a test playbook** on `ansible-core` 2.16 and 2.21 with the same value set in every place and removing the winner one place at a time (the same result on both versions).

#	Place	Notes
1	Extra variables: job template <i>Variables</i> , a survey answer, a prompt on launch, -e on the command line	beats everything below, including the built-in protection further down
2	<code>playbooks/host_vars/<host>.yml</code>	this project, one host

#	Place	Notes
3	the host's Variables box in AAP	
4	<code>playbooks/group_vars/<group>.yaml</code>	a child group beats its parent group
5	<code>playbooks/group_vars/all.yaml</code>	
6	the group's Variables box in AAP	a deeper group beats its parent
7	the inventory's Variables box in AAP	
8	<code>roles/<role>/defaults/main.yaml</code>	the built-in default

Things that surprise people:

- **A file in this project beats the AAP group and inventory boxes**, even when the box is on a more specific group. If you type a value in an AAP box and see no effect, look for the same setting in `playbooks/group_vars/`. Every job prints a notice, `NOTE - settings in this project's files win over the same setting in AAP's inventory Variables box`, listing which files define which settings. (Silence it with `site_settings_notice: false`.) Not shown for `playbooks/host_vars/`.
- **A host's own Variables box (3) beats a group file (4/5)**, but a `playbooks/host_vars` file (2) beats the host's box.
- **A survey answer or template extra variable beats everything**: it is the one place that can override the rest for a single run.
- Two sibling groups (neither is inside the other) that set the same variable for the same host: do not rely on the result. Put it in a host file, or in one group.
- An inventory **sourced from the project** (`inventories/site/`, SETUP_AAP.md step 5): AAP imports its `group_vars/all.yaml` into the inventory's Variables box and each `group_vars/<group>.yaml` into that group's box. So they rank as places 7 and 6, and the project's `playbooks/` files beat them. Keep a setting in one of the two, not both.
- **Updated from 0.4.1 or older?** Your `playbooks/group_vars/all.yaml` is yours, so the update left it as it was: it still sets `site_act_diagnose_only_groups`, `patch_never_patch_groups`, `patch_never_reboot_groups`, `site_act_provider` and `site_act_models` as real (uncommented) lines. They beat the same settings in AAP's inventory and group boxes. If you want those set in AAP (or the new built-in defaults), put `#` in front of those lines. See the upgrade notes in `CHANGELOG.md` (0.5.0).

Built-in protection: settings that cannot be weakened from a file

Some lists are protected in code, so a mistake in a settings file cannot expose the AAP host:

What	Built in	What your file can do
Never patched, never rebooted	<code>aap, aap_hosts</code> (<code>roles/patch/vars/main.yaml</code>)	your <code>patch_never_patch_groups</code> / <code>patch_never_reboot_groups</code> list replaces the shipped one (<code>sat, netapp, ...</code>): repeat the names you keep. <code>aap</code> and <code>aap_hosts</code> stay protected whatever you write
ACT never fixes by itself	<code>aap, aap_hosts</code> (<code>roles/site_act/vars/main.yaml</code>)	your <code>site_act_diagnose_only_groups</code> list replaces the shipped one (<code>sat, idm, netapp, ...</code>): repeat the names you keep. <code>aap</code> and <code>aap_hosts</code> stay protected whatever you write

Only an extra variable (place 1) can override these, and nothing in these docs asks you to do that.

Examples

Everyone: warn earlier on disks. `playbooks/group_vars/all.yml`:

```
YAML

check_disk_warn_pct: 80
check_disk_crit_pct: 92
```

One group: the STIG Manager hosts need these containers watched.

`playbooks/group_vars/stigman.yml` (shipped; already has `watch_containers`).

One host: a database server whose audit partition must stay emptier.

`playbooks/host_vars/db01.example.mil.yml` (the file name is the host's name in AAP):

```
YAML

check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}
```

One template: the "Troubleshoot" template always uses Ask Sage. In the template's **Variables**:

```
YAML

site_act_provider: asksage
```

A list you want to extend. Where the role has a `..._extra` setting, use it: it *adds*.

```
YAML

check_services_required_extra: [tomcat]           # required on top of the built-in five
```

Where it has none, look up the default (`check_services_required` is `[sshd, crond, rsyslog, chronyd, auditd]`) and write the whole list with your addition:

```
YAML

check_services_required: [sshd, crond, rsyslog, chronyd, auditd, tomcat]
```

How to verify a setting is in effect

1. **Run the check on one host: Health check** with `target` = that host and `health_checks` = the check. The finding text quotes the limit it used (`/var is 96% full (critical at 92%)`), so you see the value that won.
2. **Break it on purpose** (or lower the limit) on a test host, and confirm the finding appears. A check that never fires proves nothing.
3. **Read the job's notice** (NOTE - settings in this project's files ...): it names every settings file that set something, so you can spot a file that overrides an AAP box.
4. **From the command line** (optional, in a terminal with Ansible and the repository): create a temporary file `playbooks/show_settings.yml`, **do not commit it**, and run it:

YAML

```
- name: Show the settings a host gets
hosts: "{{ target | default('all') }}"
gather_facts: false
tasks:
  - ansible.builtin.import_role: {name: check_disk}    # the role that owns the setting
    when: false                                         # loads its defaults, runs nothing
  - ansible.builtin.debug:
      msg: "{{ item }} = {{ lookup('vars', item, default='(not set)') }}"
      loop: [check_disk_warn_pct, check_disk_crit_pct]
```

shell

```
ansible-playbook -i inventories/site/hosts.yml playbooks/show_settings.yml -e
target=db01.example.mil
```

It shows the value from the role default plus the files in this project. (Values typed into AAP's Variables boxes exist only inside AAP; there, use steps 1-3.)

If a setting does nothing

Symptom	Likely cause
You set it in AAP and nothing changes	the same setting is in <code>playbooks/group_vars/</code> : that file wins (table above)
You set it in a file and nothing changes	the file name does not match the group or host name exactly; a spelling or capital letter
A list lost entries	a list replaces the default: repeat the entries you keep
YAML error at the start of the job	indentation (spaces, never tabs), a missing space after <code>:</code> , or a value with <code>:</code> inside it that needs quotes
A survey answer is ignored	the survey question's variable name is not exactly the setting's name

12. Approved commands: what ACT may run by itself

ACT is a tool that runs **on each Linux server** when a job asks it to. This page explains, in plain words, which commands ACT runs by itself, which ones wait for a person, and how you add a command to the "run without asking" list.

Short version. There are three kinds of command:

Kind	Examples	What happens
Reading	<code>systemctl status chronyd, df -h, journalctl -u nginx, ss -tlnp</code>	ACT runs it. It never asks. Nothing on this page limits reading.
Changing , not on your list	<code>systemctl restart nginx, dnf -y install httpd</code>	Refused. The job says NEEDS APPROVAL and shows the exact command. A person approves it in the workflow.
Changing , on your list	the ones you wrote in <code>site_act_allow</code>	ACT runs it by itself (only at level <code>self-heal</code>).

The list is one setting: `site_act_allow`. You add a command by adding one line to it.

Read this first: two safety rules that you cannot switch off

1. **Some commands are never run by a job**, whatever you write. ACT itself never pre-approves a *catastrophic* command (recursive delete, disk or volume destruction, power off, reboot). And the step that applies an approved fix (the *Apply approved ACT fix* job, and the service-watch fix step) has its own last check: reboot, shutdown, `kexec`, `mkfs`, `fdisk`, `usermod`, `passwd`, recursive deletes of system folders and a few more are refused **even after a person approved them** - also with a path in front (`/sbin/reboot`) or inside `bash -c '...' /$( )`. Patching and restarts have their own job: RUNBOOKS.md, section "Patch hosts".
2. **AAP itself is never fixed by a job**. Hosts in the groups `aap` and `aap_hosts` are diagnose-only: ACT reads and proposes, and the apply step prints the command for a person to run by hand. Nothing you set can remove those two groups (they are built into the code).

Everything below works inside those two rules.

How a command becomes "approved": the three ways

Way 1: a person approves it (the workflow), nothing to configure

This is the default at level `diagnose`.

1. The health check finds a problem. ACT investigates by reading, then proposes the fix.
2. Because the fix changes something and is not on any list, ACT is **refused** and the job fails with **NEEDS APPROVAL - ... ACT proposes: `systemctl start chronyd`**.
3. The workflow stops at its **Approval** step. The approver reads the proposal and approves or denies.
4. *Apply approved ACT fix* runs **exactly the approved text**, no model involved, then the checks run again.

What the approver sees is what runs. To set this up, see `ADDING_ACT.md` and `WORKFLOWS_AND_SCHEDULES.md`.

Two details worth knowing:

- Only a **plain single command** can go through this path (see the character rule below). A proposal such as `cd /x && ./fix.sh` or `something | tee file` is listed in the report as a **manual fix**: a person runs it by hand. That is deliberate.
- The approval covers that command on the hosts that proposed it, once. It does not add the command to any list. Next week the same problem needs the same approval.

Way 2: you put it on the list (`site_act_allow`), for fixes you trust

Use this for boring, safe, repeated fixes: start a stopped service, restart a hung one. It only takes effect at level `self-heal`. At `explain` and `diagnose`, the list is ignored.

YAML

```
site_act_allow:
- 'systemctl restart (chronyd|rsyslog|crond)'
- 'systemctl start (chronyd|rsyslog|crond)'
```

Where to put it, step by step, is in the next section.

Way 3: `--allow` on the ACT command line (when you run ACT by hand)

If you run `act` on a server yourself, the same idea is `--allow`:

shell

```
act --allow 'systemctl restart (nginx|httpd)' "the web server is down; find out why and fix it"
```

`site_act_allow` is this, done for you: each list item becomes one `--allow` when the job calls ACT. (Full ACT reference: "Approved commands" in ACT's own README.)

At the prompt that ACT shows when you run it by hand (`run? [y]es / [n]o / [e]dit / [q]uit`) there is **no "always allow" answer**. This is on purpose: the list is the only way to make ACT stop asking, and the list is in Git, where a reviewer can see it.

Add an approved command, step by step

You need: VS Code with this repository, and the command as you would type it on the server.

Step 1: write the command exactly as ACT would run it

Try it by hand on a lab server first: `sudo systemctl restart chronyd`. Note the words in between: `systemctl restart chronyd` (drop `sudo`; ACT already runs as root on the host).

Step 2: turn it into a pattern

A pattern is a *regular expression* that must match the **whole** command, from the first character to the last. `systemctl restart chronyd` on its own is already a valid pattern that matches exactly that command and nothing else. Start there. Widen it only when you have to:

Pattern	Matches	Does not match		
<code>systemctl restart chronyd</code>	<code>systemctl restart chronyd</code>	<code>systemctl restart chronyd.service</code>		
<code>`systemctl restart (chronyd\ rsyslog\ crond)`</code>	<code>rsyslog\</code>	<code>crond)`</code>	<code>systemctl restart rsyslog</code>	<code>systemctl</code>
<code>`systemctl (start\ restart) (chronyd\ rsyslog\ crond)`</code>	<code>restart) (chronyd\ rsyslog\ crond)`</code>	<code>rsyslog)`</code>	<code>systemctl start rsyslog</code>	<code>systemctl</code>

Pattern	Matches	Does not match
<code>systemctl restart chronyd*.service</code>	<code>systemctl restart chronyd.service</code>	<code>systemctl restart chronydXservice</code>
<code>`podman restart (mysql\ nginx)`</code>	<code>nginx`</code>	<code>podman restart nginx</code> <code>podman rm -f nginx</code>

(In the YAML file, write `|` normally. The `\|` above is only how a table shows a bar.)

Rules for the pattern:

- **The whole command must match.** `systemctl restart .*` looks like "restart anything", but it can only match a plain command. It is also far too wide: it would let ACT restart `sshd` and the network. Name the services.
- **Only a plain single command counts.** The command may contain only letters, digits and these characters: `space _ . / : = @ % + , -`. Anything else is never pre-approved, however you write the pattern: no `;` && `||` (chaining), no `|` (pipes), no `>` `<` (redirection), no `$( )` or backticks, no `*` or `?` (globs), no quotes, no `~`, no line breaks. Such a command still needs a person.
- **A dot in a pattern means "any character".** Write `\.` when the exact dot matters, as in `chronyd\*.service`.
- **No file edits.** A pattern covers `run` commands only; ACT changing a file (edit or write) always needs a person.
- **Dangerous commands are never approved by a pattern**, whatever you write: ACT's danger tier (package removal such as `dnf -y remove httpd, usermod -aG wheel alice, chmod 777, podman rm -f mysql, rm -f, ...`) and reboots, power-off and disk tools always need a person (tested with ACT 0.6.21). They come back as **NEEDS APPROVAL** / manual fixes.
- **Never put these on the list** anyway: `.*` on its own, `systemctl (stop|disable|mask) .*`, `kill .*`. Those are ordinary changes a pattern *can* approve, and far too wide: `systemctl stop .*` would let ACT stop `sshd`.

Step 3: put it in the right settings file

`site_act_allow` is a setting like any other (VARIABLES.md explains where settings live). Pick the place that matches who should get it:

Who gets the command	Where to write it
Every host	<code>playbooks/group_vars/all.yml</code> : add the lines at the end of the file (copy the example below).
One group (for example <code>stigman</code>)	<code>playbooks/group_vars/stigman.yml</code> (the file name is the AAP group name)
One host	<code>playbooks/host_vars/<host name as in the inventory>.yml</code> (create the folder and file if they do not exist)
One template, every run	the job template's Variables box (extra variables), YAML: <code>site_act_allow: ['systemctl restart chronyd']</code>

A list replaces, it does not add. If `all.yml` has two commands and `stigman.yml` has one command, the `stigman` hosts get **only that one**. Repeat the ones you keep. (The exact order of who wins is in VARIABLES.md.)

Example, `playbooks/group_vars/all.yml`:

```
YAML
site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Use single quotes. Keep the indentation. Spaces, never tabs.

Step 4: save, commit, sync

In VS Code: save, **Commit**, **Sync Changes**. AAP picks the file up on the next run (the project syncs when *Update revision on launch* is ticked).

Step 5: use the level self-heal

Launch the check with `use_act = yes` and `site_act_level = self-heal` (the survey question on the Health check and Troubleshoot templates; `ADDING_ACT.md`).

How to verify it worked

Do all three. They take ten minutes and catch the usual mistakes.

1. Check the pattern matches (and only what you meant). No server needed. In PowerShell:

```
$p = 'systemctl restart (chronyd|rsyslog|crond)'      # your pattern, without the quotes' worth of
YAML
foreach ($c in 'systemctl restart chronyd',          # should be True
               'systemctl restart sshd',             # should be False
               'systemctl restart chronyd; reboot') { # should be False
    '{0,-40} {1}' -f $c, ($c -cmatch ('^A(?:' + $p + ')\z'))
}
```

Expected output: `True, False, False`. (Verified. The same test in Python, on a machine that has `vendor/act/act`: load it with `importlib` and call `compile_preapproved/preapproved_pattern`; ACT applies exactly `^A(?:your pattern)\z`.) Add a line for every command you want to *stay* refused and check it prints `False`.

2. Check ACT itself agrees, on a lab server. Log in, put your key in the environment the way `SECRETS.md` describes, and run ACT in the same mode a job uses. It will not ask anything; it either runs what you allowed or refuses:

```
shell
sudo systemctl stop chronyd          # break it on purpose (lab server only)
act --non-interactive --allow 'systemctl (start|restart) chronyd' \
  --result-file /tmp/r.json "chronyd is stopped; find out why and fix it"
echo "exit code: $?"                 # 0 = fixed, 4 = something was refused
python3 -m json.tool /tmp/r.json | less # see commands[].approval and denied[]
```

In `/tmp/r.json`: `commands[].approval` says `pre_approved` for what your list allowed (with the `pattern` that matched) and `auto` for reads. `denied[]` lists what was refused, each with `pre_approvable`: `true` means "a pattern could approve this" (add one if you want it); `false` means a person has to do it.

3. Check the job. On the same lab server, launch **Health check** with `use_act = yes`, `site_act_level = self-heal`. In the output:

- ACT | what ACT says shows `APPLIED (pre-approved): systemctl start chronyd;`

- the report afterwards says **cleared**:

Then break something that is **not** on your list and check the job says **NEEDS APPROVAL** and does not touch it. A dry run (job type *Check*) does not call ACT, so it cannot test this; see DRY_RUNS.md.

If it does not work

Where	Why	Fix	
NEEDS APPROVAL for a command and you listed	the pattern does not match the whole command; or a listed command contains a <code>;</code> , <code>\</code>	<code>,</code> quote or glob; or the level is explain/diagnose, not self-heal	run check 1 with the exact command from the job output; fix the pattern the level
command is in manual not proposed	it is not a plain single command	ask ACT for a simpler fix, or apply by hand	
something seems ignored	a more specific place wins (the group or host file, or the template's Variables box)	see the order in VARIABLES.md; the job prints a notice when a settings file overrides the AAP Variables box	
invalid pattern	ACT does not run: the job shows ACT error ... invalid --allow pattern '...' and the reason (for instance an unbalanced bracket)	fix the YAML/regex	
command on an AAP host does not	by design: AAP is diagnose-only	run it by hand from the printed line	
approved command was not at apply time	the apply-time guard (reboot, shutdown, mkfs , usermod , recursive delete of a system folder, ...)	do it by hand if you really mean it	

Reference: what the guard rails are

Where	Rule	Can you change it?
ACT, every command	proven read-only commands run without asking; everything else asks	no (it is the design)
ACT, --allow / site_act_allow	anchored to the whole command; plain characters only; never catastrophic; never a file edit	you choose the patterns
ACT, --auto	runs reads and ordinary changes without asking, still asks for the danger tier. Not used by these jobs	not used
site_act_diagnose_only_groups	hosts where ACT never fixes; the list <i>you</i> write replaces the shipped one (sat , idm , ...), but aap , aap_hosts are always kept	you choose the groups; you cannot remove aap/aap_hosts
Apply-time guard (_apply_refuse , roles/site_findings/vars/main.yml)	last check before an approved command runs	edit the code to change it
Host log	/var/log/site-automation-fix.log on each host lists what the apply job ran, when, and for which AAP job and user	

The apply-time guard applies to the **approval** path (Way 1). Commands run by **self-heal** (Way 2) are run by ACT itself, so what protects you there is your list. Keep it short and boring.

Windows

`site_act_allow` is read by the Linux jobs. Windows servers use the approval path only (ACT proposes, a person approves); see WINDOWS.md. If you run ACT for Windows by hand, its `-Allow` option follows the same rules as Linux `--allow`:

	Linux ACT (--allow)	Windows ACT (-Allow)
A pattern that names a danger-tier command (for example <code>dnf -y remove httpd, podman rm -f mysql</code>)	does not approve it	does not approve it
A danger-tier command in an unattended run	always refused ; only a person at an interactive prompt can approve it	the same
Catastrophic commands	never	never
<code>git</code> , cmd built-ins, aliases such as <code>ls</code> and <code>dir</code> in an unattended run	follow the rules above	still ask , because they were never on Windows ACT's unattended list
Result file (<code>--result-file / -ResultFile</code>)	written (since 0.6.16)	written (since 0.6.16; its <code>tokens</code> field is filled since 0.6.21)

(ACT 0.6.20 for Linux, released for a few minutes on 2026-09-30, let a pattern approve a danger-tier command; 0.6.21, the version in this project, does not. The Windows rows are taken from ACT-Windows 0.6.21 and its self-tests and were not run by hand on a Windows server.)

13. The MariaDB and MySQL check, step by step

There are **two ways to run the database check**, and both check **MariaDB and MySQL** (MySQL 5.7, 8.0 and 8.4, MariaDB 10.x and 11.x, and Percona), installed on the host or **in podman containers** (one, several, found automatically, rootless):

How	Job template	Use it when
Its own template	Database health - MariaDB / MySQL (<code>playbooks/database_health.yml</code> ; <code>SETUP_AAP.md</code>)	the database team wants its own job, schedule and tickets. Runs only this check, on every database server it is pointed at
Part of the Health check	Health check with <code>health_checks = mariadb, mysql</code> or <code>database</code> (all three are the same check; <code>daily</code> includes it)	you already run the Health check and want the databases in it

This page shows how to turn the check on, how to give it a login, and how to see that it works. Nothing here needs you to know Ansible. It changes nothing on the database: the check only asks status questions and never reads your data.

Words used here:

- **container name** - the name podman gives a container. You read it with `podman ps -a`.
- **rootless** - the container belongs to an ordinary user (not root). Only that user can see it.
- **settings file** - a small file in `playbooks/group_vars/` that holds your choices for one group of hosts. See `VARIABLES.md`.

The short version

1. On the database host, find the container name: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`).
2. Put the name in the settings file for your MariaDB group (section 3).
3. Ask the DBA for a read-only monitoring account and put it in an AAP credential (section 4).
4. Run the **Database health** job (or the *Health check* with `health_checks = mariadb`) and read "How to verify" below.

1. Which hosts the check runs on

The check runs on a host when **any** of these is true:

- the host is in one of the inventory groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql` (any of the five works, for either engine), or
- the host has a container name set (section 2), or
- you set `check_mariadb_enabled: true` for the host or its group.

The **Database health** template does not need this: every server it runs on is checked. It runs on the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` and `mysql` (since 0.6.0; before that only the first three). A **Limit** only narrows that list; it cannot add a server that is in none of them. For a server outside those groups, put `target: <group or host>` in the template's **Variables** box, or add the server to one of those groups. If `target` matches no host, the job stops with a message that lists the groups your inventory does have. On every other host, in the Health check, the job prints `skipped: the MariaDB/MySQL check is`

off for `<host>` and moves on. That is normal; it is not an error.

2. Tell it where MariaDB is

Pick **one** of the four ways. Put the lines in the settings file (section 3).

Your situation	Setting
MariaDB or MySQL is installed on the host itself (a service)	nothing to set. The check finds the service <code>mariadb</code> , <code>mysqld</code> or <code>mysql</code> and tells the engine and version apart by itself. To name the service: <code>check_mariadb_service: mariadb</code>
One container	<code>check_mariadb_container: snow-mariadb</code>
Several containers on the same host	<code>check_mariadb_containers: [snow-mariadb, other-db]</code>
You do not want to list them: find them	<code>check_mariadb_container_discover: true</code>

Details that matter:

- **Discovery** looks at every container on the host, running or not, and picks the ones whose **image name** matches `check_mariadb_container_image_regex` (default `(mariadb|mysql|percona)`). If your image has another name, change the pattern, for example `'(mariadb|snowdb)'`. Discovery that finds nothing is reported as a warning `mariadb:no-container`, so a wrong pattern is not silent.
- With **several** instances on one host, each is checked on its own. Their findings start with the container name, for example `[snow-mariadb] 143 of 151 connections in use`.
- The check runs the database's own client **inside** the container (`podman exec`), so you do not need a client on the host and you do not open any network port.
- The engine (MariaDB or MySQL) and its version are read from the server, so the same settings work for both.
- A container that is named but does not exist is reported as `mariadb:down` ("MariaDB/MySQL is not running (container ... is missing)"), not skipped.

Rootless containers

When the containers belong to a user, name that user:

```
YAML
check_mariadb_container_user: appuser
```

The check then runs `podman as that user` (the job needs privilege escalation, which the templates in `SETUP_AAP.md` already have). If that user does not exist on the host you get the critical finding `mariadb:user`, with the hint `getent passwd <user>`. Leave the setting empty for containers that belong to root.

3. Where to put the settings

Open `playbooks/group_vars/mariadb.yml` (MySQL: `playbooks/group_vars/mysql.yml`) in VS Code. It already holds the lines below as comments. Remove the `#` in front of the ones you need and replace **CHANGE-ME**:

YAML

```
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
# check_mariadb_container_user: appuser      # only for rootless containers
```

A settings file applies to the group with the same name as the file. `mariadb.yml` is for the group `mariadb`, `mysql.yml` for `mysql`. If your hosts are in a group with another name (the example inventory uses `mariadb_hosts` and `mysql_hosts`), copy the file to `playbooks/group_vars/<your group name>.yml`. The check accepts all five group names, but the *settings* are only read from the file that matches the group. (The **Database health** template runs on the `_hosts` groups only: for a group called `mariadb` or `mysql`, give it a `target`, see section 1.) One host that differs from the rest: `playbooks/host_vars/<host name>.yml` (create it). Which place wins, and how to set the same thing in AAP's own Variables boxes, is in `VARIABLES.md`.

Then commit, push, and run **Sync** on the project in AAP.

4. The login: the "MariaDB monitor" credential

Where the database runs	What to do
MariaDB installed on the host	Usually nothing. Run with privilege escalation and the check logs in as root through the local socket (RHEL's default).
MySQL installed on the host	Usually the credential: MySQL's root account normally has a password.
In a container (either engine)	Use the credential. The official images give root a password, and without it the job reports a finding that contains <code>Access denied</code> and ends with <code>attach the "MariaDB monitor" credential</code> .

Steps:

1. Ask the DBA to run this **inside that database** (the account needs no data access). The privileges differ by engine:

SQL

```
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';

-- MySQL 5.7 / 8.0 / 8.4:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
-- MySQL Group Replication only, also:
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';

-- MariaDB 10.5.9 and newer:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
-- MariaDB before 10.5.9:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
```

On MariaDB 10.5.9 and newer, `REPLICATION CLIENT` is not enough: it no longer covers the replication status, so the account needs `REPLICATION MONITOR`. `PROCESS` lets the check see other sessions (for long queries). Without the replication privilege the check still works and reports `mariadb:replication-query` (below) with the grant to ask for. The host is `localhost` because the client runs inside the container.

2. In AAP create a credential of type **MariaDB monitor** (the type is in `aap/credential_types/mariadb_monitor.yml`; `SETUP_AAP.md` shows how to load it). Type the account name and password into the credential. **Never** put the password in a settings file, the inventory or `Git: SECRETS.md`.

3. Attach the credential to the **Database health**, **Health check** and **Troubleshoot** job templates.

Credentials attach to templates, not to groups; the check only uses it on hosts that have a database.

The password reaches the client only through its environment (`MYSQL_PWD`, and `podman exec -e MYSQL_PWD` for containers), never on a command line. If your client needs other options, for example a socket path or a TCP address, use `check_mariadb_client_args`: `['--socket=/var/lib/mysql/mysql.sock']`.

5. What it checks, and the limits you can change

Replication is read with the statement that fits the engine: `SHOW ALL SLAVES STATUS` (MariaDB), `SHOW REPLICA STATUS` (MySQL 8.0.22 and newer, 8.4) or `SHOW SLAVE STATUS` (older MySQL). The job's status line names the engine, for example `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections, ...`

Each limit is a setting. The defaults are in the table; change one the same way as in section 3.

Finding	Turns on when	Setting (default)
<code>mariadb:down</code>	the service or container is not running, or the named container does not exist	-
<code>mariadb:connect</code>	it runs but the status query failed (login, hang)	-
<code>mariadb:restarted</code>	restarted less than this many minutes ago	<code>check_mariadb_restart_warn_minutes</code> (60)
<code>mariadb:connections</code>	connections in use, percent of <code>max_connections</code>	<code>check_mariadb_conn_warn_pct</code> (80), <code>check_mariadb_conn_crit_pct</code> (95)
<code>mariadb:refused</code>	clients were refused because the limit was reached	-
<code>mariadb:threads-running</code>	queries executing at this moment	<code>check_mariadb_threads_running_warn</code> (32)
<code>mariadb:long-queries</code>	a query ran longer than this many seconds (never the query text)	<code>check_mariadb_long_query_seconds</code> (300)
<code>mariadb:bufferpool</code>	percent of reads served from memory (judged after 1 million reads)	<code>check_mariadb_bufferpool_hit_warn</code> (95)
<code>mariadb:replica:<name>, mariadb:replica-lag:<name></code>	replication stopped, or behind by seconds	<code>check_mariadb_replica_lag_warn</code> (300), <code>check_mariadb_replica_lag_crit</code> (1800)
<code>mariadb:replication-query</code>	the replication status could not be read (usually a missing privilege; the finding names the grant)	section 4
<code>mariadb:galera, mariadb:galera-size</code>	Galera / XtraDB Cluster node not healthy; cluster smaller than expected	<code>check_mariadb_galera_size</code> (0 = not Galera; set 3 for three nodes)

Finding	Turns on when	Setting (default)
<code>mariadb:group</code> , <code>mariadb:group-size</code>	MySQL Group Replication: a member is not ONLINE (or none is listed); fewer ONLINE members than expected	<code>check_mariadb_group_size</code> (0 = do not check the size; set 3 for three members)
<code>mariadb:group-query</code>	the Group Replication members could not be read (needs <code>SELECT</code> on <code>performance_schema</code>)	section 4
<code>mariadb:errorlog</code>	<code>[ERROR]</code> lines in its log this recent (days)	<code>check_mariadb_error_since_days</code> (1)
<code>mariadb:disk</code>	the disk holding the data directory is this full (percent)	<code>check_mariadb_disk_warn_pct</code> (85), <code>check_mariadb_disk_crit_pct</code> (95)
<code>mariadb:user</code>	<code>check_mariadb_container_use</code> names a user that does not exist	-
<code>mariadb:no-container</code>	discovery found no container with a matching image	<code>check_mariadb_container_image_regex</code>

The finding ids keep the `mariadb:` prefix for MySQL too, so existing tickets and filters still match. What each finding means and the usual cause are in `RUNBOOKS.md`. Every setting with its default is in `VARIABLES_REFERENCE.md`. With several containers on one host, the finding id includes the container name, for example `mariadb:snow-mariadb:connections`.

How to verify

1. **The names are right.** On the database host: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`). The name you put in the settings file is in the `NAMES` column.
2. **The account works.** On the host, run the client inside the container as the monitoring account (it asks for the password; type it here, not in a file):

```
shell
sudo podman exec -it snow-mariadb mariadb --user=aap_monitor -p -e "SHOW GLOBAL STATUS LIKE 'Uptime'"
```

A number back means the login and the `PROCESS` rights work. (Some images have `mysql` instead of `mariadb`.)

3. **Run the job.** In AAP launch **Database health - MariaDB / MySQL** (first as *Check*, then as *Run*; `SETUP_AAP.md`), or the **Health check** with `health_checks = mysql` (or `mariadb`, or `database`) and a Limit of the database host. In the output, find the task `MariaDB | status`. It prints one line per instance, like `MariaDB 11.8.9 - container snow-mariadb (podman, image ...): up 312.4 h, 23/151 connections, 1 running, 0 long queries, replication connections 0`. That line proves the check reached the database. Each host then prints `HEALTHY` or its findings, and the job ends with `N healthy, M with findings`.
4. **No findings** means the database is healthy by these limits. To see a finding on purpose, set a limit that is sure to trip, for example `check_mariadb_conn_warn_pct: 1`, run again, look for `mariadb:connections`, then remove the line.

What was tested, and what was not

Tested against real containers (throwaway rootless podman)	Tested only against a fake podman
MySQL 8.0.46, MySQL 8.4.11, MariaDB 11.8.9: the engine and version are recognised; a replication source that never connects is reported as broken; the error log is read; a login with no credential gives <code>mariadb:connect</code> with the grant to ask for; the <code>REPLICA MONITOR</code> grant and revoke on MariaDB behave as described above	MySQL 5.7, and MySQL Group Replication (<code>mariadb:group, group-size, group-query</code>). They follow the documented statements but have not met a real server.

The fake-podman test (`bash tests/mariadb/run_mariadb_test.sh`) covers one container, several, discovery, a stopped and a missing container, a rootless user that does not exist, MySQL 5.7, 8.0 and 8.4, replication, Group Replication, a denied grant, and that the password never appears on a command line. Nothing here has been run against **your** database: step 3 is the real proof.

If it does not work

You see	Cause	Fix
<code>skipped: the MariaDB/MySQL check is off for <host></code> (Health check only)	the host is in none of the five groups and no container is named	section 1, or use the Database health template
<code>mariadb:replication-query</code>	the account lacks the replication privilege (MariaDB 10.5.9+: <code>REPLICA MONITOR</code> , not <code>REPLICATION CLIENT</code>)	section 4
<code>mariadb:no-container</code>	discovery matched nothing, or podman is run as the wrong user	<code>podman ps -a</code> as the right user; set <code>check_mariadb_container_image_regex</code> or <code>check_mariadb_container_user</code>
<code>mariadb:user</code>	the rootless user name is wrong	<code>getent passwd <user></code> on the host
<code>status query failed: ... Access denied ...</code> and attach the "MariaDB monitor" credential	container root has a password	section 4
<code>... is not running (container ... is missing)</code> but it is running	the name is wrong, or the containers belong to another user	compare with the <code>podman ps -a</code> output; <code>check_mariadb_container_user</code>
The check runs on the wrong host, or not at all after a rename	a settings file only applies to the group named like the file	section 3
Database health says <code>skipping: no hosts matched (0.5.0)</code> , or <code>target '...' matches no host</code>	the server is in none of <code>mariadb_hosts</code> , <code>mysql_hosts</code> , <code>database_hosts</code> , <code>mariadb</code> , <code>mysql</code> (a Limit cannot add it)	section 1: <code>target</code> in the template's Variables, or add it to one of those groups
Your setting has no effect	another place wins	VARIABLES.md

Appendix: credential types to paste into AAP

Each block below is one file in `aap/credential_types/`. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Paste the part under `inputs:` into **Input configuration** and the part under `injectors:` into **Injector configuration** (without those two top-level lines). Copy from the repository file where you can; PDF copy can change spaces.

ServiceNow API (`servicenow_api.yml`)

Used by playbooks/`servicenow_tickets.yml`, `servicenow_health.yml` and `servicenow_test_ticket.yml` (REST Table API, basic auth only). The same variable names as the certified `servicenow.itsm` collection, so this credential also works if you move to that collection later.

YAML

```
inputs:
  fields:
    - id: instance
      type: string
      label: Instance URL
      help_text: e.g. https://servicenow.example.mil
    - id: username
      type: string
      label: API user name
    - id: password
      type: string
      label: API password
      secret: true
  required: [instance, username, password]
injectors:
  env:
    SN_HOST: "{{ instance }}"
    SN_USERNAME: "{{ username }}"
    SN_PASSWORD: "{{ password }}"
```

MariaDB monitor (`mariadb_monitor.yml`)

For the mariadb health check. Needed when MariaDB runs in a container (the official images give root a password); optional on a host install, where the check can log in as root through the local socket. The password reaches the client only through its environment (`MYSQL_PWD`), never a command line. The account needs no data access at all (`docs/MARIADB.md` section 4 has the GRANT).

YAML

```
inputs:
  fields:
    - id: username
      type: string
      label: Monitor account
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MARIADB_MONITOR_USER: "{{ username }}"
    MARIADB_MONITOR_PASSWORD: "{{ password }}"
```

Keystore password (keystore_password.yml)

Optional, for the certs check on a PKCS12 keystore (JKS keystores list without a password). In the inventory, name the variable, never the password: check_certs_keystores: [{path: /path/to/keystore, password_env: KEYSTORE_PASSWORD}]

```
YAML
inputs:
  fields:
    - id: password
      type: string
      label: Keystore password
      secret: true
      required: [password]
  injectors:
    env:
      KEYSTORE_PASSWORD: "{{ password }}"
```

STIG Manager API (stigman_api.yml)

Optional, for the STIG Manager cross-check in playbooks/poam_status.yml. The client is created by your Keycloak realm owner (client-credentials grant, scope stig-manager:collection:read, and a grant in the collections to read) - this repository never changes the realm.

```
YAML
inputs:
  fields:
    - id: token_url
      type: string
      label: Token URL
      help_text: https://keycloak.example.mil/realms/<realm>/protocol/openid-connect/token
    - id: client_id
      type: string
      label: Client ID
    - id: client_secret
      type: string
      label: Client secret
      secret: true
      required: [token_url, client_id, client_secret]
  injectors:
    env:
      STIGMAN_TOKEN_URL: "{{ token_url }}"
      STIGMAN_CLIENT_ID: "{{ client_id }}"
      STIGMAN_CLIENT_SECRET: "{{ client_secret }}"
```

ACT model key (act_model_key.yml)

```
YAML
inputs:
  fields:
    - id: genai_key
      type: string
      label: GenAI API key
      secret: true
    - id: asksage_key
      type: string
      label: AskSage API key
      secret: true
injectors:
  env:
    GENAI_KEY: "{{ genai_key }}"
    ASKSAGE_API_KEY: "{{ asksage_key }}"
```

ACT GenAI beta key (act_genai_beta_key.yml)

Only for ACT with site_act_provider=genai-beta (the GenAI.mil beta proxy). A separate type, so an "ACT model key" type that is already in use does not have to change (AAP refuses to change the fields of a credential type that credentials use). Attach it next to "ACT model key".

```
YAML
inputs:
  fields:
    - id: genai_beta_key
      type: string
      label: GenAI beta API key
      secret: true
      required: [genai_beta_key]
injectors:
  env:
    GENAI_BETA_KEY: "{{ genai_beta_key }}"
```

STIG Manager database (stigman_database.yml)

```
YAML
inputs:
  fields:
    - id: mysql_root_password
      type: string
      label: MySQL root password
      secret: true
      help_text: Used only when the database is first created (MYSQL_ROOT_PASSWORD_FILE).
    - id: stigman_db_password
      type: string
      label: STIG Manager database user password
      secret: true
      required: [mysql_root_password, stigman_db_password]
injectors:
  env:
    STIGMAN_MYSQL_ROOT_PASSWORD: "{{ mysql_root_password }}"
    STIGMAN_DB_PASSWORD: "{{ stigman_db_password }}"
```

TLS certificate (tls_certificate.yml)

AAP writes each value to a private temporary file inside the job's execution environment and puts the file's PATH in the environment variable; the playbook copies the files to the host.

```
YAML
inputs:
  fields:
    - id: certificate
      type: string
      label: Certificate (PEM; server certificate first, then intermediates)
      multiline: true
    - id: private_key
      type: string
      label: Private key (PEM)
      multiline: true
      secret: true
    - id: ca_chain
      type: string
      label: CA chain (PEM) - also used to trust your Keycloak/other internal services
      multiline: true
  required: [certificate, private_key]
injectors:
  file:
    template.cert: "{{ certificate }}"
    template.key: "{{ private_key }}"
    template.ca: "{{ ca_chain }}"
  env:
    TLS_CERT_FILE: "{{ tower.filename.cert }}"
    TLS_KEY_FILE: "{{ tower.filename.key }}"
    TLS_CA_FILE: "{{ tower.filename.ca }}"
```

Container registry login (hosts) (registry_login.yml)

For podman on the MANAGED HOSTS (the built-in "Container Registry" credential type is for pulling execution-environment images on AAP itself, not for the hosts).

```
YAML
inputs:
  fields:
    - id: registry
      type: string
      label: Registry (e.g. docker.io, quay.io)
    - id: username
      type: string
      label: Username
    - id: password
      type: string
      label: Password or access token
      secret: true
  required: [registry, username, password]
injectors:
  env:
    REGISTRY_HOST: "{{ registry }}"
    REGISTRY_USERNAME: "{{ username }}"
    REGISTRY_PASSWORD: "{{ password }}"
```

Teams webhook (teams_webhook.yml)

YAML

```
inputs:
  fields:
    - id: webhook_url
      type: string
      label: Teams Workflows webhook URL
      secret: true
      required: [webhook_url]
injectors:
  env:
    TEAMS_WEBHOOK_URL: "{{ webhook_url }}"
```